



Last updated on **Apr 9, 2026**

Data warehousing on Databricks

Databricks SQL is a cloud data warehouse built on lakehouse architecture. It runs directly on your data lake, supports ANSI SQL with Delta Lake extensions, and provides the tools to build highly performant, cost-effective data warehouses without moving your data.

Interfaces and tools

Databricks SQL runs on [SQL warehouses](#) and is accessible from multiple interfaces for querying, visualization, pipeline management, and automation.

SQL editor

Write and run SQL queries with integrated AI assistance, code comments, and version history.

Notebooks

Attach a notebook to a SQL warehouse to run SQL alongside Python, Scala, or R. See [Notebooks and SQL warehouses](#) for limitations.

Jobs

Schedule SQL queries as jobs for automated data processing and reporting workflows.

Dashboards

Create interactive AI/BI dashboards with AI-assisted authoring to share insights across your organization.

Metric views

 Ask Genie

Define business metrics with consistent calculations using a semantic layer. Reuse metrics across queries and dashboards.

Alerts

Monitor query results, evaluate conditions, and deliver notifications automatically.



ETL

Define and refresh streaming tables and materialized views directly in Databricks SQL for incremental ETL pipelines.

REST API

Automate and manage Databricks SQL objects programmatically.

Monitor and optimize

Query history

Review past query runs, execution times, and resource usage across your warehouse.

Query profile

Inspect the execution plan for a query to identify bottlenecks and optimization opportunities.

Query performance insights

Get automatic insights and recommendations when queries run inefficiently.

Get started

If you're new to Databricks SQL, start with the concepts and then follow a hands-on walkthrough.

Databricks SQL concepts

 Ask Genie

Learn core concepts including queries, SQL warehouses, dashboards, and data management.

Data warehousing architecture

Understand lakehouse architecture, medallion layers, and data modeling approaches for Databricks SQL.

Get started with data warehousing

Follow a complete walkthrough covering sample dashboards, notebooks, jobs, data ingestion, and SQL warehouse setup.

Unity Catalog metric views

Define consistent, reusable business metrics with a semantic layer for use across queries and dashboards.

Create an AI/BI dashboard

Build and publish your first dashboard with datasets, visualizations, and filters using AI-assisted authoring.

Reference

- [SQL language reference](#)
- [Connect to a SQL warehouse](#)
- [Databricks SQL release notes](#)

Is this page

as this page helpful?

☐ Yes

☐ No

☐ Send feedback

 Ask Genie

Last updated on **Mar 23, 2026**

Get started with data warehousing using Databricks SQL

If you're a data analyst who works primarily with SQL queries and your favorite BI tools, Databricks SQL provides an intuitive environment for running ad-hoc queries and creating dashboards on data stored in your data lake.

Learn the basics

Step	Description
Basic Databricks SQL concepts	Familiarize yourself with the core interfaces, query concepts, data management tools, and authentication model in Databricks SQL.
Data warehousing architecture	Understand lakehouse architecture, medallion layers, and data modeling approaches for building a data warehouse.

Build your first assets

Step	Description
Explore sample dashboards	Import and interact with dashboards from the Dashboard Samples Gallery that visualize queries on sample data.
 Ask Genie	

Step	Description
Create an AI/BI dashboard	Build a dashboard to explore data and share insights with your team using AI-assisted authoring and an enhanced visualization library.
Create a SQL warehouse	Provision a SQL warehouse to start running queries on your data.

Connect and automate

Step	Description
Use Databricks SQL with a notebook	Attach a notebook to a SQL warehouse to run SQL alongside Python, Scala, or R.
Load data with COPY INTO	Use COPY INTO to ingest data from cloud object storage into your Databricks SQL tables.
Use Databricks SQL in a job	Create, schedule, and monitor workflows that include Databricks SQL queries, dashboards, and alerts.
Connect BI tools with Partner Connect	Connect your Databricks workspace to a BI and visualization partner solution.

Is this page

as this page helpful?

☐ Yes

☐ No

☐ Send feedback

 Ask Genie

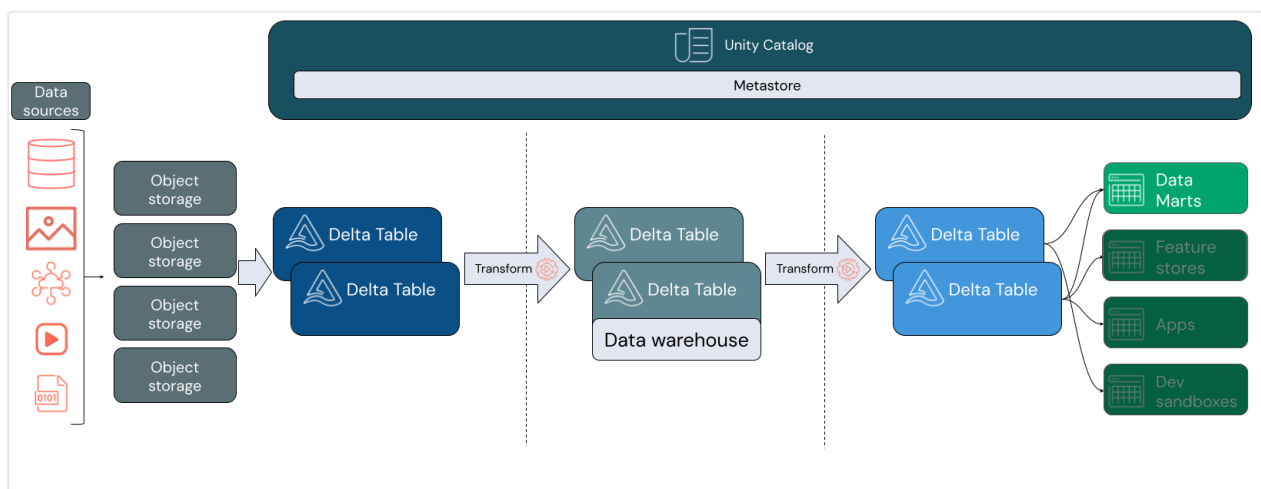
Last updated on **May 7, 2026**

Data warehousing architecture

Data warehousing refers to collecting and storing data from multiple sources so it can be quickly accessed for business insights and reporting. This page contains key concepts for building a data warehouse in your data lakehouse.

Data warehousing in your lakehouse

The lakehouse architecture and Databricks SQL bring cloud data warehousing capabilities to your data lakes. Using familiar data structures, relations, and management tools, you can model a highly-performant, cost-effective data warehouse that runs directly on your data lake. For more information, see [What is a lakehouse?](#)



As with a traditional data warehouse, you model data according to business requirements and then serve it to your end users for analytics and reports. Unlike a traditional data warehouse, you can avoid siloing your business analytics data or creating redundant copies that quickly become stale.

Building a data warehouse inside your lakehouse allows you to bring all your data into a single system and take advantage of features such as Unity Catalog and

[Unity Catalog](#) adds a unified governance model so that you can secure and audit data access and provide lineage information on downstream tables. [Delta Lake](#) adds ACID transactions and schema evolution, among other powerful tools for keeping your data reliable, scalable, and high-quality.

What is Databricks SQL?

Databricks SQL is the collection of services that bring data warehousing capabilities and performance to your existing data lakes. Databricks SQL supports open formats and standard ANSI SQL. An in-platform SQL editor and dashboarding tools allow team members to collaborate with other Databricks users directly in the workspace.

Databricks SQL also integrates with a variety of tools so that analysts can author queries and dashboards in their favorite environments without adjusting to a new platform.

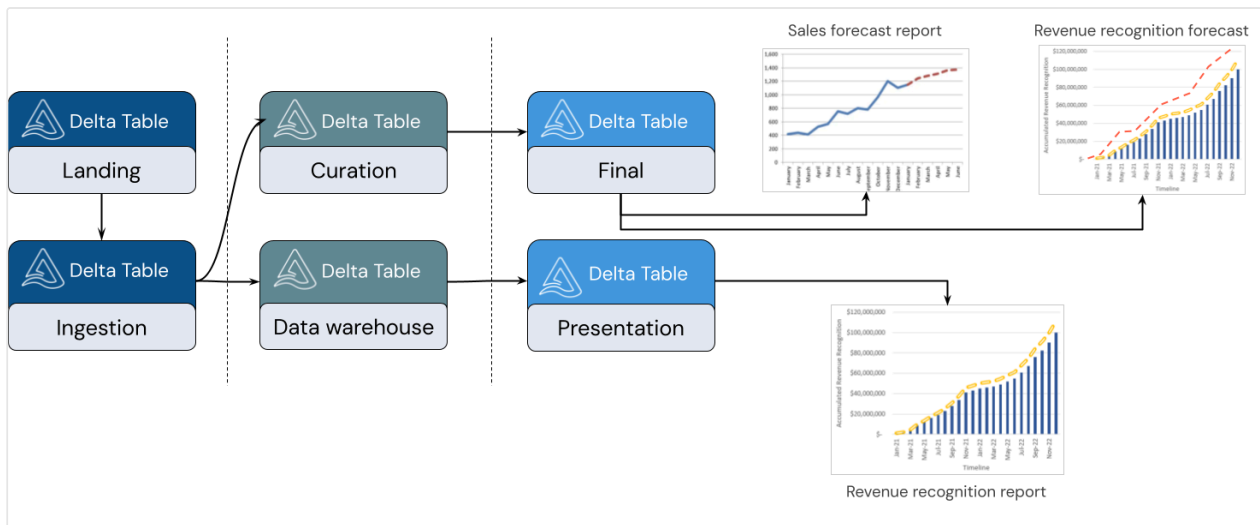
Databricks SQL provides general compute resources that are executed against the tables in the lakehouse. Databricks SQL is powered by [SQL warehouses](#), formerly called SQL endpoints, offering scalable SQL compute resources decoupled from storage.

See [SQL warehouses](#) for more information on SQL Warehouse defaults and options.

Databricks SQL integrates with Unity Catalog, allowing you to discover, audit, and govern data assets from one place. To learn more, see [What is Unity Catalog?](#)

Data modeling on Databricks

A lakehouse supports a variety of modeling styles. The following image shows how data is curated and modeled as it moves through different layers of a lakehouse.



Medallion architecture

The medallion architecture is a data design pattern that describes a series of incrementally refined data layers that provide a basic structure in the lakehouse. The bronze, silver, and gold layers signify increasing levels of data quality, with gold representing the highest quality. For more information, see [What is the medallion lakehouse architecture?](#)

Inside a lakehouse, each layer can contain one or more tables. The data warehouse is modeled in the silver layer and feeds specialized data marts in the gold layer.

Bronze layer

Data can enter your lakehouse in any format and through any combination of batch or streaming transactions. The bronze layer provides the landing space for all of your raw data in its original format. That data is converted to Delta tables.

Silver layer

The silver layer brings the data from different sources together. For the part of the business that focuses on data science and machine learning applications, this is where you start to curate meaningful data assets. This process is often marked by a focus on speed and agility.

The silver layer is also where you can carefully integrate data from disparate sources to build a data warehouse in alignment with your existing business processes. Often,

this data follows a Third Normal Form (3NF) or Data Vault model. Specifying primary and foreign key constraints allows end users to understand table relationships when using Unity Catalog. Your data warehouse should serve as the single source of truth for your data marts.

The data warehouse itself is schema-on-write and atomic. It is optimized for change, so you can quickly modify the data warehouse to match your current needs when your business processes change or evolve.

Gold layer

The gold layer is the presentation layer, which can contain one or more data marts. Frequently, data marts are dimensional models in the form of a set of related tables that capture a specific business perspective.

The gold layer also houses departmental and data science sandboxes to enable self-service analytics and data science across the enterprise. Providing these sandboxes and their own separate compute clusters prevents the Business teams from creating copies of data outside the lakehouse.

Next steps

To learn more about the principles and best practices for implementing and operating a lakehouse using **Databricks**, see [Lakehouse architecture](#).

Is this page

as this page helpful?

☐ Yes

☐ No

☐ Send feedback

 Ask Genie

Last updated on **Apr 27, 2026**

Databricks SQL concepts

This page introduces the core concepts you need to use Databricks SQL effectively.

Interfaces and tools

Interface	Description
SQL editor	A browser-based interface for writing and running SQL queries. Includes integrated AI assistance, code comments, and version history.
AI/BI	A business intelligence solution that includes dashboards and Genie Spaces for self-service data analysis and conversational data exploration.
Notebooks	Interactive documents that combine SQL with Python, Scala, or R. Attach a notebook to a SQL warehouse to run SQL alongside other languages.
Metric views	A semantic layer for defining reusable business metrics with consistent calculations across queries and dashboards.
Alerts	Automated monitors that evaluate query results against a condition and deliver notifications when the condition is met.
Jobs	Scheduled execution of SQL queries for automated data processing and reporting workflows.
ETL	Define and refresh streaming tables and materialized views in Databricks SQL for incremental ETL pipelines.

 Ask Genie

Interface	Description
REST API	A programmatic interface for automating and managing Databricks SQL objects.

Queries and computation

Concept	Description
Query	A SQL statement that retrieves or transforms data. Queries in Databricks SQL are saved, versioned, and reusable.
SQL warehouse	The compute resource that executes SQL queries. All Databricks SQL interfaces run queries on a SQL warehouse.
Query history	A record of executed queries including execution time, resource usage, and performance characteristics.
Query profile	A detailed view of a query's execution plan. Use it to identify bottlenecks and optimization opportunities.
Visualization	A graphical representation of query results, such as a bar chart, line chart, or table.

Data management

Concept	Description
Dashboard	A collection of visualizations and commentary built from one or more queries. Dashboards are shareable and support interactive filtering.

Concept	Description
Metric view	A named, reusable definition of a business metric backed by a SQL expression. Enforces consistent calculations across teams and tools.
Streaming table	A table that ingests data incrementally from a streaming source, managed by a Databricks SQL pipeline.
Materialized view	A precomputed view whose results are stored and refreshed automatically by a Databricks SQL pipeline.

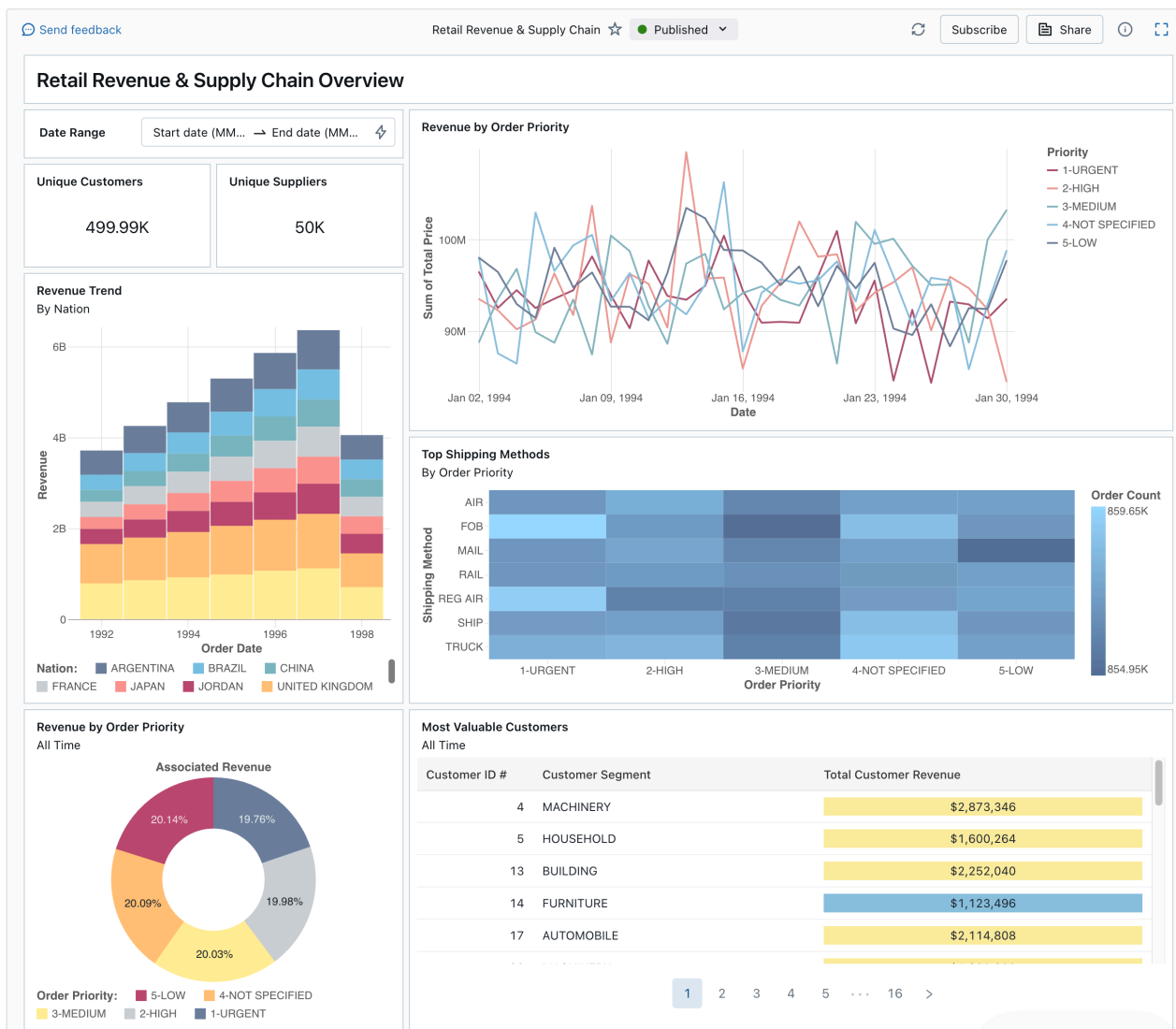
Authentication and authorization

Concept	Description
User and group	A user is a unique individual with access to the Databricks workspace. A group is a collection of users managed together for access control purposes.
Personal access token	A credential used to authenticate to the REST API and to connect third-party tools to SQL warehouses.
Access control list	A set of permissions attached to an object that controls which principals can perform which actions on it.
Unity Catalog	The governance layer for Databricks. Provides centralized access control, auditing, lineage, and data discovery across workspaces.

Last updated on **Apr 29, 2026**

Tutorial: Use sample dashboards

This tutorial shows you how to import and use sample dashboards from the samples gallery. These dashboards illustrate some of the rich visualizations you can use to gain insights from your data. No setup is required. These dashboards use data already available in your workspace and rely on a compute resource (called a SQL warehouse) already configured. You don't need to be an administrator to get started.



See [Dashboards](#) to learn about all of the visualization types and features available for dashboards.

Import a dashboard

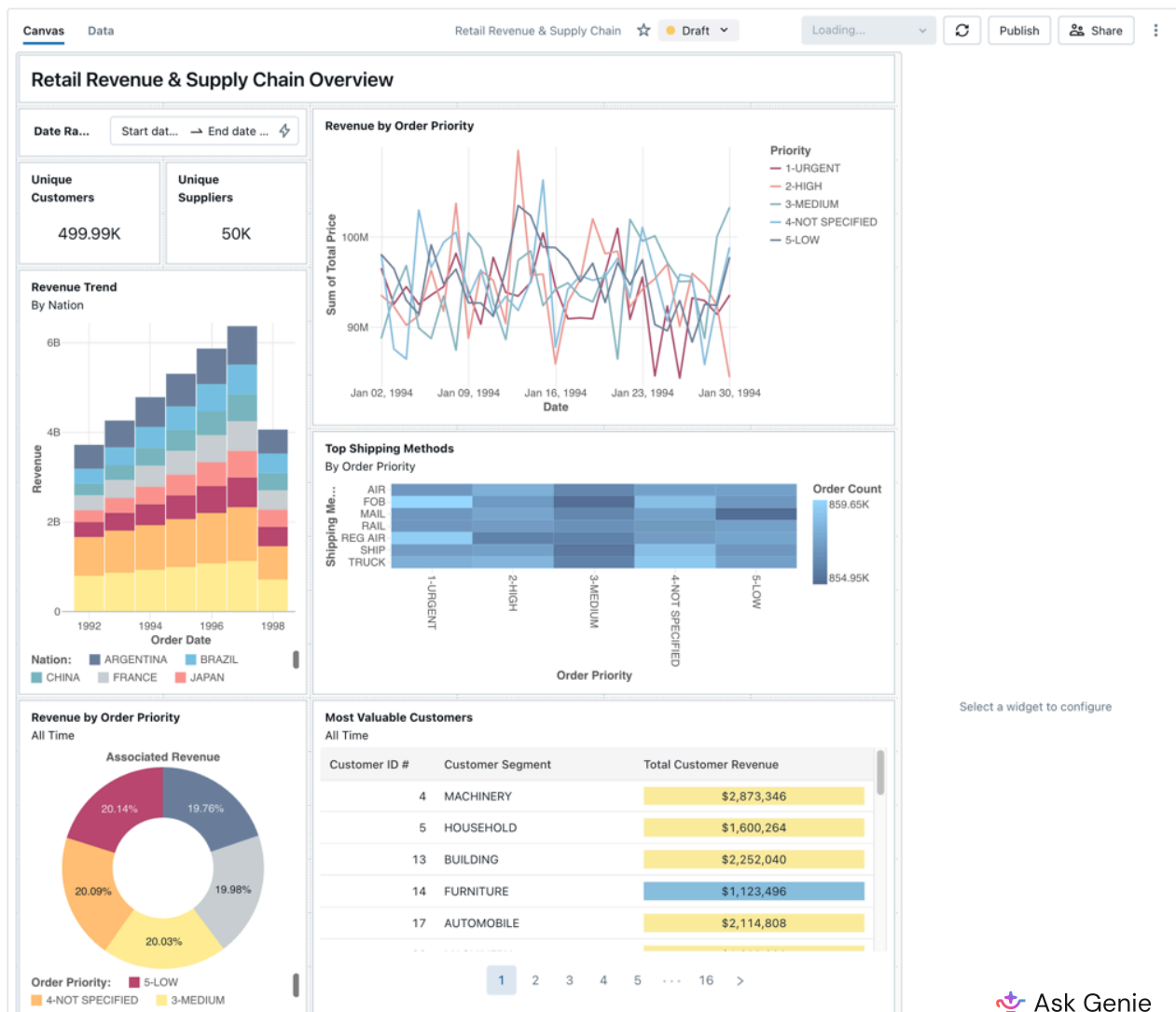
1. In the sidebar, click  **Dashboards**

If your workspace has any saved dashboards, they are listed.

2. Click **View samples gallery**.

3. In the **Retail Revenue & Supply Chain** tile, click **Import**. The dashboard is imported into your workspace, and you are the owner.

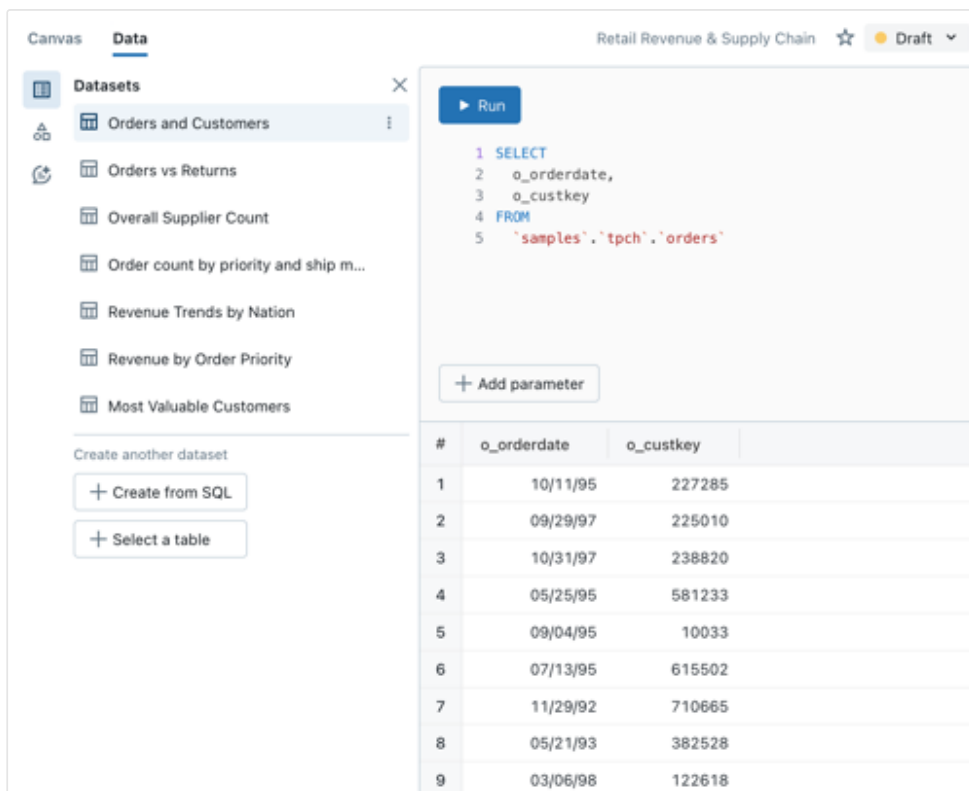
The imported draft dashboard appears, and its visualizations are refreshed.



You can import a sample dashboard multiple times, and multiple users can each import it. You can also import the **NYC Taxi Trip Analysis** dashboard.

Explore a visualization's query

1. Each visualization in a dashboard is the result of a query. You can access all queries in the **Data** tab on the draft dashboard. Click **Data** in the upper-left corner of the screen. Then, click the dataset you want to view to see the associated query.



The screenshot shows the 'Data' tab in a dashboard editor. On the left, a list of datasets is displayed, including 'Orders and Customers', 'Orders vs Returns', 'Overall Supplier Count', 'Order count by priority and ship m...', 'Revenue Trends by Nation', 'Revenue by Order Priority', and 'Most Valuable Customers'. The 'Orders and Customers' dataset is selected. On the right, the SQL query editor is open, showing the following query:

```
1 SELECT
2   o_orderdate,
3   o_custkey
4 FROM
5   'samples'. 'tpch'. 'orders'
```

Below the query, there is a table with 9 rows and 3 columns: '#', 'o_orderdate', and 'o_custkey'. The data is as follows:

#	o_orderdate	o_custkey
1	10/11/95	227285
2	09/29/97	225010
3	10/31/97	238820
4	05/25/95	581233
5	09/04/95	10033
6	07/13/95	615502
7	11/29/92	710665
8	05/21/93	382528
9	03/06/98	122618

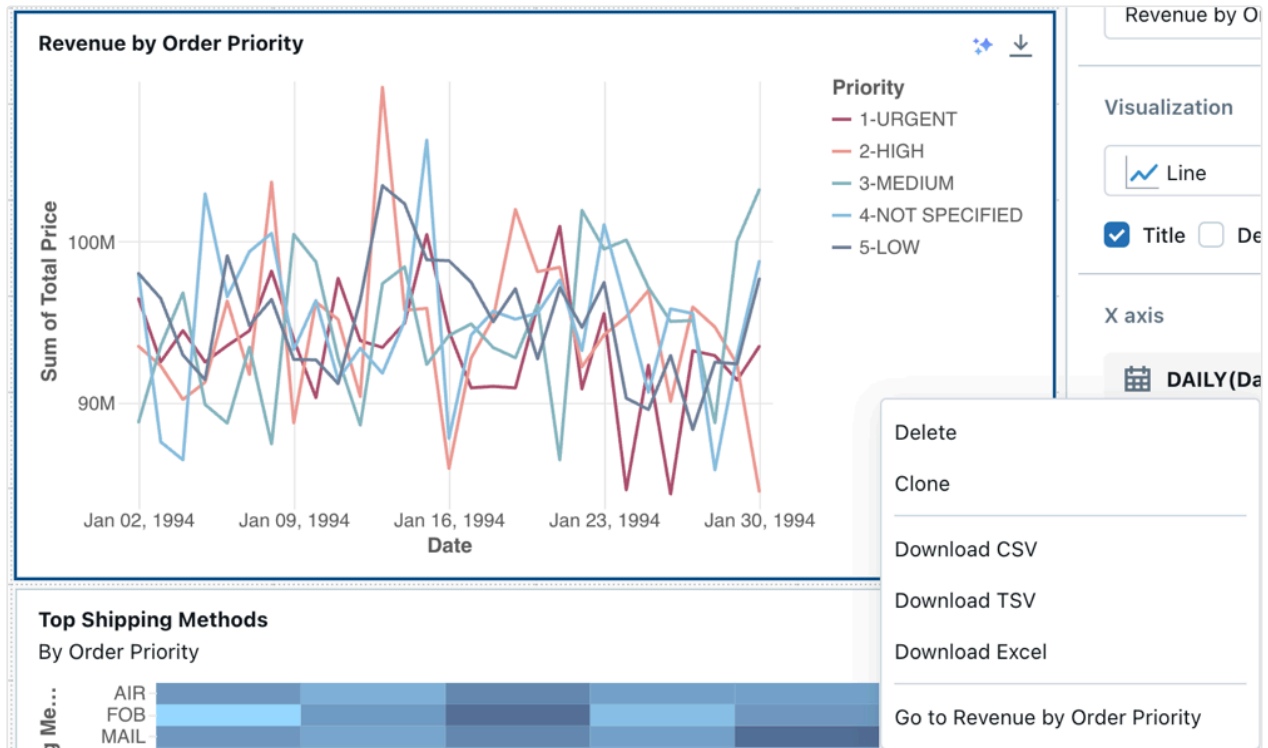
The SQL editor includes the query and results, which are shown in a table below the query.

The sample dashboards use data in the `samples` catalog, separate from data in your workspace. The `samples` catalog is available to every workspace but is read-only.

1. Click the **Canvas** tab to go back to the canvas that shows the dashboard's visualization widgets.

Interact with a visualization

1. Hover over the **Revenue by Order Priority** visualization.
2. Click each **Priority** in the legend to focus on that group of data and hide the other lines.
3. Right-click on the visualization to see its context menu. You can delete or clone a visualization. You can also download the associated dataset as a CSV, TSV, or Excel file. Click **Go to Revenue by Order Priority** to view the associated query.



The query opens on the **Data** tab of your dashboard.

Publish the dashboard

- Click **Publish** at the top of the page. A **Publish** dialog appears.
- Click **Publish** in the dialog to create a sharable, non-editable version of your dashboard. This dashboard is published with your credentials embedded by default. This means that other viewers use your credentials to access the data and compute to generate visualizations on your dashboard. See [Publish a dashboard](#).
- Use the switcher at the top of the page to view your published dashboard.

Draft

▼

✔ d...

Serverless

✔ Draft

Edit the layout of this dashboard

Published

Feb 26, 2024 03:52 PM

Share the dashboard

To share a dashboard with colleagues in your workspace:

1. Click **Share** at the top of the page.
2. Select a user or group in your workspace.

To share the dashboard with all users in the workspace, select **All workspace users**. Then, click **Add**.

3. Select the permission to grant.

Sharing: Retail Revenue & Supply Chain

Do you want information about permission levels? [Learn more](#)

All workspace users X

+

▼

Can Manage ▼

☐ Email new users (excluding groups)

Cancel

Add

[Copy link](#)

To share a dashboard with account users:

1. Under **Sharing settings** at the bottom of the sharing dialog, click **Anyone in my account can view**.

This means that anyone who is registered to your Databricks account can use a link to access your dashboard. If you have embedded your credentials, account-level users don't need workspace access to view your dashboard.

2. Close the form.

Schedule automatic dashboard refreshes

You can schedule the dashboard to refresh at an interval automatically.

1. At the top of the page, click **Schedule**.
2. Click **Add schedule**.
3. Select an interval, such as **Every 1 hour at 5 minutes past the hour**. The SQL warehouse that you selected to run your queries is used to run the dashboard's queries and generate visualizations when the dashboard is refreshed.

Workspace admin users can create, configure, and delete SQL warehouses.

4. Click **Create**.

The dialog shows all schedules associated with the dashboard.

5. Optionally, click **Subscribe** to add yourself as a subscriber and receive an email with a PDF snapshot of the dashboard after a scheduled run completes.

You can use the kebab menu  to edit the schedule and add more subscribers. See [Manage scheduled dashboard updates and subscriptions](#).

6. To delete an existing schedule for a dashboard:

- i. Click **Subscribe**.
- ii. Click the kebab menu  on the right.

iii. Click **Delete**.

Is this page

as this page helpful?

 Yes

 No

 Send feedback

Last updated on **Apr 10, 2026**

ETL in Databricks SQL

When dealing with large amounts of data, you need a pipeline that can process only the new and changed records instead of reprocessing the entire dataset. This is called incremental ETL. In Databricks SQL, you can build incremental ETL pipelines using streaming tables and materialized views, without writing procedural code or scheduling manual refreshes.

This tutorial walks you through a common pattern: tracking product changes over time. You create a source table, capture change events, build a dimension table that preserves the full history of each product, and add an aggregate reporting layer on top.

The key feature in this tutorial is `AUTO CDC`. In a traditional warehouse, you would write complex `MERGE INTO` statements to reconcile insert, update, and delete events into a target table. This approach is error-prone, especially when events arrive out of order. `AUTO CDC` handles this for you. You declare the business key, the sequencing column, and whether you want SCD Type 1 (latest value only) or SCD Type 2 (full history), and Databricks applies the correct merge logic automatically. For an overview of CDC, see [The AUTO CDC APIs: Simplify change data capture with pipelines](#).

By the end of this tutorial, you will have:

1. Created a source table that tracks changes with Change Data Feed.
2. Inspected the raw change data to understand the CDC event stream.
3. Used `AUTO CDC` to build an SCD Type 2 dimension table from those events.
4. Processed delete events incrementally through the pipeline.
5. Created a materialized view that incrementally maintains an aggregate report.
6. Configured `SCHEDULE REFRESH EVERY 1 DAY` so changes propagate automatically through the pipeline.

Requirements

To complete this tutorial, you must meet the following requirements:

- A Databricks workspace with [Unity Catalog](#) enabled.
- A [SQL warehouse](#) (serverless or pro).
- Have permission to [create a compute resource](#) or [access to a compute resource](#).
- [Serverless compute](#) enabled for your account. See [Features with limited regional availability](#).

Step 1: Set up your catalog and schema

Open the [Databricks SQL editor](#) and set your working catalog and schema. You must have permission to `USE` the catalog and schema you select:

SQL

```
USE CATALOG <your-catalog>;  
USE SCHEMA <your-schema>;
```

Step 2: Create a source table and load data

Create a `products` table with [Use Delta Lake change data feed on Databricks](#) (CDF) enabled. CDF is a Delta Lake feature that records every insert, update, and delete as a queryable change log. This is similar to a CDC stream from a transactional source system, except the changes are captured directly within the Delta table rather than from an external log. You use CDF here to generate the change events that the downstream pipeline will consume.

1. Create the table and load initial records:

SQL

 Ask Genie

```
CREATE OR REPLACE TABLE products (
  product_id INT,
  product_name STRING,
  category STRING,
  warehouse STRING
)
TBLPROPERTIES (delta.enableChangeDataFeed = true);

INSERT INTO products VALUES
  (1, 'Spoon', 'Cutlery', 'Seattle'),
  (2, 'Fork', 'Cutlery', 'Portland'),
  (3, 'Knife', 'Cutlery', 'Denver'),
  (4, 'Chair', 'Furniture', 'Austin'),
  (5, 'Table', 'Furniture', 'Chicago'),
  (6, 'Lamp', 'Lighting', 'Boston'),
  (7, 'Mug', 'Kitchenware', 'Seattle'),
  (8, 'Plate', 'Kitchenware', 'Atlanta'),
  (9, 'Bowl', 'Kitchenware', 'Dallas'),
  (10, 'Glass', 'Kitchenware', 'Phoenix');
```

2. Simulate upstream changes, including new products, a warehouse move, and a category reassignment:

SQL

```
INSERT INTO products VALUES
  (11, 'Napkin', 'Dining', 'San Francisco'),
  (12, 'Coaster', 'Dining', 'New York');

UPDATE products SET warehouse = 'Los Angeles' WHERE product_id = 1;
UPDATE products SET category = 'Dining' WHERE product_id = 2;
```

Step 3: Query the change data feed

Before building the downstream pipeline, it helps to look at the raw change events so you can understand what `AUTO CDC` will process. The `table_changes()` function reads the CDF log and returns every captured operation along with metadata columns:

SQL

```
SELECT
  product_id, product_name, warehouse,
  _change_type, _commit_version
FROM table_changes('products', 1)
ORDER BY _commit_version, product_id;
```

For example, the Spoon has three events: an `insert` (Seattle), an `update_preimage` (Seattle), and an `update_postimage` (Los Angeles).

Notice that a single logical change (for example, moving the Spoon to a different warehouse) produces multiple events: a preimage and a postimage. In a traditional warehouse, you would write a `MERGE` statement to reconcile all of these events into a target table, handling inserts, updates, and deletes with separate logic, and making sure events are applied in the correct order. This is exactly the complexity that `AUTO CDC` eliminates in the next step.

Step 4: Build an SCD Type 2 dimension with `AUTO CDC`

ⓘ BETA

`AUTO CDC` is in [Beta](#). Requires Databricks Runtime 17.3 or above.

A [streaming table](#) processes data incrementally. On each refresh, it reads only the new rows since the last run, so it does not need to reprocess the full dataset. This makes it well-suited for high-volume or frequently changing sources.

`AUTO CDC` adds change data capture processing on top of a streaming table. Instead of writing a `MERGE INTO` statement that manually handles inserts, updates, and deletes, you declare the business key and sequencing column and let Databricks apply the correct logic. `AUTO CDC` also handles out-of-order events automatically, which is a common problem when using `MERGE INTO` to handle events arriving from distributed systems or batch loads with overlapping timestamps.

The following statement creates an SCD Type 2 table that preserves the full version history of each product. Each version gets `__START_AT` and `__END_AT` timestamps. A `NULL` in `__END_AT` marks the current version.

 Ask Genie

SQL

```
CREATE OR REFRESH STREAMING TABLE products_history
SCHEDULE REFRESH EVERY 1 DAY
FLOW AUTO CDC
FROM STREAM products WITH (readChangeFeed = true)
KEYS (product_id)
APPLY AS DELETE WHEN _change_type = 'delete'
SEQUENCE BY _commit_timestamp
COLUMNS * EXCEPT (_change_type, _commit_version, _commit_timestamp)
STORED AS SCD TYPE 2;
```

- `SCHEDULE REFRESH EVERY 1 DAY`: refreshes the table on a daily schedule.
- `FLOW AUTO CDC`: declares this as a CDC flow. Databricks applies insert, update, and delete semantics automatically.
- `KEYS (product_id)`: the business key. Events with the same key are merged into versioned rows.
- `APPLY AS DELETE WHEN _change_type = 'delete'`: closes out the current version when a delete event arrives. This lets you define the condition that identifies a delete event.
- `SEQUENCE BY _commit_timestamp`: establishes event ordering. Handles out-of-order arrivals correctly.
- `STORED AS SCD TYPE 2`: retains full history. `AUTO CDC` supports both SCD Type 1 and SCD Type 2.

Query the dimension table:

SQL

```
SELECT product_id, product_name, warehouse, __START_AT, __END_AT
FROM products_history
ORDER BY product_id, __START_AT;
```

- Spoon: two versions. Seattle (closed, `__END_AT` set) and Los Angeles (current, `__END_AT = NULL`).
- Fork: two versions. Cutlery category (closed) and Dining category (current).
- Napkin and Coaster: one version each (newly inserted, `__END_AT = NULL`).
- All other products: one version each (`__END_AT = NULL`).

 Ask Genie

Step 5: Process deletes through the pipeline

Now simulate two discontinued products by deleting them from the source table:

SQL

```
DELETE FROM products WHERE product_id = 9;  
DELETE FROM products WHERE product_id = 10;
```

These delete events are recorded in the CDF log but the streaming table has not seen them yet. Refresh the streaming table to process the new events:

SQL

```
REFRESH STREAMING TABLE products_history;
```

Query the dimension table to verify the deletes were applied:

SQL

```
SELECT product_id, product_name, warehouse, __START_AT, __END_AT  
FROM products_history  
ORDER BY product_id, __START_AT;
```

Bowl and Glass are now closed out with `__END_AT` set, marking them as discontinued. All other current products remain unchanged. The streaming table only processed the new delete events without reprocessing the inserts and updates from the previous refresh.

Step 6: Create an aggregate materialized view

Now that you have a dimension table that stays current with source changes, you can add a reporting layer on top.

A [materialized view](#) stores pre-computed query results as a physical table. Unlike a regular view, which re-executes the query each time you read from it, a materialized view persists the results and only recomputes the rows affected by upstream changes on each refresh. This makes it well-suited for dashboards and reports where query performance matters.

SQL

```
CREATE OR REPLACE MATERIALIZED VIEW products_by_category
SCHEDULE REFRESH EVERY 1 DAY
AS
SELECT
    category,
    COUNT(*) AS active_products
FROM products_history
WHERE __END_AT IS NULL
GROUP BY category;
```

`SCHEDULE REFRESH EVERY 1 DAY` means this view refreshes on a daily schedule.

Combined with the same schedule on the streaming table, you now have a three-stage pipeline where changes to the source table cascade through the dimension and into the aggregate on each refresh cycle. There is no manual refresh to run.

SQL

```
SELECT * FROM products_by_category ORDER BY active_products DESC;
```

Step 7: Verify the end-to-end cascade

To verify the full pipeline cascade, make a change to the source table:

SQL

```
UPDATE products SET warehouse = 'Seattle' WHERE product_id = 3;
```

The Knife moves from Denver to Seattle. This single DML change triggers the full pipeline cascade, demonstrating how the three stages work together:

1. `products` records the change event via CDF.
2. `products_history` processes the event and adds a new version for the Knife.
3. `products_by_category` recomputes only the affected Cutlery row.

Verify:

SQL

```
SELECT product_id, product_name, warehouse, __START_AT, __END_AT
FROM products_history
WHERE product_id = 3
ORDER BY __START_AT;

SELECT * FROM products_by_category ORDER BY active_products DESC;
```

Clean up

To clean up the resources created by this tutorial, use the following SQL:

SQL

```
DROP MATERIALIZED VIEW IF EXISTS products_by_category;
DROP STREAMING TABLE IF EXISTS products_history;
DROP TABLE IF EXISTS products;
```

Additional resources

- [Streaming tables](#)
- [Materialized views](#)
- [CREATE STREAMING TABLE ... FLOW AUTO CDC](#)
- [The AUTO CDC APIs: Simplify change data capture with pipelines](#)
- [Lakeflow Spark Declarative Pipelines](#)

Is this page

as this page helpful?

☐ Yes	☐ No
☐ Send feedback	

Last updated on **Apr 29, 2026**

Write queries and explore data in the new SQL editor

The Databricks UI includes a SQL editor that you can use to author queries, collaborate with colleagues, browse available data, and create visualizations. This page explains how to use the SQL editor to write, run, manage, and share queries.

This article explains how to use the new SQL editor. To learn about working with the legacy SQL editor, see [Write queries and explore data in the legacy SQL editor](#).

NOTE

Starting in late May 2026, the new SQL editor will be enabled by default for all workspaces. The ability to turn off the new SQL editor at the workspace level will no longer be available. Individual users will still be able to switch to the legacy editor during this period. Starting in late July 2026, the legacy SQL editor will be retired and all users will use the new SQL editor. See [What's coming?](#) for details.

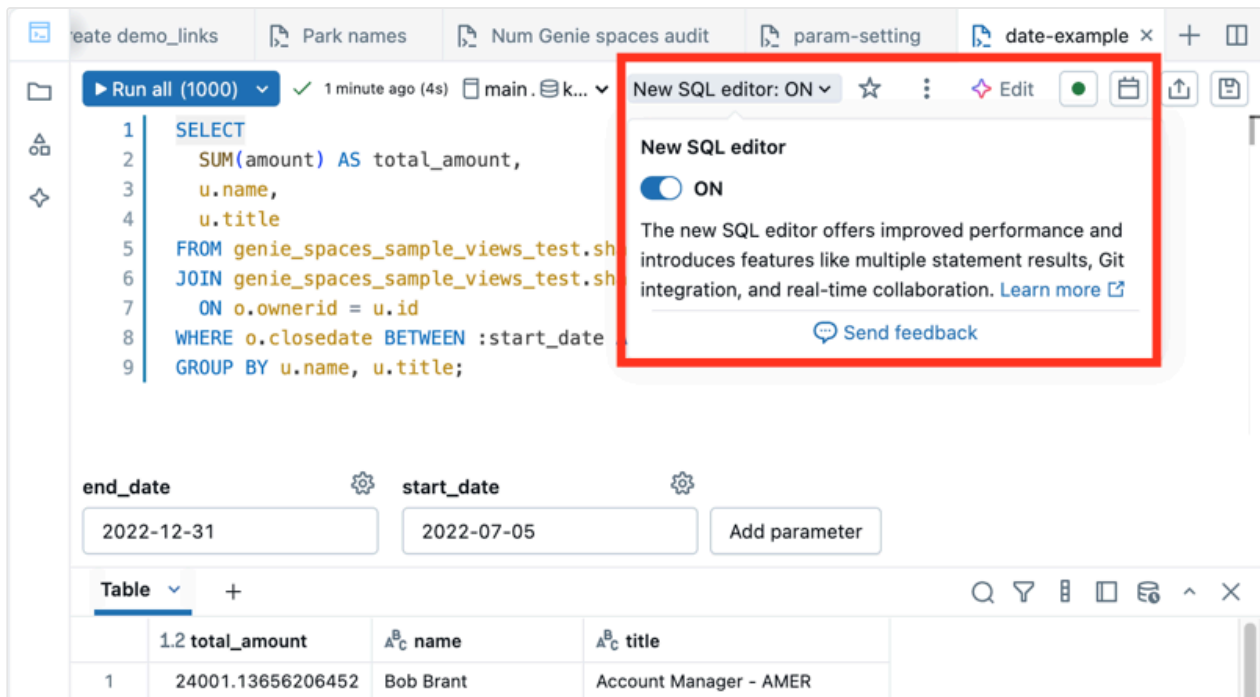
Open the SQL editor

To open the SQL editor in the Databricks UI, click  **SQL Editor** in the sidebar.

The SQL editor opens to your last open query. If no query exists, or all of your queries have been explicitly closed, the SQL editor landing page opens.

Turn on the new SQL editor

Use the **New SQL editor** toggle, to the right of the catalog and schema drop-down selectors, to turn the new editor on. On narrow displays, you can find the **New SQL editor** toggle by clicking the kebab menu  to the right of the catalog a  Ask Genie selectors.



Enable the new editor for any query that you own:

- Click **New SQL editor: OFF**. A **New SQL editor** popover menu opens.
 - The **Apply to all of my queries** checkbox is selected by default. Keep this setting to enable the editor for all queries that you own.
 - To enable the new editor for the active query only, unselect the **Apply to all of my queries** checkbox.
- Set the **New SQL editor** toggle to **ON**.

The new SQL editor cannot be used for all queries. You cannot switch to the new SQL editor for a query if any of the following are true:

- You do not own the query.
- The query contains **Query-based Dropdown Lists**, which are not supported. See [Use named parameter markers](#).

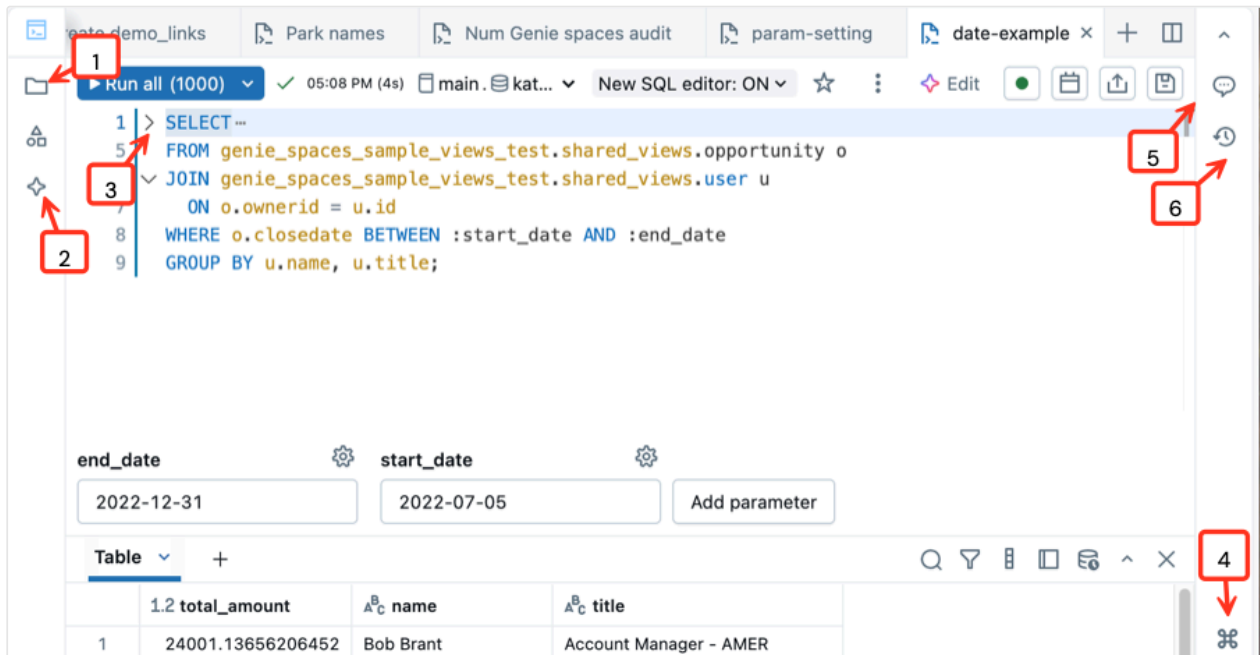
When you turn on the new SQL editor, your query reopens in the new UI. The result set is empty until you run the query.

NOTE

Query results are shared with all collaborators and are limited to 64,000 rows.

Orientation

This section highlights key differences in the UI and explains how to find and use available features.

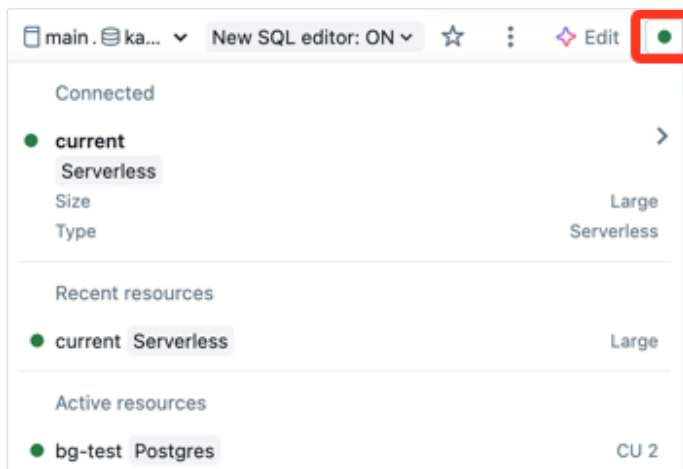


- 1. Access the workspace file system:** You can access and organize your workspace objects directly from the SQL editor.
- 2. Use the integrated Genie Code:** Chat with Genie Code to help you write, refine, or understand queries. Run code generated by Genie Code from the side panel.
- 3. Collapse sections of code:** Click the collapse arrow to the left of a code block to selectively hide and expand code sections. This can make working through large files easier.
- 4. Open the command palette:** The command palette includes common actions, keyboard shortcuts, custom themes, and links to help you quickly find what you need. After you open it, use the search bar to look for specific tools and shortcuts.
- 5. Comment on code:** Add comments to code to enhance collaboration.
- 6. Review version history:** Review recent changes to your query.

Connect to compute

You must have at least CAN USE permissions on a SQL warehouse to run queries. You can use the drop-down near the top of the editor to see available options. To filter the

list, enter text in the text box.



If you have a default SQL warehouse, the SQL editor automatically uses it when you create a query. If no default warehouse is set, you select from an alphabetical list of available warehouses. Subsequent queries use the last selected warehouse. To set a default warehouse, see [Set a user-level default warehouse](#).

The icon next to the SQL warehouse indicates the status:

- Running 
- Stopped 

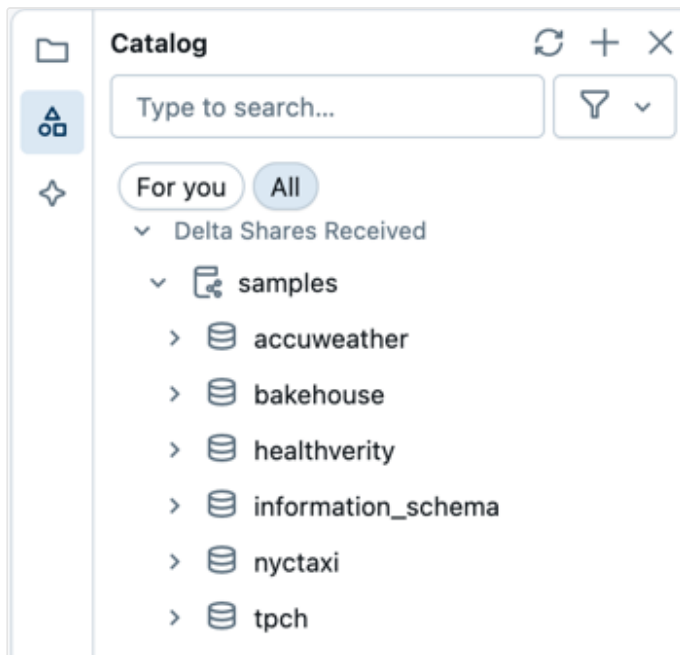
NOTE

If there are no SQL warehouses in the list, contact your workspace administrator.

The selected SQL warehouse will restart automatically when you run your query. See [Start a SQL warehouse](#) to learn other ways to start a SQL warehouse.

Browse assets and get help

Use the left pane in the SQL editor to browse workspace files, view data objects, and get help from Databricks Assistant.



Browse workspace files

Click  the folder icon to open your workspace user folder. You can navigate across all of the workspace files you have access to from this part of the UI.

Browse data objects

If you have metadata read permission, the schema browser in the SQL editor shows the available databases and tables. You can also browse data objects from [Catalog Explorer](#).

You can navigate Unity Catalog-governed database objects in Catalog Explorer without active compute. To explore data in the `hive_metastore` and other catalogs not governed by Unity Catalog, you must attach to compute with appropriate privileges. See [Data governance with Databricks](#).

NOTE

If no data objects exist in the schema browser or Catalog Explorer, contact your workspace administrator.

Click  near the top of the schema browser to refresh the schema. You can type in the search bar to filter assets by name. Click the  filter icon to filter objects by type.

Click the name of an object in the browser to see more details on the object. For example, click a schema name to show the tables in that schema. Click a table name to show the columns in that table.

Get help from Genie Code

Click  Genie Code icon to open a chat window with Genie Code. Click a suggested question or enter your own question to interact with Genie Code.

Create a query

You can enter text to create a query in the SQL editor. You can insert elements from the schema browser to reference catalogs and tables.

1. Type your query in the SQL editor.

The SQL editor supports autocomplete. As you type, autocomplete suggests completions. For example, if a valid completion at the cursor location is a column, autocomplete suggests a column name. If you type `select * from table_name as t where t.`, autocomplete recognizes that `t` is an alias for `table_name` and suggests the columns inside `table_name`. You can also use autocomplete to reference [query snippets](#).

```

1 SELECT
2   *
3 FROM
4   sales_customers sc
5 where
6   sc.

```

 address	string
 city	string
 continent	string
 country	string
 customerID	bigint
 email_address	string
 first_name	string
 gender	string
 last_name	string
 phone_number	string
 postal_zip_code	bigint
 state	string

2. (Optional) When you are done editing, click **Save**. By default, the query is saved to your user home folder. To save the query to a different location, select the target folder and click **Move**.

NOTE

New queries are automatically named **New query** with the creation timestamp appended in the title. By default, new queries created without a specific folder context are created in the **Drafts** folder in your home directory. When new queries are saved or renamed, they are removed from **Drafts**.

Query data sources

You can identify a query source using a [fully-qualified table name](#) in the query itself or by selecting a combination of catalog and schema from the drop-down selectors along with the table name in the query. A fully-qualified table name in the query overrides the catalog and schema selectors in the SQL editor. If a table or column name includes spaces, wrap those identifiers in backticks in your SQL queries.

NOTE

 Ask Genie

The maximum number of results returned in a table is 64,000 rows or 10MB, whichever is smaller.

The following examples demonstrate how to query various table-like objects that you can store in a catalog.

Query a standard table or view

The following example queries a table from the `samples` catalog.

SQL

```
SELECT
  o_orderdate,
  o_orderkey,
  o_custkey,
  o_totalprice,
  o_shippriority
FROM
  samples.tpch.orders
```

Query a metric view

The following example queries a metric view that uses a table from the `samples` catalog as its source. It evaluates the three listed measures and aggregates over `Order Month` and `Order Status`. It returns results sorted by `Order Month`. To create a similar metric view in your workspace, see [Tutorial: Build a complete metric view with joins](#).

All measure evaluations must be wrapped in the `MEASURE` function. See [measure aggregate function](#).

SQL

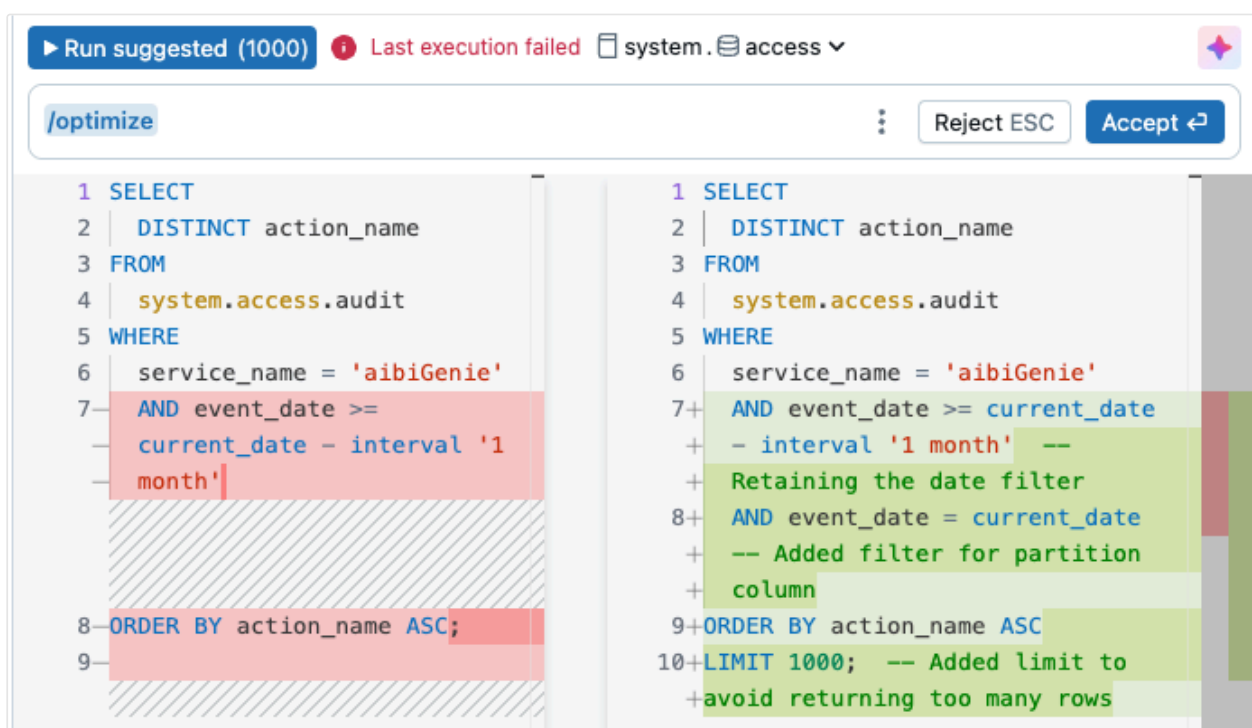
```
SELECT
  `Order Month`,
  `Order Status`,
  MEASURE(`Order Count`),
  MEASURE(`Total Revenue`),
  MEASURE(`Total Revenue per Customer`)
FROM
```

 Ask Genie


```
orders_metric_view
GROUP BY ALL
ORDER BY 1 ASC;
```

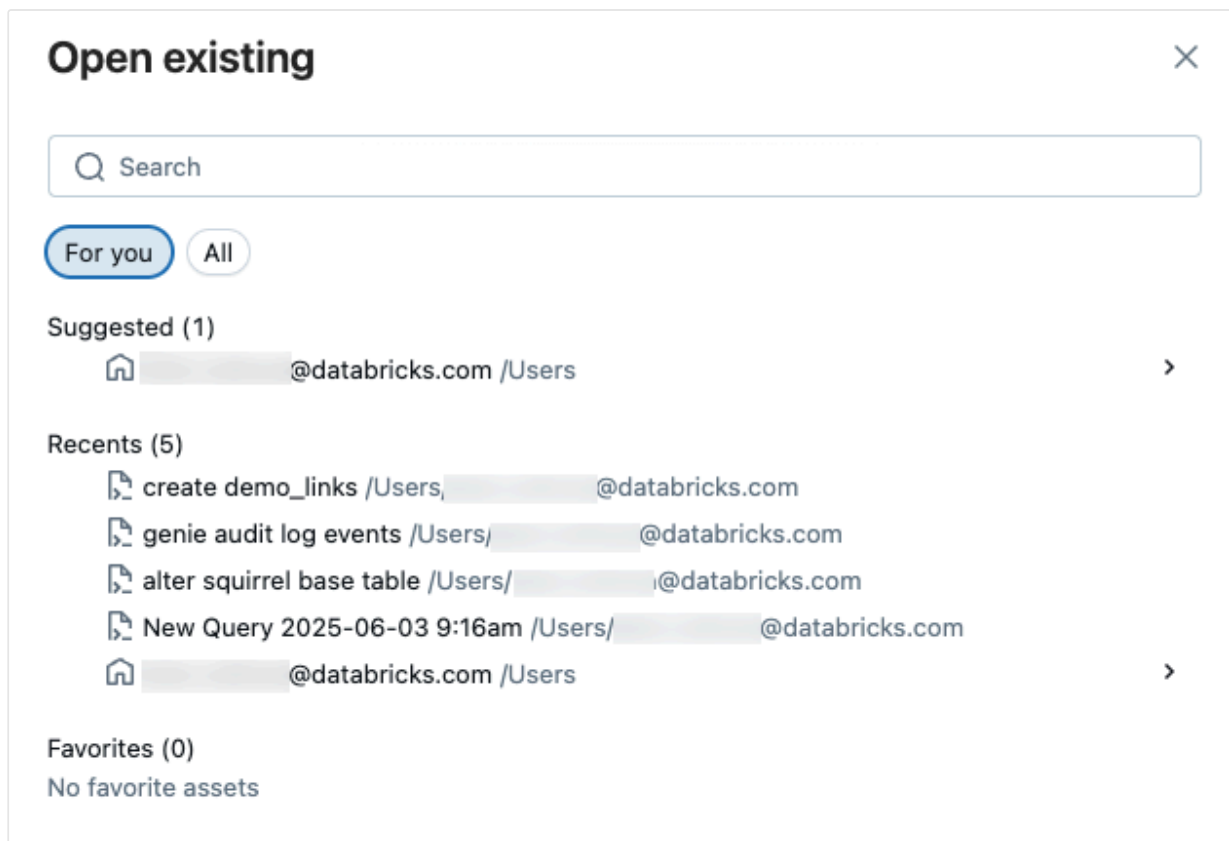
Optimize a query with Genie Code

Click the  Assistant icon on the right side of the editor to get inline help and suggestions when writing queries. The `/optimize` slash command prompts Assistant to evaluate and optimize queries. For more information, see [Optimize Python, PySpark, and SQL code](#).



Edit multiple query tabs

By default, the SQL editor uses tabs so you can open and edit multiple queries simultaneously. To open a new tab, click **+**, then select **Create new query** or **Open existing query**. Click **Open existing query** to see a list of queries. The **For you** tab offers a curated list of suggestions based on your usage. Use the **All** tab to find any query that you have access to.



Run a single query or multi-statement queries

You can write multi-statement queries into a single editor tab. To use multiple query statements in a single tab, end each statement with a semicolon (;).

To run a single query:

1. Select a SQL warehouse.
2. Highlight a query in the SQL editor (if multiple queries are in the query pane).
3. Press **Ctrl/Cmd + Enter** or click **Run (1000)** to display the results as a table in the results pane.

NOTE

By default, **Run all statements** and **Limit 1000** are selected.

- To run only the active statement, unselect **Run all statements**.
- If a query is saved with the **Limit 1000** setting, this setting applies to all query runs. To return all rows (up to 64,000 rows or 10 MB, whichever is

 Ask Genie

smaller), clear **LIMIT 1000** from the **Run (1000)** drop-down menu. To specify a different limit on the number of rows, add a `LIMIT` clause in your query.

Terminate a query

To terminate a query while it is running, click **Cancel**. An administrator can stop an running query that another user started by viewing the [Terminate an executing query](#).

Share a query

The new SQL editor handles permissions in two parts:

Interactive execution: Queries in the new SQL editor always run using **Run as viewer** credentials. Query results are accessible to anyone who has access to the query. Users with CAN EDIT permissions or higher can modify the shared query draft. Additionally, unsaved edits are visible to anyone with CAN VIEW permissions or above. Users must have at least CAN USE permission on the associated warehouse to run queries.

Credential settings: The permission settings for legacy alerts and [Workflows](#) are the same as those for the legacy SQL editor. Depending on your preferences, these assets can be run using either the owner's or the viewer's credentials.

See [Configure query permissions](#) to learn about **Run as owner** and **Run as viewer** credential settings. See [Query ACLs](#) to learn how to manage access to the query object.

Collaborate on queries in real time

```
1 SELECT
2   T.pickup_zip,
3   T.dropoff_zip,
4   T.route as `Route`,
5   T.frequency as `Number Trips`,
6   T.total_fare as `Total Revenue`
7 FROM
8   (
9     SELECT
10      pickup_zip,
11      dropoff_zip,
12      concat(pickup_zip, '-', dropoff_zip) AS route,
13      count(*),
14      SUM(fare_amount) as total_fare
15    FROM
16
17    GROUP BY
18      1,2,3
19  ) T
20 ORDER BY
21   1 ASC
```

After sharing the query, you can coauthor or debug queries live through collaborative editing. Editors can also comment on individual lines of code to ask questions or add suggestions.

Source control a query

Databricks SQL query files (extension: `.dbquery.ipynb`) are supported in [Databricks Git folders](#). You can use a Git folder to source control your query files and to share them in other workspaces with Git folders that access the same Git repository. If you choose to opt out of the new SQL editor after committing or cloning a query in a Databricks Git folder, delete and reclone that Git folder to avoid unexpected behaviors.

Copy the legacy query ID

To get the legacy query ID for a query that has been migrated to the new SQL editor, copy it from the **File** menu. This can be useful for API calls or other integrations that reference the legacy query ID.

1. Open the query in the new SQL editor.

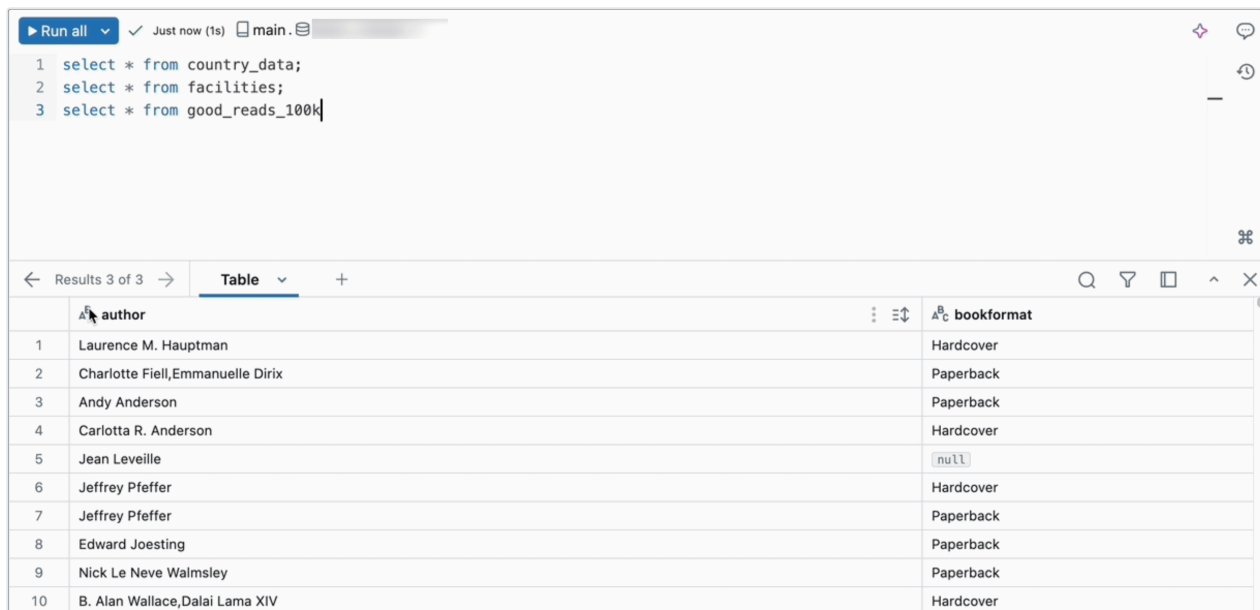
2. Click the kebab menu  at the top of the editor.
3. Click **File** > **Copy legacy query ID**.

Save a query

Query content in the new SQL editor is continuously autosaved. The **Save** button controls whether the draft query content should be applied to related assets, like workflows or legacy alerts. If the query is shared with the **Run as owner** credential, only the query owner can use the **Save** button to propagate changes. If the credential is set to **Run as viewer**, any user with at least `CAN MANAGE` permission can save the query.

View query results

You can compare the output from each statement with multiple result statements. When you run multiple SQL statements from a single edit window, you can use the UI in the **Output** panel near the bottom of the screen to scroll through the different result sets.



The screenshot shows a SQL editor window with a query and its results. The query is:

```
1 select * from country_data;
2 select * from facilities;
3 select * from good_reads_100k
```

The results are displayed in a table with 10 rows and 2 columns: **author** and **bookformat**.

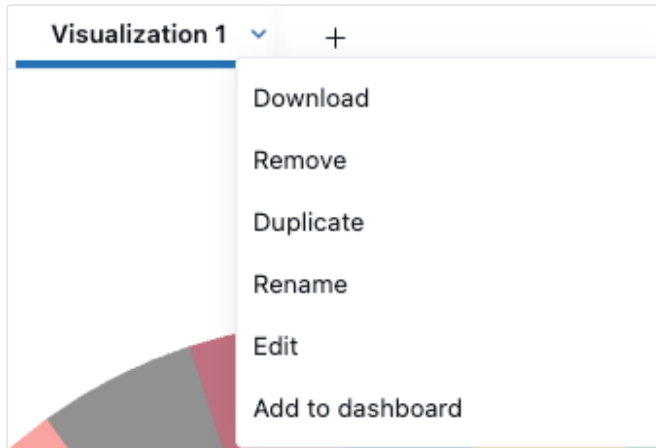
	author	bookformat
1	Laurence M. Hauptman	Hardcover
2	Charlotte Fiell,Emmanuelle Dirix	Paperback
3	Andy Anderson	Paperback
4	Carlotta R. Anderson	Hardcover
5	Jean Leveille	null
6	Jeffrey Pfeffer	Hardcover
7	Jeffrey Pfeffer	Paperback
8	Edward Joesting	Paperback
9	Nick Le Neve Walmsley	Paperback
10	B. Alan Wallace,Dalai Lama XIV	Hardcover

You can interact with and explore your query results using the result pane. The result pane includes the following features for exploring results.

Visualizations

Click the  to add a visualization. Visualizations can help explore the result set. See [Notebook and SQL editor visualization types](#) for a complete list of available visualization types.

Click the  on a visualization tab to show additional options.



- Click **Download** to download the data represented in the visualization as a CSV, TSV, or Excel file.
- Click **Remove** to delete the visualization.
- Click **Duplicate** to clone the results tab.
- Click **Rename** to retitle the visualization.

1. Click **Edit** to edit the visualization.

2. Click **Add to dashboard** to copy the query and visualization to a new [AI/BI dashboard](#).

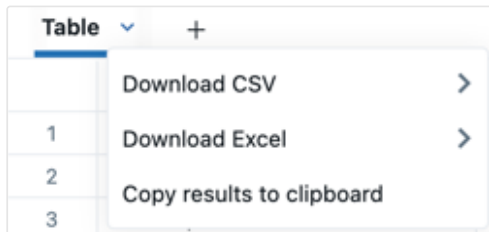
3. Select **Create new dashboard** or **Add to existing dashboard**.

- **Create new dashboard:** Creates a new dashboard that includes all the visualizations associated with the query. You are prompted to choose a name for the new dashboard. The new dashboard is saved to your home folder.
- **Add to existing dashboard:** Adds your query and associated visualizations to an existing dashboard. Use the drop-down menu to choose the dashboard that the dataset and visualization should be added to.

Download results

You can download results as a CSV or Excel file. CSV downloads support up to approximately 5GB of data. Click the  in a results tab to view download options. The

final file download size might be slightly more or less than 5GB, as the 5GB limit is applied to an earlier step than the final file download.



NOTE

If you cannot download a query, your workspace administrator has disabled download for your workspace.

Filters

Filters allow you to limit the result set after a query has run. You can apply filters to selectively show different subsets of the data. Use Genie Code to help you add a filter for results.

To add a filter:

1. Click  the filter icon.
2. Enter text to describe the kind of filter you want to apply. A named filter is applied to the result set.

To edit a filter:

1. Click the name of the filter.
2. Use the drop-down menus and text fields to adjust the filter settings.
3. (Optional) Select **Case sensitive** to make the filter case sensitive.
4. (Optional) Click **Add OR condition** to add an additional filter.

Add parameters

Parameters allow you to limit the result set by substituting values into a query at runtime. See [Use named parameter markers](#) to learn how to apply parameters.

Parameter syntax changes

Queries written in the legacy SQL editor that use double curly braces ({{}}) to mark parameters must be modified to run in the new SQL editor. The new SQL editor uses the same parameter syntax as other Databricks tools, like AI/BI dashboards, Genie Spaces, and notebooks.

Parameters in the new SQL editor are defined using named parameter syntax. Named parameters are parameters in a query that are prefixed with a colon, for example

:parameter_name.

See [Named parameter markers](#) for the SQL reference documentation. See [Named parameter syntax examples](#) for examples of converting parameter syntax from the classic SQL editor to named parameter markers.

Supported filter types

The following table shows filter types supported in the new SQL editor. The second column indicates whether the filter also works in the legacy SQL editor.

Filter type	Supported in legacy editor
Is equal to	Yes
Is not equal to	No
Is one of	Yes
Is not one of	No
Contains	Yes
Does not contain	No
Starts with	Yes
Does not start with	No

Filter type	Supported in legacy editor
Ends with	No
Does not end with	No
Is null	No
Is not null	No
Greater than	No
Greater than or equal to	No
Less than	No
Less than or equal to	No
Between	Yes
Before	No
After	No
Is true	Yes
Is false	Yes
OR conditions	No

Use keyboard shortcuts

The new SQL editor includes many built-in keyboard shortcuts to help you work quickly. The following table identifies commonly used keyboard shortcuts.

Function	macOS shortcut	Windows shortcut
Run a query	Cmd + Enter	Ctrl + Enter
Run a single highlighted statement	Cmd + Shift + Enter	Ctrl + Shift + Enter
Autocomplete	Ctrl + Space	Ctrl + Space
Increase font size	Option + + (plus key)	Alt + + (plus key)
Decrease font size	Option + - (minus key)	Alt + - (minus key)
Format SQL code	Cmd + Shift + F	Ctrl + Shift + F
View all shortcuts	Esc then H	Esc then H

Disable the new SQL editor

NOTE

Starting in late May 2026, the workspace-level opt-out for the new SQL editor will be removed. Individual users can still switch to the legacy editor until late July 2026, when the legacy SQL editor will be fully retired. See [What's coming?](#) for details.

To disable the new SQL editor:

- Click **New SQL editor: ON**. A **New SQL editor** popover menu opens.
- Set the **New SQL Editor** toggle to **OFF**.
- In the confirmation dialog, choose to disable the new editor for the active query only or for all eligible queries.
 - The **Disable for all eligible queries** is unselected. Keep this setting if you want to disable the new editor for only the active query.

- To disable the new SQL editor for all your queries, select the checkbox **Disable for all eligible queries**.

To prevent queries from automatically opening in the new SQL editor:

1. In the Databricks workspace, click your username in the top bar and select **Settings**.
2. In the left pane, under **User**, click the **Developer** tab.
3. Turn off **Create and open all eligible queries in the new SQL editor**.

Workspace admins can turn off this feature at the workspace level by visiting the preview portal and searching for **SQL editor**. See [Manage Databricks previews](#).

Is this page

as this page helpful?

☐ Yes

☐ No

☐ Send feedback

Last updated on **Apr 29, 2026**

Custom format SQL statements

ⓘ PREVIEW

This feature is in [Public Preview](#).

This article explains how to customize SQL auto-formatting options in the Databricks UI.

Overview

SQL formatting improves the readability and maintainability of your queries. By standardizing indentation, keyword casing, and other stylistic preferences, a well-formatted query is easier to understand, debug, and share with your team.

In Databricks, you can configure formatting options using a `.dbsql-formatter-config.json` file saved in your workspace home directory. After saving the file, your SQL code is automatically formatted based on your preferences when you run or format code in the editor or a SQL notebook.

How to configure SQL formatting settings

To configure your personal SQL formatting options:

1. Click  **Workspace** in the sidebar.
2. Click **Create > File**.
3. Name the file `.dbsql-formatter-config.json`.

 Ask Genie

- 4. Add your desired settings as key-value pairs in JSON format. See [Formatter options](#)
- 5. To apply the latest formatting settings, refresh the page.

Example configuration file

The following example shows a valid `.dbsql-formatter-config.json` configuration.

JSON

```
{
  "printWidth": 80,
  "indentationStyle": "spaces",
  "indentationWidth": 4,
  "keywordCasing": "uppercase",
  "shouldExpandExpressions": true
}
```

Formatter options

The following table lists the supported configuration options.

Option	Config file name	Allowed values	Default	De
Print width	<code>printWidth</code>	Any integer >= 0	<code>100</code>	Se lin for
Indentation style	<code>indentationStyle</code>	<code>'spaces'</code> , <code>'tabs'</code>	<code>'spaces'</code>	Sp to tak inc in is i
 Ask Genie				

Option	Config file name	Allowed values	Default	De
Indentation width	<code>indentationWidth</code>	Integer from 0 to 99	2	Nu us in is s 's
Keyword casing	<code>keywordCasing</code>	'uppercase', 'lowercase', 'none'	'none'	Co for key lea un
Function name casing	<code>functionNameCasing</code>	'uppercase', 'lowercase', 'none'	'none'	Co for fun 'n ca
Comma placement	<code>commaPosition</code>	'beginning', 'end'	'end'	Co co pla
Line breaks between statements	<code>numNewLinesBetweenStatements</code>	Integer from 0 to 99	1	Nu line be sta
Line breaks between clauses	<code>numNewLinesBetweenClauses</code>	Integer from 0 to 99	0	Nu line be wit sta

Option	Config file name	Allowed values	Default	De
Expand boolean expressions	<code>shouldExpandExpressions</code>	<code>true</code> , <code>false</code>	<code>true</code>	Ex exp se
Expand CASE statements	<code>shouldExpandCaseStatements</code>	<code>true</code> , <code>false</code>	<code>true</code>	Ex cla sta ow
Expand IN lists	<code>shouldExpandInStatements</code>	<code>true</code> , <code>false</code>	<code>false</code>	Ex IN se
Expand BETWEEN conditions	<code>shouldExpandBetweenConditions</code>	<code>true</code> , <code>false</code>	<code>false</code>	Ex BE co se
Break JOIN conditions	<code>shouldBreakOnJoinSections</code>	<code>true</code> , <code>false</code>	<code>true</code>	Br co se

Is this page

Is this page helpful?

☐ Yes

☐ No

☐ Send feedback

Last updated on **Apr 29, 2026**

Schedule a query

You can use scheduled query executions to update your dashboards or enable routine alerts. By default, your queries do not have a schedule.

NOTE

If an alert uses your query, the alert runs on its own refresh schedule and does not use the query schedule.

To set the schedule:

1. In the Query Editor, click **Schedule** > **Add schedule** to open a menu with schedule settings.

Add schedule

Settings

Schedule

Every 1 

 Hour  at 35 minutes past the hour 

☐ Show cron syntax

Timezone

(UTC-04:00) Eastern Time (US and Canada) 

More options 

 Collaborators may have schedules which are not visible to you. Consult with them to avoid creating duplicate schedules.

[Cancel](#) [Create](#)

 Ask Genie

2. Choose when to run the query.

- Use the dropdown pickers to specify the frequency, period, starting time, and time zone. Optionally, select the **Show cron syntax** checkbox to edit the schedule in [Quartz Cron Syntax](#).
- Choose **More options** to show optional settings. You can also choose:
 - A name for the schedule.
 - A SQL warehouse to power the query. By default, the SQL warehouse used for ad hoc query execution is also used for a scheduled job. Use this optional setting to select a different warehouse to run the scheduled query.

3. Click **Create**. Your query will run automatically according to the schedule. If you experience a scheduled query not executing according to its schedule, you should manually trigger the query to make sure it doesn't fail.

If a query execution fails during a scheduled run, Databricks retries with a back-off algorithm. This means that retries happen less frequently as failures persist. With persistent failures, the next retry might exceed the scheduled interval.

After you create a schedule, the label on the **Schedule** button reads **Schedule(#)**, where the **#** is the number of scheduled events that are visible to you. You cannot see schedules that have not been shared with you.

❗ IMPORTANT

New schedules are not automatically shared with other users, even if those users have access to the query. To make scheduled runs and results visible to other users, use the sharing settings described in the next step.

4. Share the schedule

Query permissions are not linked to schedule permissions. After creating your scheduled run interval, edit the schedule permissions to provide access to other users.

- Click **Schedule(#)**.
- Click the  kebab menu and select **Edit schedule permissions**.
- Choose a user or group from the drop-down menu in the dialog.

 Ask Genie

- Choose CAN VIEW to allow the selected users to view the results of scheduled runs.

Refresh behavior and execution context

When a query is “Run as Owner” and a schedule is added, the query owner's credential is used for execution, and anyone with at least CAN RUN sees the results of those refreshed queries.

When a query is “Run as Viewer” and a schedule is added, the schedule owner's credential is used for execution. Only the users with appropriate schedule permissions see the results of the refreshed queries; all other viewers must manually refresh to see updated query results.

Is this page

as this page helpful?

☐ Yes

☐ No

☐ Send feedback

[Data warehousing](#)[SQL editor](#)[Use the legacy SQL editor](#)

Last updated on **Apr 29, 2026**

Write queries and explore data in the legacy SQL editor

ⓘ IMPORTANT

The legacy SQL editor will be retired in late July 2026. Starting in late May 2026, the new SQL editor will be enabled by default for all workspaces and the workspace-level opt-out will be removed. Individual users can still switch to the legacy editor until late July 2026, when the legacy editor will be fully retired. Databricks recommends transitioning to the [new SQL editor](#). See [What's coming?](#) for details.

The Databricks UI includes a SQL editor that you can use to author queries, browse available data, and create visualizations. You can also share your saved queries with other team members in the workspace. This article explains how to use the SQL editor to write, run, and manage queries. To learn how to enable and work with the new SQL editor, see [Write queries and explore data in the new SQL editor](#).

[Ask Genie](#)

The screenshot displays the Databricks SQL Editor. On the left is a sidebar with navigation icons. Next to it is the 'Catalog' panel with a search bar and a list of databases: 'hive_metastore', 'ml', 'samples', and 'system'. The main area is the SQL editor, which contains a query to select the top 10 accounts by country based on order revenue. The query is as follows:

```

1 SELECT
2   o_orderdate AS Date,
3   o_orderpriority AS Priority,
4   sum(o_totalprice) AS `Total Price`
5 FROM
6   `samples`.`tpch`.`orders`
7 WHERE
8   o_orderdate > '1994-01-01'
9   AND o_orderdate < '1994-01-31'
10 GROUP BY
11   1,
12   2
13 ORDER BY
14   1,
15   2

```

Below the query editor is a table showing the results. The table has columns: 'Date', 'Priority', and 'Total Price'. The results are sorted by 'Total Price' in descending order. The first 9 rows are visible:

	Date	Priority	Total Price
1	1994-01-02	1-URGENT	96444609.82
2	1994-01-02	2-HIGH	93497904.94
3	1994-01-02	3-MEDIUM	88800085.02
4	1994-01-02	4-NOT SPECIFIED	97955477.98
5	1994-01-02	5-LOW	98015661.37
6	1994-01-03	1-URGENT	92534508.96
7	1994-01-03	2-HIGH	92286715.43
8	1994-01-03	3-MEDIUM	93521575.91
9	1994-01-03	4-NOT SPECIFIED	97569521.46

At the bottom of the results table, it says '12 s 751 ms | 145 rows returned' and 'Refreshed a minute ago'.

After opening the editor, you can author a SQL query or browse the available data. The text editor supports autocomplete, autoformatting, and various other keyboard shortcuts.

You can open multiple queries using the query tabs at the top of the text editor. Each query tab has controls for running the query, marking the query as a favorite, and connecting to a SQL warehouse. You can also **Save**, **Schedule**, or **Share** queries.

Open the SQL editor

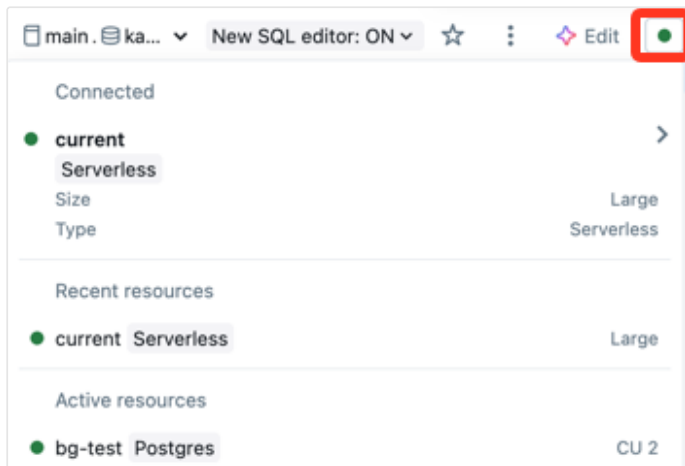
To open the SQL editor in the Databricks UI, click  **SQL Editor** in the sidebar.

The SQL editor opens to your last open query. If no query exists, or all of your queries have been explicitly closed, a new query opens. It is automatically named **New Query** and the creation timestamp is appended in the title.

If the **New SQL editor** is turned on, you can disable it to access the legacy SQL editor. See [Disable the new SQL editor](#).

Connect to compute

You must have at least CAN USE permissions on a running SQL warehouse to run queries. You can use the drop-down near the top of the editor to see available options. To filter the list, enter text in the text box.



The first time you create a query, the list of available SQL warehouses appears alphabetically. The last used SQL warehouse is selected the next time you create a query.

The icon next to the SQL warehouse indicates the status:

- Running
- Stopped

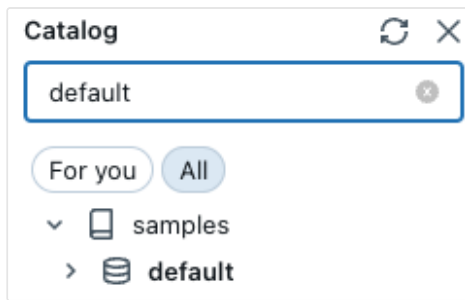
NOTE

If there are no SQL warehouses in the list, contact your workspace administrator.

The selected SQL warehouse will restart automatically when you run your query. See [Start a SQL warehouse](#) to learn other ways to start a SQL warehouse.

Browse data objects in SQL editor

If you have metadata read permission, the schema browser in the SQL editor shows the available databases and tables. You can also browse data objects from [Catalog Explorer](#).



You can navigate Unity Catalog-governed database objects in Catalog Explorer without active compute. To explore data in the `hive_metastore` and other catalogs not governed by Unity Catalog, you must attach to compute with appropriate privileges. See [Data governance with Databricks](#).

NOTE

If no data objects exist in the schema browser or Catalog Explorer, contact your workspace administrator.

Click  near the top of the schema browser to refresh the schema. You can filter the schema by typing filter strings in the search box.

Click a table name to show the columns for that table.

Create a query

You can enter text to create a query in the SQL editor. You can insert elements from the schema browser to reference catalogs and tables.

1. Type your query in the SQL editor.

The SQL editor supports autocomplete. As you type, autocomplete suggests completions. For example, if a valid completion at the cursor location is a column, autocomplete suggests a column name. If you type `select * from table_name as t where t.`, autocomplete recognizes that `t` is an alias for `table_name` and suggests the columns inside `table_name`.

```
1 SELECT
2   *
3 FROM
4   sales_customers sc
5 where
6   sc.
```

 address	string
 city	string
 continent	string
 country	string
 customerID	bigint
 email_address	string
 first_name	string
 gender	string
 last_name	string
 phone_number	string
 postal_zip_code	bigint
 state	string

2. (Optional) When you are done editing, click **Save**. By default, the query is saved to your user home folder, or you can select a different location. Then, click **Save**.

Query data sources

You can identify a query source using a [fully-qualified table name](#) in the query itself or by selecting a combination of catalog and schema from the drop-down selectors along with the table name in the query. A fully-qualified table name in the query overrides the catalog and schema selectors in the SQL editor. If a table or column name includes spaces, wrap those identifiers in backticks in your SQL queries.

The following examples demonstrate how to query various table-like objects that you can store in a catalog.

Query a standard table or view

The following example queries a table from the `samples` catalog.

SQL

```
SELECT
  o_orderdate,
  o_orderkey,
  o_custkey,
  o_totalprice,
  o_shippriority
FROM
  samples.tpch.orders
```

Query a metric view

The following example queries a metric view that uses a table from the samples catalog as its source. It evaluates the three listed measures and aggregates over `Order Month` and `Order Status`. It returns results sorted by `Order Month`. To create a similar metric view in your workspace, see [Tutorial: Build a complete metric view with joins](#).

All measure evaluations must be wrapped in the `MEASURE` function. See [measure aggregate function](#)

SQL

```
SELECT
  `Order Month`,
  `Order Status`,
  MEASURE(`Order Count`),
  MEASURE(`Total Revenue`),
  MEASURE(`Total Revenue per Customer`)
FROM
  orders_metric_view
GROUP BY ALL
ORDER BY 1 ASC;
```

Turn autocomplete on and off

Live autocomplete can complete schema tokens, query syntax identifiers (like `SELECT` and `JOIN`), and the titles of [query snippets](#). It's enabled by default unless your database schema exceeds five thousand tokens (tables or columns).

 Ask Genie

Use the toggle beneath the SQL editor to turn live autocomplete off or on.

- To turn off live autocomplete, press **Ctrl + Space** or click the  button beneath the SQL editor.

Optimize a query with Genie Code

The `/optimize` slash command prompts Assistant to evaluate and optimize queries. For more information, see [Optimize Python, PySpark, and SQL code](#).

Save queries

The **Save** button near the top-right of the SQL editor saves your query.

ⓘ IMPORTANT

When you modify a query but don't explicitly click **Save**, that state is retained as a query draft. Query drafts are retained for 30 days. After 30 days, query drafts are automatically deleted. To retain your changes, you must explicitly save them.

Edit multiple queries

By default, the SQL editor uses tabs so you can edit multiple queries simultaneously. To open a new tab, click **+**, then select **Create new query** or **Open existing query**. Click **Open existing query** to see your list of saved queries. click **My Queries** or **Favorites** to filter the list of queries. In the row containing the query you want to view, click **Open**.

Open query

My queries

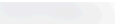
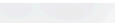
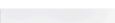
Favorites

Trash

All queries

Q

Search queries

Name	Created at 	
top ten accounts by country by 	2023-12-07 15:34	<button>Focus</button>
New query by 	2023-12-04 14:49	<button>Open</button>
Order Revenue quick-tag by 	2023-10-12 16:08	<button>Focus</button>
National Revenue Map Sample by 	2023-10-12 16:08	<button>Open</button>
TPCH - Number Suppliers Sample	2023-10-12 16:08	<button>Open</button>

Run a single query or multi-statement queries

To run a query statement or all query statements:

1. Select a SQL warehouse.
2. Highlight a query in the SQL editor (if multiple queries are in the query pane).
3. Press **Ctrl/Cmd + Enter** or click **Run (1000)** to display the results as a table in the results pane.

New query

Run (1000)

hive_meta.... def...

Serverless Starter W... (S)

Save*

Schedule

Share

```

1 SELECT year(birthDate) as birthYear, count(*) AS total
2 FROM default.people10m
3 WHERE firstName = 'Mary' AND gender = 'F'
4 GROUP BY birthYear
5 ORDER BY birthYear

```

Results

#	birthYear	total
1	1952	27
2	1953	25
3	1954	15
4	1955	23
5	1956	28
6	1957	29
7	1958	26
8	1959	22

1

2

>

19.28 s runtime | 49 rows

Refreshed 3 minutes ago

NOTE

By default, **Limit 1000** is selected to restrict query results to 1,000 rows.

- If the query is saved with **Limit 1000**, this setting applies to all query runs, including in dashboards.
- To return all rows (up to 64,000), deselect **Limit 1000** from the **Run (1000)** drop-down menu.
- To specify a different row limit, add a LIMIT clause to your SQL query.

The maximum number of rows returned in a table is 64,000.

Terminate a query

To terminate a query while it is running, click **Cancel**. An administrator can stop an running query that another user started by viewing the [Terminate an executing query](#).

Ask Genie

Query options

You can use the  kebab context menu near the top of the query editor to access menu options to clone, revert, format, and edit query information.

Revert to saved query

When you edit a query, a **Revert changes** option appears in the context menu for the query. You can click **Revert** to go back to your saved version.

Discarding and restoring queries

To move a query to trash:

- Click the kebab context menu  next to the query in the SQL editor and select **Move to Trash**.
- Click **Move to trash** to confirm.

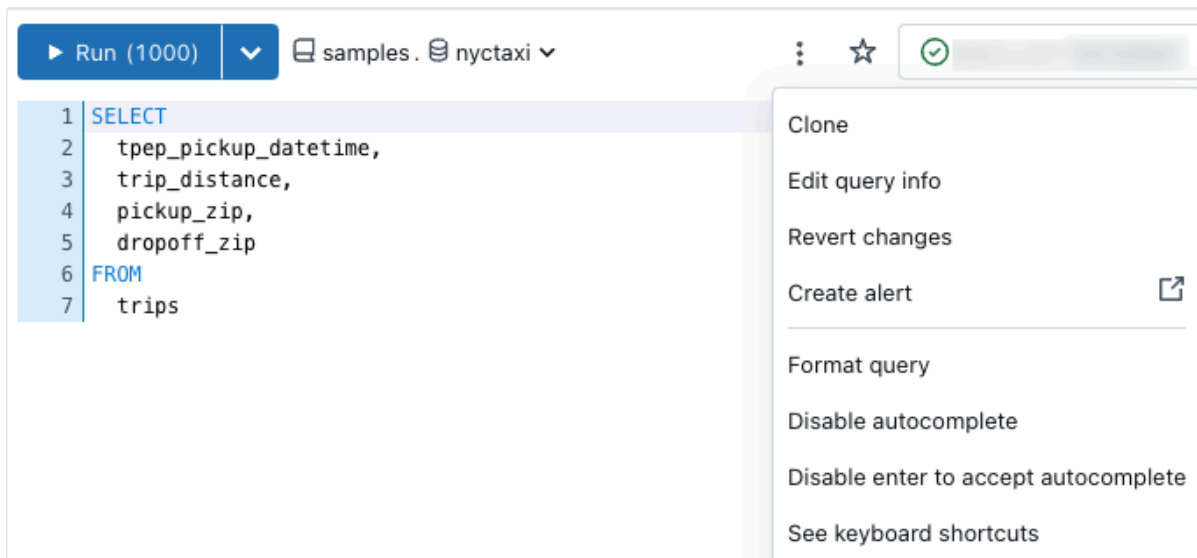
To restore a query from trash:

1. In the All Queries list, click .
2. Click a query.
3. Click the kebab context menu  at the top-right of the SQL editor and click **Restore**.

Set query description and view query info

To set a query description:

1. Click the  kebab context menu next to the query and click **Edit query info**.



2. In the **Description** text box, enter your description. Then, click **Save**. You can also view the history of the query, including when it was created and updated, in this dialog.

Favorite and tag queries

You can use favorites and tags to filter the lists of queries and dashboards displayed on your workspace landing page, and on each of the listing pages for dashboards and queries.

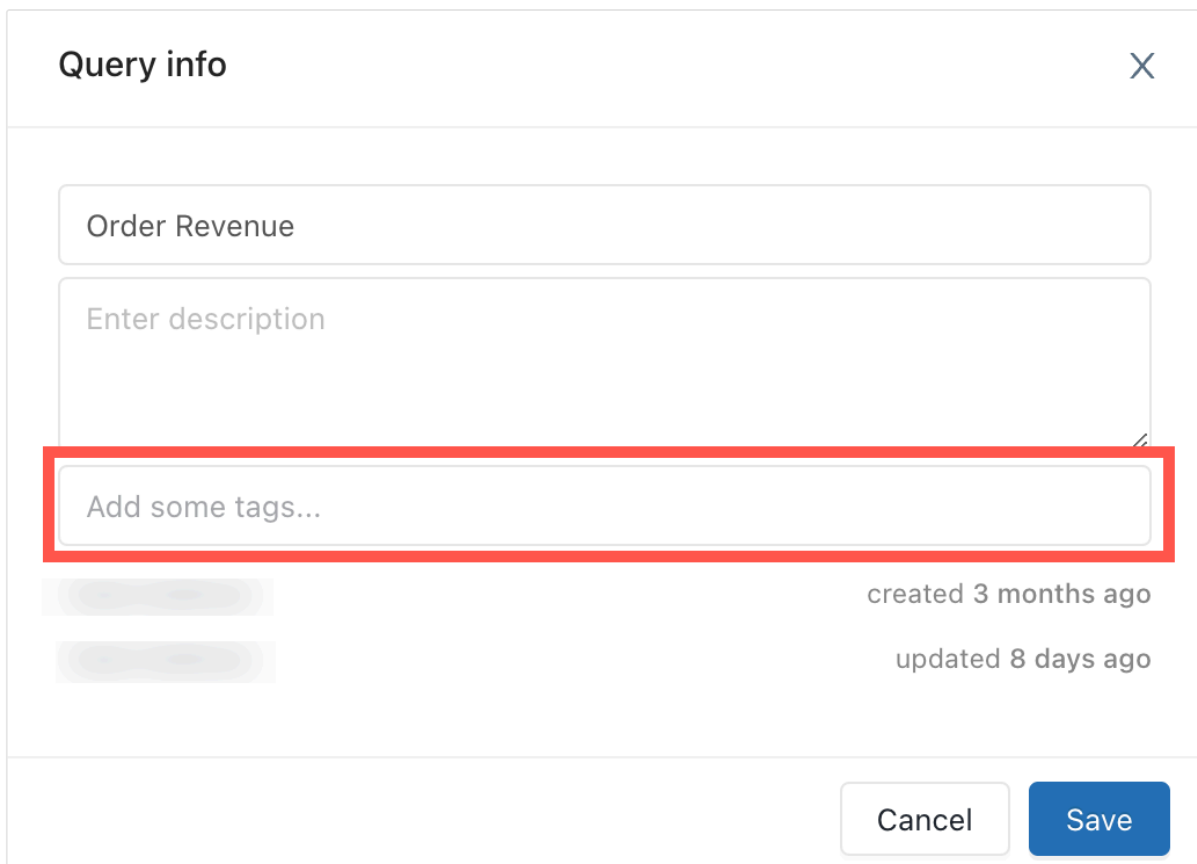
Favorites: To favorite a query, click the star to the left of its title in the Queries list. The star will turn yellow.

Tags: You can tag queries and dashboards with any meaningful string to your organization.

Add a tag

Add tags in the query editor.

1. Click the  kebab context menu next to the query and click **Edit query info**. A **Query info** dialog appears.
2. If the query has no tags applied, **Add some tags** shows in the text box where tags will appear. To create a new tag, type it into the box. To enter multiple tags, press  tab between entries.

A dialog box titled "Query info" with a close button (X) in the top right corner. It contains three input fields: "Order Revenue", "Enter description", and "Add some tags...". The "Add some tags..." field is highlighted with a red border. Below the input fields, there are two tags represented by rounded rectangles. The first tag has a close button (X) and the text "created 3 months ago". The second tag has a close button (X) and the text "updated 8 days ago". At the bottom right, there are two buttons: "Cancel" and "Save".

Query info

Order Revenue

Enter description

Add some tags...

created 3 months ago

updated 8 days ago

Cancel Save

3. Click **Save** to apply the tags and close the dialog.

Remove tags

1. Click the  kebab context menu next to the query and click **Edit query info**.
2. Click **X** on any tag you want to remove.
3. Click **Save** to close the dialog.

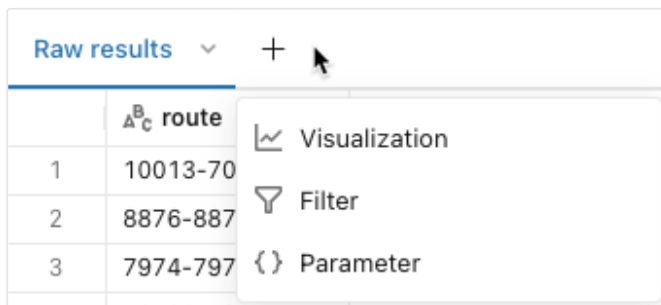
View query results

After a query runs, the results appear in the pane below it. The **New result table** is **ON** for new queries. If necessary, click the drop-down to turn it off. The images in this section use the new result table.

You can interact with and explore your query results using the result pane. The result pane includes the following features for exploring results.

Visualizations, filters, and parameters

Click the  to add a visualization, filter, or parameter. The following options appear:



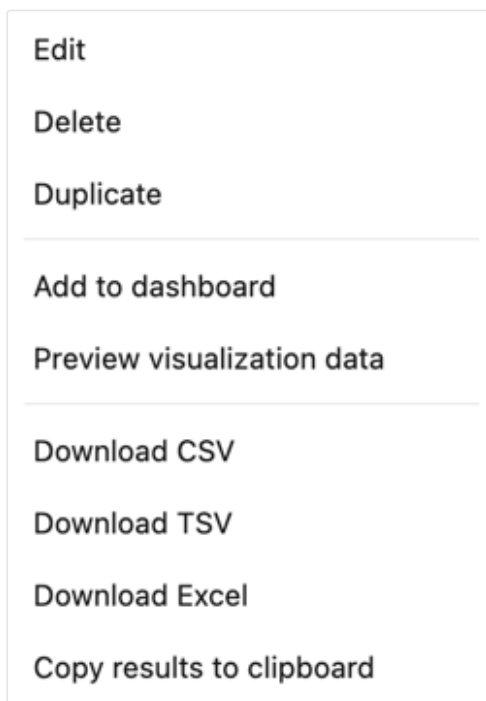
Visualization: Visualizations can help explore the result set. See [Notebook and SQL editor visualization types](#) for a complete list of available visualization types.

Filter: Filters allow you to limit the result set after a query has run. You can apply filters to selectively show different subsets of the data. See [Query filters](#) to learn how to use filters.

Parameter: Parameters allow you to limit the result set by substituting values into a query at runtime. See [Use named parameter markers](#) to learn how to apply parameters.

Edit, download, or add to a dashboard

Click the  in a results tab to view more options.



1. Click **Edit** to customize the results shown in the visualization.
2. Click **Delete** to delete the results tab.
3. Click **Duplicate** to clone the results tab.
4. Click **Add to dashboard** to add the visualization to an AI/BI dashboard. You are prompted to choose one of the following:
 - **Create new dashboard:** Enter a name for the new dashboard.
 - **Add to existing dashboard:** Search for an existing dashboard in the workspace.

In both cases, select the visualizations to include and optionally check **Automatically converts legacy parameter syntax to named parameter syntax, when possible** to convert legacy parameter syntax when adding to the dashboard. See [Dashboards](#) to learn how to edit your dashboard.

5. Click **Preview visualization data** to view the underlying data for the visualization.
6. Click any of the download options to download results. See the following description for details and limits.

Download results: You can download results as a CSV, TSV, or Excel file.

You can download up to approximately 1GB of results data from Databricks SQL in CSV and TSV format and up to 100,000 rows to an Excel file.

The final file download size might be slightly more or less than 1GB, as the 1GB limit is applied to an earlier step than the final file download.

NOTE

If you cannot download a query, your workspace administrator has disabled download for your workspace.

Past executions

You can view previous runs for the query, including the complete query syntax. Past executions open in read-only mode and include buttons to **Clone to new query** or **Resume editing**. This tab does not show [scheduled runs](#).



Past executions

Show past executions for:

Current query

 Current draft



```
select * from
samples.nyctaxi.trips
where trip_distance > 1 ...
```

rows, 5 ms, just now



```
select * from
samples.nyctaxi.trips
limit 500
```

500 rows, 1.86 s, a minute ago



```
select * from
samples.nyctaxi.trips
LIMIT 1000
```

1000 rows, 1.71 s, an hour ago

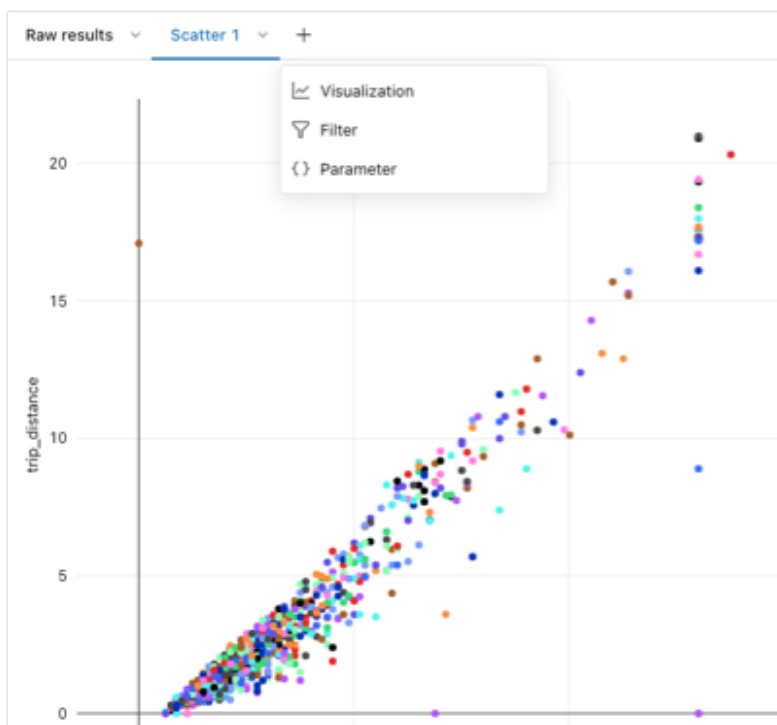
Explore results

Returned query results appear below the query. The **Raw results** tab populates with the returned rows. You can use built-in filters to reorder the results by ascending or descending values. You can also use the filter to search for result rows that include a specific value.

 Ask Genie

	tpep_pickup_datetime	tpep_dropoff_datetime	trip_distance	fare_amount	pickup_zip	dropoff_zip
1	2016-02-01T03:06:48.000+00:00	2016-02-01T03:12:09.000+00:00	1.7	7	10003	10002
2	2016-02-01T07:50:59.000+00:00	2016-02-01T07:55:03.000+00:00	0.56	4.5	10110	10110
3	2016-02-01T08:10:34.000+00:00	2016-02-01T08:14:36.000+00:00	1.1	5.5	10001	10011
4	2016-02-01T08:57:50.000+00:00	2016-02-01T09:05:34.000+00:00	0.9	6.5	10153	10021
5	2016-02-01T09:15:43.000+00:00	2016-02-01T09:20:49.000+00:00	0.55	5	10022	10153
6	2016-02-01T09:22:46.000+00:00	2016-02-01T09:24:26.000+00:00	0.3	3.5	10167	10022
7	2016-02-01T10:04:20.000+00:00	2016-02-01T10:17:12.000+00:00	1.5	10	10103	10199
8	2016-02-01T11:31:26.000+00:00	2016-02-01T11:42:33.000+00:00	1.6	9	10017	10199
9	2016-02-01T11:31:09.000+00:00	2016-02-01T11:47:28.000+00:00	5.15	18	10162	10032
10	2016-02-01T12:19:05.000+00:00	2016-02-01T12:34:19.000+00:00	3.92	14.5	11370	11106
11	2016-02-01T14:19:32.000+00:00	2016-02-01T14:29:56.000+00:00	3.8	13	10005	10016
12	2016-02-01T15:04:42.000+00:00	2016-02-01T15:13:08.000+00:00	1.05	7	10119	10110
13	2016-02-01T15:20:35.000+00:00	2016-02-01T15:30:45.000+00:00	1.2	8.5	10019	10171
14	2016-02-01T15:39:23.000+00:00	2016-02-01T15:46:28.000+00:00	1.1	6.5	10026	10035
15	2016-02-01T16:06:45.000+00:00	2016-02-01T16:15:31.000+00:00	1.18	7.5	10023	10021
16	2016-02-01T16:44:06.000+00:00	2016-02-01T17:07:44.000+00:00	3.49	17.5	10019	10009
17	2016-02-01T17:04:34.000+00:00	2016-02-01T17:09:40.000+00:00	0.63	5	10001	10001
18	2016-02-01T17:45:57.000+00:00	2016-02-01T18:03:33.000+00:00	6.2	20	10029	11369
19	2016-02-01T18:19:38.000+00:00	2016-02-01T18:27:40.000+00:00	1.2	7.5	10031	10030
20	2016-02-01T18:45:48.000+00:00	2016-02-01T19:02:25.000+00:00	2.88	13.5	10021	10018
21	2016-02-01T19:34:27.000+00:00	2016-02-01T19:37:41.000+00:00	0.8	4.5	10110	10153
22	2016-02-01T20:30:31.000+00:00	2016-02-01T20:42:59.000+00:00	2.33	10.5	10065	10009
23	2016-02-01T20:39:52.000+00:00	2016-02-01T20:52:12.000+00:00	3.35	13	10282	10001
24	2016-02-01T20:49:59.000+00:00	2016-02-01T20:55:07.000+00:00	0.7	5.5	10019	10153

You can use tabs in the result pane to add visualizations, filters, and parameters.



Filter the list of saved queries in the queries window

In the queries window, you can filter the list of all queries by the list of queries you have created (**My Queries**), by favorites, and by tags.

Automate updates

You can use the **Schedule** button to set an automatic cadence for query runs. Automatic updates can help keep your dashboards and reports up-to-date with the most current data. Scheduled queries can also enable Databricks SQL alerts, a special type of scheduled task that sends notifications when a value reaches a specified threshold.

See [Schedule a query](#).

See [Databricks SQL alerts](#).

Share queries

The **Share** button lets you share your query with other users in your workspace. When sharing, choose the between the following options:

- **Run as owner (owner's credentials):** This setting means that viewers are able to see the same query results as the query owner. This applies to scheduled or manual query runs.
- **Run as viewer (viewers credentials):** This setting limits results to the viewer's assigned permissions.

See [Configure query permissions](#).

Next step

See [Access and manage saved queries](#) to learn how to work with queries with the Databricks UI.

Is this page
as this page helpful?

☐ Yes	☐ No
☐ Send feedback	

Last updated on **Apr 29, 2026**

Mustache parameter syntax

❗ IMPORTANT

Mustache parameter syntax is supported in the legacy SQL editor only. Databricks recommends using [named parameter markers](#) for new queries. If you copy a query using mustache syntax into a notebook, AI/BI dashboard dataset editor, or Genie space, you must convert it to named parameter markers before it will run.

In the legacy SQL editor, any string wrapped in double curly braces (`{{ }}`) is treated as a query parameter. A widget appears above the results pane where you set the parameter value.

Add a mustache parameter

1. Press `Cmd + I`. The parameter is inserted at the cursor position and the **Add Parameter** dialog appears.
2. Enter the **Keyword**, optionally change the **Title**, and select a **Type**.
3. Click **Add Parameter**.
4. Set the parameter value in the widget.
5. Click **Apply Changes**.
6. Click **Save**.

Alternatively, type `{{ }}` directly in the query and click the gear icon in the parameter widget to configure it.

To re-run the query with a different value, update the widget and click **Apply Changes**.

Edit, remove, and reorder parameters

Edit: Click the gear icon beside the parameter widget. To prevent users who don't own the query from changing the parameter value, click **Show Results Only**.

Remove: Delete the parameter marker from the query. The widget is removed automatically.

Reorder: Click and drag parameter widgets to the desired order.

Widget types

The following widget types are available:

Text

Takes a string as input. Backslash, single, and double quotation marks are escaped automatically. Databricks adds quotation marks around the value.

SQL

```
SELECT * FROM samples.tpch.customer WHERE c_name = {{ name_param }}
```

Number

Takes a number as input.

SQL

```
SELECT * FROM users WHERE age = {{ number_param }}
```

Date and Time

Parameterizes date and timestamp values. Three options are available:

 Ask Genie

Type	Precision
Date	Day
Date and Time	Minute
Date and Time (with seconds)	Second

All date and time values are passed as string literals and must be wrapped in single quotation marks in your query:

SQL

```
SELECT * FROM usage_logs WHERE date = '{{ date_param }}'
```

When you select a **Range** option, Databricks creates two parameters using `.start` and `.end` suffixes:

SQL

```
SELECT * FROM usage_logs
WHERE modified_time > '{{ date_range.start }}'
AND modified_time < '{{ date_range.end }}'
```

NOTE

The Date Range widget returns correct results only for `DATE`-type columns. For timestamp columns, use a Date and Time Range widget.

Dynamic date values: Date widgets include a blue lightning bolt icon. Click it to select dynamic values such as `today`, `yesterday`, `this week`, `last week`, `last month`, or `last year`. These values update automatically.

IMPORTANT

Dynamic date values are not compatible with scheduled queries.

Dropdown List

Restricts input to a predefined static list. Supports both single-value and multi-value selection.

- **Single value:** Wrap the parameter in single quotation marks in the query.
- **Multi-value:** Enable **Allow multiple values** in the widget settings. Use the **Quotation** option to control whether values are wrapped in single quotes, double quotes, or none. Update your `WHERE` clause to use `IN`:

SQL

```
SELECT * FROM orders WHERE status IN ( {{ status_param }} )
```

With double quotation marks selected, the query resolves to: `WHERE status IN ("value1", "value2", "value3")`.

Dropdown List values are strings. To use dates or timestamps, enter them in the format your data source requires.

Query-Based Dropdown List

Populates dropdown choices from a saved query. Behaves like Dropdown List but the options are dynamic.

1. Under **Type** in the settings panel, select **Query Based Dropdown list**.
2. Click the **Query** field and select a saved query.

If the query returns more than one column, Databricks uses the first column. If the query returns columns named `name` and `value`, the widget displays the `name` column but passes the associated `value` to the query at runtime.

Example:

SQL

```
SELECT user_uuid AS 'value', username AS 'name' FROM users
```

 Ask Genie

value	name
1001	John Smith
1002	Jane Doe
1003	Bobby Tables

The value passed to the database at runtime is `1001`, `1002`, or `1003`, not the display name.

NOTE

Performance degrades if the source query returns a large number of records.

Is this page

as this page helpful?

☐ Yes

☐ No

☐ Send feedback

Last updated on **May 11, 2026**

Work with parameter widgets

When you add a [named parameter marker](#) to a query, Databricks displays a parameter widget in the UI. Widgets let users set parameter values without editing the query directly. You can configure each widget's type, title, and default value.

Parameter widgets are supported in the SQL editor, notebooks, AI/BI dashboards, and Genie Spaces, but behave differently across these surfaces. This page describes parameter widgets in the SQL editor. For other surfaces, see:

- [Work with dashboard parameters](#) for dashboards
- [Add query parameters](#) for Genie Spaces
- [Databricks widgets](#) for notebooks

In the SQL editor, any parameter type (String, Integer, Decimal, Date, Timestamp) can use any widget type.

Configure a parameter widget

1. Add a named parameter marker to your query. A widget appears in the UI.

2. Click the gear icon beside the widget to open the widget dialog.

Parameter name

Widget label

Widget type

Text ▾

Parameter type ⓘ

String ▾

Default parameter value

Remove parameter

3. Set the following fields:

- **Parameter name:** The parameter name as it appears in the query. If you change the parameter name, in the widget dialog, you must also change it in the query.
- **Widget label:** A string that described the widget
- **Widget type:** Controls how users input the value. See [Widget types](#) below.
- **Parameter type:** The parameter data type. See [Parameter types](#).

4. Click away from the widget dialog to save your changes.

Edit, remove, and reorder widgets

Edit: Click the gear icon beside the widget to reopen the settings pane.

Remove: Delete the parameter marker from the query. The widget is removed automatically.

Reorder: Use the drag handle to the left of a widget to reorder.

Widget types

Databricks supports the following widget types for query parameters:

Widget type	Description
Dropdown	Users must choose from a predefined list.
Combobox	Users can choose from a predefined list or type a custom value.
Text input	Accepts any free-form value with no suggestions.
Multiselect	Users can select more than one value from a predefined list.
Dynamic dropdown	Populates choices from a saved query instead of a static list.
Date and Timestamp range	Defines a start and end range using <code>.min</code> and <code>.max</code> parameters.

Text input

Accepts a free-form value directly from the user. Use this widget when no predefined options are needed.

SQL

```
SELECT * FROM samples.tpch.region WHERE r_name = :region_param
```

Dropdown

Presents a predefined list of values. Users must select from the list — free-form entry is not allowed. Enter allowed values in the settings pane, one per line.

SQL

```
SELECT * FROM samples.tpch.orders WHERE o_orderstatus = :status_param
```

To create a dropdown widget type:

1. Click the gear icon beside the `:status_param` widget.
2. Set **Widget type** to **Dropdown**.
3. Set **Parameter type** to **String**.
4. Enter values into the **Choices for parameter value** text entry field. Click **Add** or press enter between each value.

Combobox

Presents a predefined list of suggested values but also allows users to type a custom value not in the list. Use a combobox when common options are useful for convenience but you want to allow free-form input.

SQL

```
SELECT * FROM samples.tpch.part WHERE p_brand = :brand_param
```

Multiselect

Allows users to select more than one value from a predefined list. The selected values are passed to the query as a collection.

SQL

```
SELECT * FROM samples.nyctaxi.trips WHERE
  array_contains(
    TRANSFORM(SPLIT(:list_parameter, ','), s -> TRIM(s)),
    CAST(dropoff_zip AS STRING)
  )
```

To add options for a multiselect dropdown list:

1. Click the gear icon beside the `list_parameter` widget.
2. Set **Widget type** to **Multiselect**.
3. Set **Parameter type** to **String**.
4. Enter values into the **Choices for parameter value** text entry field. Click **Add** or press enter between each value.

Dynamic dropdown

NOTE

Dynamic dropdown widgets are available in the SQL editor only, not in notebooks.

Populates the list of choices from a saved query instead of a static list. As the underlying data changes, the available options update automatically.

To use a dynamic dropdown:

1. Create and save a query that returns the values you want in the dropdown:

SQL

```
SELECT DISTINCT c_mktsegment FROM samples.tpch.customer ORDER BY
c_mktsegment
```

2. In a new or existing query, add a named parameter marker:

SQL

 Ask Genie

```
SELECT c_custkey, c_name, c_acctbal
FROM samples.tpch.customer
WHERE c_mktsegment = :segment_param
```

3. Click the gear icon beside the `segment_param` widget.
4. Set **Widget type** to **Dynamic dropdown**.
5. Click the **Query** field to open the **Select existing query** dialog. Select the saved query from step 1, then click **Select**.
6. Choose a **Default parameter value**.
7. Click **Apply Changes**.

Date and Timestamp range

Date and Timestamp parameters support a **Range** widget type. When selected, Databricks creates two parameters using `.min` and `.max` suffixes to define the start and end of the range.

SQL

```
SELECT * FROM samples.nyctaxi.trips
WHERE tpep_pickup_datetime
BETWEEN CAST(:date_range_min AS TIMESTAMP) AND CAST(:date_range_max AS
TIMESTAMP)
```

Click the blue lightning bolt icon to select dynamic values such as `today`, `yesterday`, `this week`, `last week`, `last month`, or `last year`. These values update automatically.

ⓘ IMPORTANT

Dynamic date values are not compatible with scheduled queries.

Is this page

as this page helpful?

☐ Yes

☐ No

☐ Send feedback

Last updated on **Apr 29, 2026**

Access and manage saved queries

This article outlines how to use the Databricks UI to access and manage queries.

View queries

You can view queries using the following methods:

- Click  **Workspace** in the sidebar. Queries are viewable, by default, in the **Home** folder. Users can organize queries into folders in the workspace browser along with other Databricks objects.
- Click  **Queries** in the sidebar. Objects in the **Queries** windows are sorted in reverse chronological order by default. You can reorder the list by clicking the **Created at** column heading. Type into the **Filter queries** text box to filter by Name, Tag, or Owner.

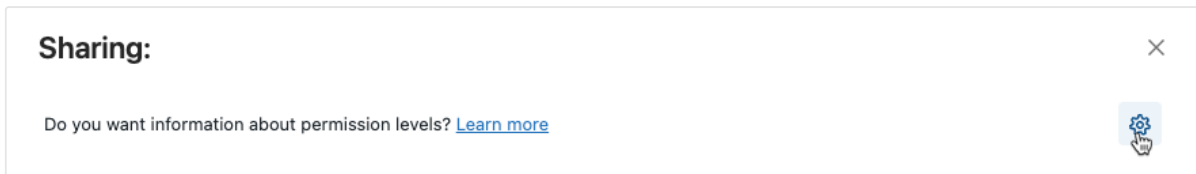
Organize queries into folders in the workspace browser

You can organize queries into folders in the [workspace browser](#) and other Databricks objects. See [Workspace browser](#).

Transfer ownership of a query

You must be a workspace admin to transfer ownership of a query. Service principals and groups cannot be assigned ownership of a query. You can also transfer ownership using the [Permissions API](#).

1. As a workspace admin, log in to your Databricks workspace.
2. In the sidebar, click **Queries**.
3. Click a query.
4. Click the **Share** button at the top right to open the **Sharing** dialog.
5. Click on the gear icon at the top right and click **Assign new owner**.



6. Select the user to assign ownership to.
7. Click **Confirm**.

Configure query permissions

Workspace admins and the query creator are automatically granted permissions to control which users can manage and run queries. You must have at least CAN MANAGE permission on a query to share queries.

Queries support two types of sharing settings:

- **Run as viewer:** The viewer's credential is used to run the query. The viewer must also have at least CAN USE permissions on the warehouse. In the legacy SQL editor, users can only be granted the CAN EDIT permission when the sharing setting is set to **Run as viewer**.
- **Run as owner:** In the new SQL editor, the owner's credential is used for legacy alerts and Lakeflow Jobs. The viewer's credential is always used to run the query. In the legacy SQL editor, the owner's credential is used to run the query.

For more information on query permission levels, see [Query ACLs](#).

1. In the sidebar, click **Queries**.
2. Click a query.

3. Click the [Share](#) button at the top right to open the **Sharing** dialog.

The screenshot shows the 'Sharing: Retail Revenue & Supply Chain' dialog. At the top, there's a title bar with a close button. Below it, a link to 'Learn more' about permission levels is available. A search bar prompts the user to 'Type to add multiple users or groups'. Below the search bar, two users are listed: a user with an email ending in '@databricks.com' with 'Can Manage (inherited)' permissions, and the 'Admins' group with 'Can Run (inherited)' permissions. A section titled 'Sharing settings' contains a 'Credentials' dropdown menu set to 'Run as owner (owner's credential)'. Below this is an orange warning box titled 'Permission adjustment' stating that when 'Run as owner' is selected, all viewers will see the latest results, and users with 'Can Manage' or 'Can Edit' permissions will be downgraded to 'Can Run'. At the bottom left, there is a 'Copy link' button.

4. Follow the steps based on the permission type you want to grant:

5. Search for and select the groups and users, and assign the permission level.

6. Click **Add**.

7. In the **Sharing settings > Credentials** field at the bottom, select either **Run as viewer** or **Run as owner**.

You can also copy the link to the query in the Sharing dialog.

Admin access to all queries

A Databricks workspace admin user has view access to all queries in the workspace. When the **All Queries** tab is selected, a workspace admin can view and delete any queries. However, a workspace admin can't edit a query when sharing setting credentials are set to **Run as owner**.

To view all queries:

1. Click  **Queries** in the sidebar.
2. Click the **All queries** tab near the top of the screen.

Creating queries in other environments

You can create queries without using the Databricks UI using the Rest API, a JDBC/ODBC connector, or a partner tool.

See [SQL connectors, libraries, drivers, APIs, and tools](#) to run SQL commands and browse database objects in Databricks.

You can also create a query with the [Databricks Terraform provider](#) and [databricks_sql_query](#).

See [Technology partners](#) to learn about partner tools you can use through Partner Connect.

Is this page

as this page helpful?

☐ Yes

☐ No

☐ Send feedback

Last updated on **Apr 29, 2026**

Query history

NOTE

If your workspace is enabled for serverless compute, your query history also contains all SQL and Python queries run on serverless compute for notebooks and jobs. See [Connect to serverless compute](#).

You can use the information available through this screen to help you debug issues with queries.

This section describes how to work with query history using the UI. To work with query history using the API, see [Query History](#).

View query history

You can view previous query runs if you are the query owner. Other users can view query runs if they have at least CAN VIEW access to the SQL warehouse that executed the query.

To view previous runs for a query:

1. Click  **Query History** in the sidebar.
2. Optionally, use the filters near the top of the page to filter the list by user, date range, compute, duration, query status, statement type, statement ID, and [query tags](#).

NOTE

Databricks SQL queries shared by a user with **Run as Owner** permissions to another user with CAN RUN permissions appear in the query history of the user executing the query and not the user that shared the query.

 Ask Genie

View query details

To view details about a query, such as the complete query statement, query source, duration, number of rows returned, and I/O performance, or [execution details](#).

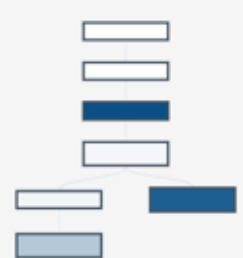
1. Access the query history as outlined in the previous section.
2. Click the query text you want to view. A summary panel opens to the right.

✔ Finished
New Query Profile: ON ▾
⋮ ×

· Serverless compute

ID: e802a963-ab26-428a-b726-3d7ebeaaaaa27
📄

```
SELECT
    o.o_orderkey,
...10 more lines
```

Bytes read 174.26 MB	▼ 79%	Bytes written 0	 See query profile >
Rows read 8		Rows written 0	
Files read 4	▼ 64%	Files written 0	

[See longest operations for this query](#)

Query wall-clock duration ⓘ

Total wall-clock duration	506 ms
---------------------------	---------------

Optimizing query & pruning files ⓘ
46%

Executing ⓘ
54%

235 ms
271 ms

Start time	3/20/2025, 03:43:53.885 PM
End time	3/20/2025, 03:43:54.391 PM
Result fetching by client ⓘ	62 ms

Query Source

> 🗂 New Query Profile Demo / 🧩 Cell(ID: 732...88187857298)

Aggregated task time ⓘ

Tasks total time	109 ms
Tasks time in Photon	71 %

[See query profile](#)

- **Query status:** The query is tagged with its current status: **Queued**, **Running**, **Finished**, **Failed**, or **Cancelled**.

- **User and compute details:** See the username, compute type, and runtime details for this query run.
- **ID:** This is the universally unique identifier (UUID) associated with the given query execution.
- **Query statement:** This section includes the complete query statement. If the query is too long to be shown in the preview, click ... **more lines** to view the full text.
- **Query metrics:** Popular metrics for query analysis are shown under the query text. The filter icons that appear with some metrics indicate the percent of data pruned during scanning.
- **See query profile:** A preview the query profile directed acyclic graph (DAG) is shown in this summary. This can be helpful for quickly estimating query complexity and execution flow. Click **See query profile** to open the detailed DAG.
- **See longest operators for this query:** Click this button to open the **Top operators** panel. This panel shows the longest running operators in the query.
- **Query wall-clock duration:** The total elapsed time between the start of scheduling and the end of the query execution is provided as a summary. A detailed breakdown of scheduling, query optimization and file pruning, and execution time appears below the summary.
- **Query source:** Click on the name of the listed object to go to the query source.
- **Aggregated task time:** View the combined time it took to execute the query across all cores of all nodes. It can be significantly longer than the wall-clock duration if multiple tasks are excuted in parallel. It can be shorter than the wall-clock duration if tasks waited for available nodes.
- **Input/Output (IO):** View details about the data read and written during the query execution.

3. For more detailed information about the query's performance, including its execution plan, click **View Query Profile** near the bottom of the page. For more details, see [Query profile](#).

Terminate an executing query



To terminate a long-running query started by you or another user:

1. Access the query history as outlined in the previous section.
2. Click the query text.
3. Next to **Status**, click **Cancel**.

NOTE

Cancel only appears when a query is running.

The query is terminated, and its status changes to **Canceled**.

NOTE

Statements that use Lakeflow Spark Declarative Pipelines compute can only be canceled from the Pipelines UI.

Access the query history system table

Databricks system tables include a query history table (`system.query.history`) where privileged users can view tabular query history records from across your account. By default, only admins have access to your account's system tables. For information on enabling and accessing the query history system table, see [Query history system table reference](#).

Is this page

as this page helpful?

☐ Yes

☐ No

☐ Send feedback

Last updated on **May 11, 2026**

Query tags

⚠️ PREVIEW

This feature is in [Public Preview](#).

This page describes how to use query tags to group, filter, and attribute costs for SQL workloads on Databricks SQL warehouses.

Query tags are custom key-value pairs (for example, `team:marketing` or `dbt_model_name:some_model_name`) that you apply to SQL workloads. These tags appear in the `system.query.history` table and on the **Query History** page of the Databricks UI, and are returned in responses to the [ListQueries API](#) if non-empty. Query tags allow you to group queries by business context, attribute warehouse costs, and identify sources of long-running queries.

⚠️ WARNING

Tag data is stored as plain text and might be replicated globally. Don't use tag keys or values that contain passwords, personal information, or other sensitive data.

Requirements

Before using query tags, verify the following:

- Confirm you can access the `system.query.history` table, specifically the `query_tags` column. If not, contact your account admin. See [Query history system table reference](#).
- Minimum version requirements based on where you set query tags:
 - **dbt:** dbt-databricks 1.11.0

- **Power BI:** October 2025 release
- **Python Connector:** v4.1.3 (session-level), v4.2.6 (statement-level)
- **Node.js Connector:** v1.12.0
- **Go Connector:** v1.9.0
- **JDBC driver (OSS):** v3.0.3

Query tag scope

Query tags are scoped to [Databricks SQL sessions](#). You can set them when creating a session as a configuration parameter, or within a session using the `SET QUERY_TAGS` SQL statement. After you set tags, all subsequent statements in the session are associated with those tags.

Set query tags

You can set query tags using session configuration parameters or SQL statements.

Use session configuration parameters

Set query tags using the `query_tags` configuration parameter (or `ssp_query_tags` in some drivers) when creating a session. The value is a serialized set of key-value pairs, with a colon (:) separating keys and values, and a comma (,) separating pairs. This string format is accepted by the client interfaces listed in [Set query tags from tools and interfaces](#). For instructions on configuring sessions in your specific client, see the examples below.

If a colon (:), comma (,), or backslash (\) appears in a value, escape it with a leading backslash (\): `\\:`, `\\,`, or `\\\\`. Backslash (\) must be escaped in both keys and values.

The following example specifies tags `team=eng`, `cost_center=701`, a key-only tag `exp`, and a `metadata` tag with a JSON blob:

Text

 Ask Genie

```
query_tags = team:eng,cost_center:701,exp,metadata:
{"foo"\:"bar"\,"baz"\:1}
```

Use SQL statements

Use the `SET QUERY_TAGS` statement to set, read, or remove query tags for the current session.

For syntax, parameters, and examples, see [SET QUERY_TAGS](#).

Set query tags from tools and interfaces

Databricks UI

Use the `SET QUERY_TAGS` SQL statement anywhere you can submit SQL to a warehouse, including the SQL editor, notebooks, and dashboards. See [SET QUERY_TAGS](#).

Databricks SQL Statement Execution API (SEA)

Include the `query_tags` field in the request body of [POST /api/2.0/sql/statements](#) to apply statement-level tags. Tags set this way apply only to that specific statement execution.

Bash

```
curl -X POST "https://${DATABRICKS_HOST}/api/2.0/sql/statements" \
-H "Authorization: Bearer ${DATABRICKS_TOKEN}" \
-H "Content-Type: application/json" \
-d '{
  "warehouse_id": "abc123",
  "statement": "SELECT * FROM samples.nyctaxi.trips LIMIT 10",
  "query_tags": [
    { "key": "team", "value": "engineering" },
    { "key": "env", "value": "prod" }
```

 Ask Genie

```
}'
```

dbt

Minimum version: dbt-databricks 1.11.0

The following reserved query tag keys are set automatically for all dbt runs and can't be overridden:

JSON

```
{
  "dbt_model_name": "my_model",
  "dbt_core_version": "1.10.7",
  "dbt_databricks_version": "1.11.0",
  "dbt_materialized": "incremental"
}
```

NOTE

Default tags are subject to syntax limitations. Model names that include unescaped colons or commas result in `tag_invalid: true` in the system table.

You can add custom query tags at both project and model levels. Model configuration takes priority over connection configuration.

Project-level tag (`~/.dbt/profiles.yml`):

YAML

```
your_profile_name:
  target: dev
  outputs:
    dev:
      query_tags: '{"team": "marketing", "cost_center": "3000"}'
```

Model-level tag (`~/.dbt/dbt_project.yml`):

YAML

```
name: 'your_project'
version: '1.0.0'
config-version: 2
models:
  your_model:
    +query_tags: '{"team": "content-marketing"}'
```

Result with both configurations:

JSON

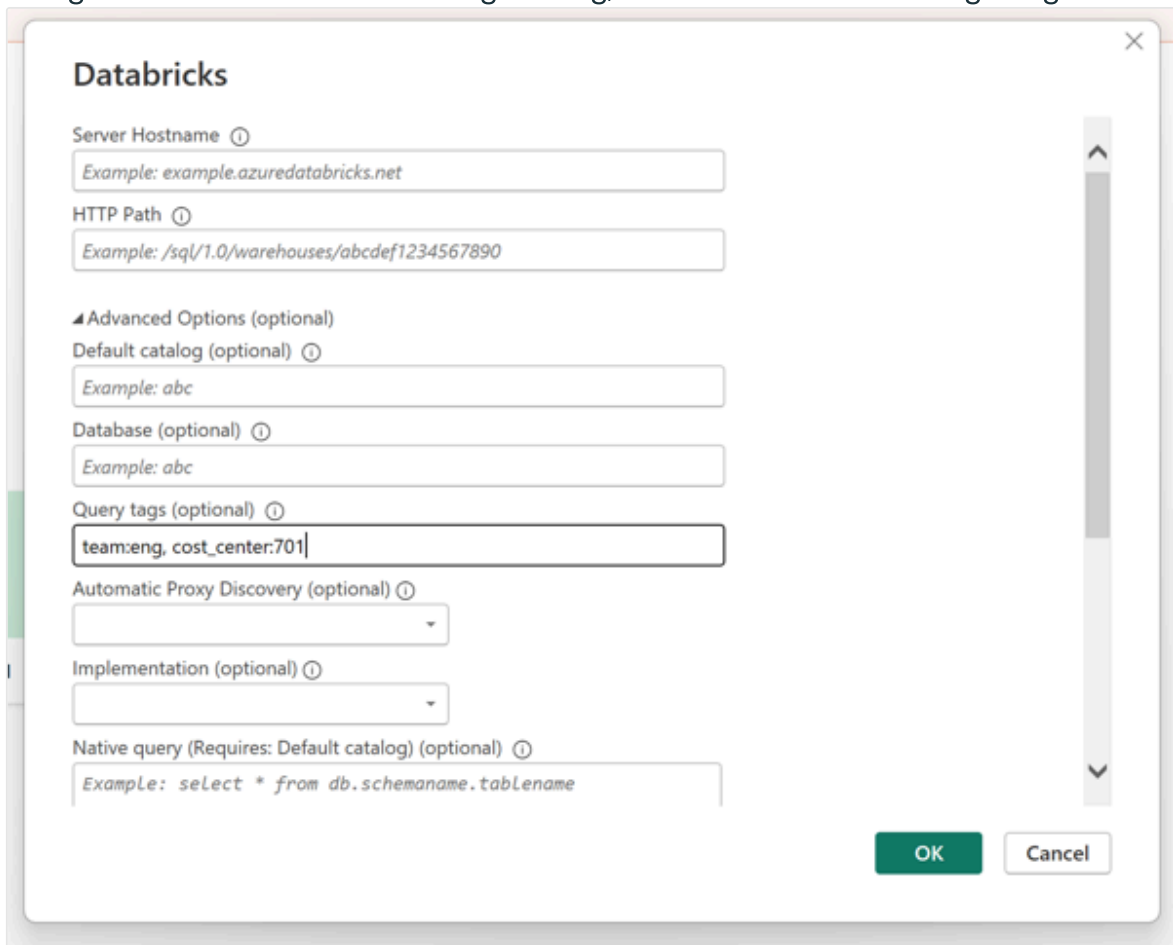
```
{
  "team": "content-marketing",
  "cost_center": "3000",
  "dbt_model_name": "model.dev.your_model",
  "dbt_core_version": "1.10.7",
  "dbt_databricks_version": "1.11.0",
  "dbt_materialized": "incremental"
}
```

Power BI

Minimum version: October 2025 release

1. Configure the connection to the warehouse.

2. Navigate to the Databricks settings dialog, as shown in the following image.



The screenshot shows a 'Databricks' settings dialog box with a close button (X) in the top right corner. The dialog contains several input fields and a section for advanced options. The fields are: 'Server Hostname' with an example 'example.azuredatabricks.net'; 'HTTP Path' with an example '/sql/1.0/warehouses/abcdef1234567890'; 'Advanced Options (optional)' which is expanded to show 'Default catalog (optional)' with example 'abc', 'Database (optional)' with example 'abc', 'Query tags (optional)' with the value 'team:eng, cost_center:701', 'Automatic Proxy Discovery (optional)' with a dropdown arrow, 'Implementation (optional)' with a dropdown arrow, and 'Native query (Requires: Default catalog) (optional)' with an example 'select * from db.schemaname.tablename'. At the bottom right are 'OK' and 'Cancel' buttons.

3. In the **Query tags** text box, enter query tags using the [session configuration parameter syntax](#).

4. Click **OK**.

NOTE

Changing query tags and clicking **OK** starts a new session. Previously set tags are discarded.

Enable auto query tags

Auto query tags automatically attach Power BI context metadata to queries sent to Databricks warehouses without requiring you to manually specify tag values. To enable auto query tags, modify the M query in the Power Query Editor to include `EnableAutoQueryTags="true"` in the connection parameters.

1. In Power BI Desktop or the Power BI service, find the semantic model that includes the query you want to tag.

2. Open the **Power Query Editor**.
3. Open the **Advanced Editor**.
4. Add `EnableAutoQueryTags="true"` to the options in the `DatabricksMultiCloud.Catalogs` call.

The following example shows an M query with auto query tags enabled:

Powerquery

```
let
    Source = DatabricksMultiCloud.Catalogs(
        "myworkspace.cloud.databricks.com",
        "/sql/1.0/warehouses/abc123",
        [Catalog=null, Database=null, EnableAutoQueryTags="true",
        EnableAutomaticProxyDiscovery=null, Implementation="2.0"]),
    samples_Database = Source{[Name="samples",Kind="Database"]}[Data],
    nyctaxi_Schema = samples_Database{[Name="nyctaxi",Kind="Schema"]}[Data],
    trips_Table = nyctaxi_Schema{[Name="trips",Kind="Table"]}[Data]
in
    trips_Table
```

When auto query tags are enabled, Power BI automatically attaches the following reserved tags (prefixed with @@) to queries. The tags available depend on the Power BI connection mode:

Tag key	DirectQuery	Import
@@powerbi_activity_id	Yes	Yes
@@powerbi_dataset_id	Yes	No
@@powerbi_report_id	Yes	No
@@powerbi_visual_id	Yes	No

NOTE

 Ask Genie

Auto query tags are not attached to metadata queries (such as catalog or schema discovery queries) in any mode.

Parameterize the setting

If your semantic model contains multiple tables, Databricks recommends defining `EnableAutoQueryTags` as a Power Query parameter. This lets you enable or disable auto query tags across all queries from a single location instead of editing each query individually.

1. In the **Power Query Editor**, select **Manage Parameters** > **New Parameter**.
2. Create a parameter named `EnableAutoQueryTags` with a type of **Text** and a current value of `"true"`.
3. In each query, replace the hardcoded `EnableAutoQueryTags="true"` with a reference to the parameter.

The following example shows an M query that references the parameter:

Powerquery

```
let
    Source = DatabricksMultiCloud.Catalogs(
        "myworkspace.cloud.databricks.com",
        "/sql/1.0/warehouses/abc123",
        [Catalog=null, Database=null,
        EnableAutoQueryTags=EnableAutoQueryTags,
        EnableAutomaticProxyDiscovery=null, Implementation="2.0"]),
    samples_Database = Source{[Name="samples",Kind="Database"]}[Data],
    nyctaxi_Schema = samples_Database{[Name="nyctaxi",Kind="Schema"]}[Data],
    trips_Table = nyctaxi_Schema{[Name="trips",Kind="Table"]}[Data]
in
    trips_Table
```

With this approach, you can toggle auto query tags for all queries in your semantic model by updating the parameter value once, without editing individual queries.

Tableau

1. Configure the connection to the warehouse.

2. Navigate to the **Initial SQL** tab.
3. Enter query tags using the [SET QUERY_TAGS](#). You can include Tableau parameters in the key or value.
4. Click **Sign In** to save and authenticate.

NOTE

Changing the Initial SQL and clicking **Sign In** starts a new session. Previously set tags are discarded.

Python Connector

Minimum version: v4.1.3 (session-level), v4.2.6 (statement-level)

Session-level tags:

Python

```
from databricks import sql
import os

with sql.connect(
    server_hostname = os.getenv("DATABRICKS_SERVER_HOSTNAME"),
    http_path       = os.getenv("DATABRICKS_HTTP_PATH"),
    access_token    = os.getenv("DATABRICKS_TOKEN"),
    query_tags = {"team": "engineering", "dashboard": "abc123"}
) as connection:
    with connection.cursor() as cursor:
        cursor.execute("SELECT * FROM samples.nyctaxi.trips LIMIT 10")
        result = cursor.fetchall()
```

Statement-level tags:

Python

```
from databricks import sql
import os

with sql.connect(
    server_hostname = os.getenv("DATABRICKS_SERVER_HOSTNAME"),  Ask Genie
    http_path       = os.getenv("DATABRICKS_HTTP_PATH"),
```

```

    access_token    = os.getenv("DATABRICKS_TOKEN"),
) as connection:
    with connection.cursor() as cursor:
        cursor.execute(
            "SELECT * FROM samples.nyctaxi.trips LIMIT 10",
            query_tags={"team": "engineering", "dashboard": "abc123"}
        )
    result = cursor.fetchall()

```

Node.js Connector

Minimum version: v1.12.0

JavaScript

```

const { DBSQLClient } = require('@databricks/sql');
const client = new DBSQLClient();

client
    .connect({
        host: process.env.DATABRICKS_SERVER_HOSTNAME,
        path: process.env.DATABRICKS_HTTP_PATH,
        token: process.env.DATABRICKS_TOKEN,
    })
    .then(async (client) => {
        const session = await client.openSession({
            configuration: {
                query_tags: 'team:engineering,env:prod',
            },
        });

        const queryOperation = await session.executeStatement('SELECT * FROM
samples.nyctaxi.trips LIMIT 10');
        const result = await queryOperation.fetchAll();

        await queryOperation.close();
        await session.close();
        await client.close();
    })
    .catch((error) => {
        console.error(error);
    });

```

Go Connector

 Ask Genie

Minimum version: v1.9.0

DSN connection string:

Go

```
package main

import (
    "database/sql"
    "fmt"
    _ "github.com/databricks/databricks-sql-go"
)

func main() {
    dsn :=
        "token:dapi1234@myworkspace.cloud.databricks.com:443/sql/1.0/endpoints/abc12
        query_tags=team:engineering,env:prod"

    db, err := sql.Open("databricks", dsn)
    if err != nil {
        panic(err)
    }
    defer db.Close()

    rows, err := db.Query("SELECT * FROM samples.nyctaxi.trips LIMIT 10")
    if err != nil {
        panic(err)
    }
    defer rows.Close()
}
```

NewConnector with WithSessionParams:

Go

```
package main

import (
    "database/sql"
    "os"
    dbsql "github.com/databricks/databricks-sql-go"
)

func main() {
```

 Ask Genie

```

connector, err := dbsql.NewConnector(
    dbsql.WithAccessToken(os.Getenv("DATABRICKS_ACCESS_TOKEN")),
    dbsql.WithServerHostname(os.Getenv("DATABRICKS_HOST")),
    dbsql.WithPort(443),
    dbsql.WithHTTPPath(os.Getenv("DATABRICKS_HTTP_PATH")),
    dbsql.WithSessionParams(map[string]string{
        "query_tags": "team:engineering,env:prod",
    }),
)
if err != nil {
    panic(err)
}

db := sql.OpenDB(connector)
defer db.Close()

rows, err := db.Query("SELECT * FROM samples.nyctaxi.trips LIMIT 10")
if err != nil {
    panic(err)
}
defer rows.Close()
}

```

JDBC driver (OSS)

Minimum version: v3.0.3

Connection URL:

Java

```

String url =
    "jdbc:databricks://myworkspace.cloud.databricks.com:443/default;" +
        "httpPath=/sql/1.0/endpoints/abc123;" +
        "query_tags=team:engineering,env:prod;" +
        "AuthMech=3;UID=token;PWD=dapi1234";

Connection conn = DriverManager.getConnection(url);
Statement stmt = conn.createStatement();
ResultSet rs = stmt.executeQuery("SELECT * FROM samples.nyctaxi.trips
LIMIT 10");

```

Properties object:

Java

```
String url =
"jdbc:databricks://myworkspace.cloud.databricks.com:443/default";
Properties properties = new Properties();
properties.put("httpPath", "/sql/1.0/endpoints/abc123");
properties.put("query_tags", "team:engineering,env:prod");
properties.put("AuthMech", "3");
properties.put("UID", "token");
properties.put("PWD", "dapi1234");

Connection conn = DriverManager.getConnection(url, properties);
Statement stmt = conn.createStatement();
ResultSet rs = stmt.executeQuery("SELECT * FROM samples.nyctaxi.trips
LIMIT 10");
```

JDBC driver (Simba)

Connection URL:

Java

```
String url = "jdbc:databricks://myworkspace.cloud.databricks.com:443;" +
"httpPath=/sql/1.0/endpoints/abc123;" +
"sdp_query_tags=team:engineering,env:prod;" +
"AuthMech=3;UID=token;PWD=dapi1234";

Connection conn = DriverManager.getConnection(url);
Statement stmt = conn.createStatement();
ResultSet rs = stmt.executeQuery("SELECT * FROM samples.nyctaxi.trips
LIMIT 10");
```

Properties object:

Java

```
String url = "jdbc:databricks://myworkspace.cloud.databricks.com:443";
Properties properties = new Properties();
properties.put("httpPath", "/sql/1.0/endpoints/abc123");
properties.put("sdp_query_tags", "team:engineering,env:prod");
properties.put("AuthMech", "3");
properties.put("UID", "token");
```

 Ask Genie

```
properties.put("PWD", "dapi1234");

Connection conn = DriverManager.getConnection(url, properties);
Statement stmt = conn.createStatement();
ResultSet rs = stmt.executeQuery("SELECT * FROM samples.nyctaxi.trips
LIMIT 10");
```

ODBC driver

Include the `ssp_query_tags` parameter in your ODBC connection configuration. You must also set `ApplySSPWithQueries=0` in your ODBC driver connection configuration.

View query tags

Query the `system.query.history` table to view query tags. You can group and filter by specific keys or key-value pairs.

SQL

```
SELECT statement_id, query_tags, executed_by, start_time
FROM system.query.history
WHERE MAP_CONTAINS_KEY(query_tags, 'team')
      AND query_tags['team'] = 'engineering'
ORDER BY start_time DESC
LIMIT 100;
```

NOTE

Key-only tags appear with a `null` value. To filter for them:

SQL

```
WHERE MAP_CONTAINS_KEY(query_tags, 'key') AND query_tags['key'] IS
NULL
```

For more information, see [Query history system table reference](#).

Limitations

- Query tags are limited to 10 KB per session. If the total size exceeds this limit, incoming query tags are dropped and a `tags_dropped: true` sentinel tag is added.
- Each query supports a maximum of 20 user-specified tags. When using non-SQL interfaces (session configuration parameters), additional tags are discarded and a `tag_truncated: true` sentinel tag is added.
- Tag keys and values must not exceed 128 characters. When using non-SQL interfaces (session configuration parameters), invalid tags are discarded and `tag_invalid` is added.
- Tag keys must not contain the characters `,`, `:`, `-`, `/`, `=`, or `.`. When using non-SQL interfaces (session configuration parameters), invalid tags are discarded and `tag_invalid` is added. SQL statements with invalid tag keys fail with an error at execution time.
- Query tags are supported for Databricks SQL workloads only. The `query_tags` column is not populated for other compute types.
- Keys starting with `@@` are reserved for internal use. Avoid this prefix to prevent conflicts.

Is this page

as this page helpful?

☐ Yes

☐ No

Last updated on **Apr 29, 2026**

What are SQL warehouse sessions?

SQL warehouse sessions allow you to define variables, create temporary views, and maintain state changes across multiple query runs. With sessions, you can build SQL logic iteratively without needing to run all statements at once.

You can use sessions in the following contexts when attached to a SQL warehouse:

- [SQL editor](#) queries
- [Notebooks](#)
- [Workspace](#) `.sql` files

Example use case

Sessions enable you to run statements individually while maintaining shared state. For example, you can declare a variable, create a temporary view that uses that variable, and then query the view separately.

The following example demonstrates this workflow:

SQL

```
-- Statement 1: Declare a variable
DECLARE VARIABLE row_limit = 10;

-- Statement 2: Create a temporary view using the variable
CREATE OR REPLACE TEMPORARY VIEW filtered_data AS
SELECT
  *
FROM
  main.default.sales_data
LIMIT row_limit;
```



Ask Genie

```
-- Statement 3: Query the temporary view
SELECT
  *
FROM
  filtered_data;
```

With sessions, you can run each statement individually. The variable and temporary view remain available, allowing you to iterate on your queries without re-running all previous statements.

How sessions work

A session is created when a user runs a query on a SQL warehouse. Sessions remain active as long as a command runs at least once every eight hours. Sessions persist even if the warehouse stops or restarts.

A session expires after eight hours of inactivity.

Shared session state

All commands run within the same session share the following:

- Variables declared with `DECLARE VARIABLE`
- Temporary views created with `CREATE TEMPORARY VIEW`
- Temporary tables created with `CREATE TEMPORARY TABLE`. See [Temporary tables](#)
- Environment settings such as the current catalog and schema
- Query tags set with `SET QUERY_TAGS` or session configuration parameters. See [Query tags](#)

Session scope and collaboration

Sessions are tied to both the query object (query, notebook, or file) and the specific SQL warehouse it is attached to. All users who share access to that object and warehouse connection also share the same session.

For example:

- User A defines a temporary view in Query 1 while connected to Warehouse X.
- User B can run a SQL command in Query 1 on Warehouse X that references that same temporary view.
- If Query 1 is reattached to a different warehouse, a new session is created with its own isolated state.

This collaborative behavior allows teams to work together on shared query development, but it also means that changes made by one user affect all other users sharing the same session.

Next steps

- [SQL editor](#)
- [Connect a notebook to a SQL warehouse](#)
- [Workspace files](#)

Is this page

as this page helpful?

☐ Yes

☐ No

☐ Send feedback

Last updated on **Apr 29, 2026**

Query caching

Caching is an essential technique for improving the performance of data warehouse systems by avoiding the need to recompute or fetch the same data multiple times. In Databricks SQL, caching can significantly speed up query execution and minimize warehouse usage, resulting in lower costs and more efficient resource utilization. Each caching layer improves query performance, minimizes cluster usage, and optimizes resource utilization for a seamless data warehouse experience.

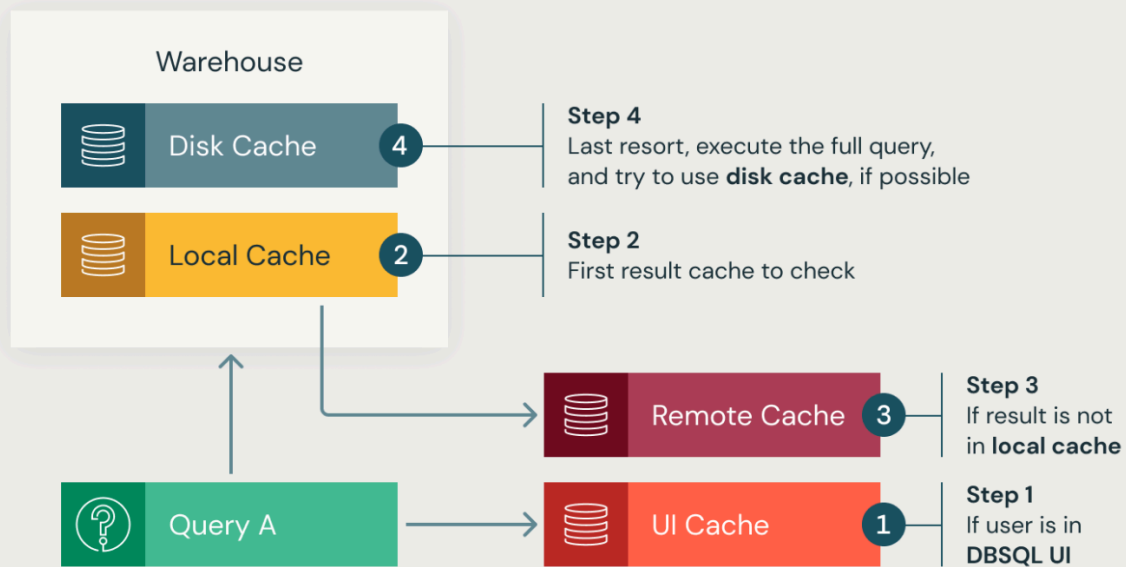
Caching provides numerous advantages in data warehouses, including:

- **Speed:** By storing query results or frequently accessed data in memory or other fast storage mediums, caching can dramatically reduce query execution times. This storage is particularly beneficial for repetitive queries, as the system can quickly retrieve the cached results instead of recomputing them.
- **Reduced cluster usage:** Caching minimizes the need for additional compute resources by reusing previously computed results. This reduces the overall warehouse uptime and the demand for additional compute clusters, leading to cost savings and better resource allocation.

Types of query caches in Databricks SQL

Databricks SQL performs several types of query caching.

Types of Caches



- **Databricks SQL UI cache:** Per user caching of all query and dashboard results in the Databricks SQL UI. When users first open a dashboard or SQL query, the Databricks SQL UI cache displays the most recent query result, including the results from scheduled executions.

The Databricks SQL UI cache has at most a 7-day life cycle. The cache is located within your Databricks filesystem in your account. You can delete query results by re-running the query that you no longer want to be stored. Once re-run, the old query results are removed from cache. Additionally, the cache is invalidated once the underlying tables have been updated.

- **Result cache:** Per cluster caching of query results for all queries through SQL warehouses. Result caching includes both local and remote result caches, which work together to improve query performance by storing query results in memory or remote storage mediums.
 - **Local cache:** The local cache is an in-memory cache that stores query results for the cluster's lifetime or until the cache is full, whichever comes first. This cache is useful for speeding up repetitive queries, eliminating the need to recompute the same results. However, once the cluster is stopped or restarted, the cache is cleaned and all query results are removed. 🧙 Ask Genie

- **Remote result cache:** The remote result cache is a serverless-only cache system that retains query results by persisting them as workspace system data. As a result, this cache is not invalidated by the stopping or restarting of a SQL warehouse. Remote result cache addresses a common pain point in caching query results in-memory, which only remains available as long as the compute resources are running. The remote cache is a persistent shared cache across all warehouses in a Databricks workspace.

Accessing the remote result cache requires a running warehouse. When processing a query, a cluster first looks in its local cache and then looks in the remote result cache if necessary. Only if the query result isn't cached in either cache is the query executed. Both the local and the remote caches have a life cycle of 24 hours, which starts at cache entry. The remote result cache persists through the stopping or restarting of a SQL warehouse. Both caches are invalidated when the underlying tables are updated.

Remote result cache is available for queries using ODBC / JDBC clients and SQL Statement API.

To disable query result caching, you can run `SET use_cached_result = false` in the SQL editor.

❗ IMPORTANT

You should use this option only in testing or benchmarking.

- **Disk cache:** Local SSD caching for data read from data storage for queries through SQL warehouses. The disk cache is designed to enhance query performance by storing data on disk, allowing for accelerated data reads. Data is automatically cached when files are fetched, utilizing a fast intermediate format. By storing copies of the files on the local storage attached to compute nodes, the disk cache ensures the data is located closer to the workers, resulting in improved query performance. See [Optimize performance with caching on Databricks](#).

In addition to its primary function, the disk cache automatically detects changes to the underlying data files. When it detects changes, the cache is invalidated. The disk cache shares the same lifecycle characteristics as the local result cache. This means that when the cluster is stopped or restarted, the cache is cleaned and needs to be repopulated.

The query results caching and [disk cache](#) affect queries in the Databricks SQL [UI](#) and [BI](#) and other external clients.

Is this page

as this page helpful?

 Yes

 No

 Send feedback

Last updated on **Apr 29, 2026**

Query filters

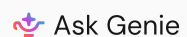
A query filter lets you interactively reduce the amount of data shown in a visualization. Query filters are similar to query parameter but with a few key differences. A query filter limits data *after* the query has been executed. This makes filters ideal for smaller datasets and environments where query executions are time-consuming, rate-limited, or costly.

The following describes some benefits of Databricks SQL.

- While previous query filters operated client-side only, these updated filters work dynamically on either client- or server-side to optimize performance.
- Simplified UI experience: click the **+Add filter** button and select a column from a dropdown to add a filter. You don't need to author, permission, and refresh a separate query in order to filter on the distinct values of a column.
- Enable “highlight relevant values” to see which selections within a filter will return results given other filter selections. For example, consider a user who has both a “State” and “City” filter. If a user chooses to highlight relevant values, selecting “California” in the state filter will highlight only the cities in California in the “City” filter. Non-highlighted options are put under a “Filtered out” menu option in the dropdown.
- Text Input filters: filters column results based on text input searches. There are three modes the search can find matches with: exact match, contains, and starts with.
- Quick date selectors enable you to filter on predefined date ranges such as last week, last month, last year, and more.
- You can set default date ranges when creating date filters.

Description of functionality

After running a query, in the **Results** panel, click **+** and then select **Filter**



The **+Add filter** button opens a popup menu where you can apply the following filters and settings.

- Column: the column on which to apply the filter.
 - Strings, numbers, and dates are currently supported.
 - If the selected column contains dates, users can choose to specify a time binning by date, month, or year.
- Type: the type of filter to apply
 - Single Select: filter to one field value only
 - Multi Select: filter to multiple field values
 - Text Input: enter a string to search for matching values in a particular column. Supports "Contains," "Exact Match," and "Starts With"
 - Date / time picker and ranger picker: ability to select a date or date range to filter on date data types
- Sort Order: The order in which values are sorted for the filter dropdown. You can choose between "Ascending" or "Descending."
- Highlight relevant values (y/n): Enable this to easily see which selections within a filter will return results given other filter selections. For example, consider a user who has both a "State" and "City" filter. If a user chooses to highlight relevant values, selecting "California" in the state filter will highlight the set of options available in the "City" filter to only show cities in California, while non-highlighted options will be put under a "Filtered out" menu option in the dropdown. Note that this requires running a query each time a filter is updated.
- Default value: When a "date" type column is selected and a time-binned value is chosen (days, months, or years), users can also choose to set a default date range for the filter. The default filter is automatically applied whenever the query is refreshed.

Limitations

- It is important to note that query filters are applied over the entirety of the dataset. However, the dropdown selector for query filters is limited to 64k unique values. If a user wishes to filter in situations where there are more than 64k unique filter values, it is recommended to use a **Text** parameter instead.

- Filters can only be applied to columns returned by a query, not all columns of a referenced table.
- Filters display the distinct list of options from the designated column in returned results. If the results are limited (i.e. query run with **Limit 1000**), then a filter will only display unique values from within those 1000 results.
- While filters applied to a query will optimize to run on either the client or server side for better performance, filters applied to a dashboard will always run on the server side.

Is this page

as this page helpful?

☐ Yes

☐ No

☐ Send feedback

Last updated on **Apr 29, 2026**

Use named parameter markers

Named parameter markers let you insert variable values into SQL queries at runtime. Instead of hard-coding specific values, you define typed placeholders that users fill in when the query runs. This improves query reuse, prevents SQL injection, and makes it easier to build flexible, interactive queries.

Named parameter markers work in the following Databricks surfaces:

- SQL editor (new and legacy)
- Notebooks
- AI/BI dashboard dataset editor
- Genie Spaces

Add a named parameter marker

Insert a parameter by typing a colon followed by a parameter name, such as `:parameter_name`. When you add a named parameter marker to a query, a widget appears where you can set the parameter type and value. See [Work with parameter widgets](#).

This example converts a hard-coded query to use a named parameter.

Starting query:

SQL

```
SELECT
  trip_distance,
  fare_amount
FROM
  samples.nyctaxi.trips
```



 Ask Genie

WHERE

```
fare_amount < 5
```

1. Delete `5` from the `WHERE` clause.
2. Type `:fare_parameter` in its place. The last line should read `fare_amount < :fare_parameter`.
3. Click the gear icon near the parameter widget.
4. Set the **Type** to **Decimal**.
5. Enter a value in the parameter widget and click **Apply changes**.
6. Click **Save**.

Parameter types

Set the parameter type in the parameter settings panel. The type determines how Databricks interprets and handles the value at runtime.

Type	Description
String	Free-form text. Backslash, single, and double quotation marks are escaped automatically. Databricks adds quotation marks around the value.
Integer	Whole number value.
Decimal	Numeric value that supports fractional values.
Date	Date value. Uses a calendar picker and defaults to the current date.
Timestamp	Date and time value. Uses a calendar picker and defaults to the current date and time.

Named parameter syntax examples

The following examples show common patterns for named parameter markers. Ask Genie

Insert a date

SQL

```
SELECT
  o_orderdate AS Date,
  o_orderpriority AS Priority,
  sum(o_totalprice) AS `Total Price`
FROM
  samples.tpch.orders
WHERE
  o_orderdate > :date_param
GROUP BY 1, 2
```

Insert a number

SQL

```
SELECT
  o_orderdate AS Date,
  o_orderpriority AS Priority,
  o_totalprice AS Price
FROM
  samples.tpch.orders
WHERE
  o_totalprice > :num_param
```

Insert a field name

Use the `IDENTIFIER` function to pass a column name as a parameter. The parameter value should be a column name from the table used in the query.

SQL

```
SELECT * FROM samples.tpch.orders
WHERE IDENTIFIER(:field_param) < 10000
```

Insert database objects

Use the `IDENTIFIER` function with multiple parameters to specify a catalog, schema, and table at runtime.

SQL

```
SELECT *  
FROM IDENTIFIER(:catalog || '.' || :schema || '.' || :table)
```

See [IDENTIFIER clause](#).

Concatenate multiple parameters

Use `format_string` to combine parameters into a single formatted string. See [format_string function](#).

SQL

```
SELECT o_orderkey, o_clerk  
FROM samples.tpch.orders  
WHERE o_clerk LIKE format_string('%s%s', :title, :emp_number)
```

Work with JSON strings

Use the `from_json` function to extract a value from a JSON string using a parameter as the key. Substituting `a` as the value for `:param` returns `1`.

SQL

```
SELECT from_json('{\"a\": 1}', 'map<string, int>') [:param]
```

Create an interval

Use `CAST` to convert a parameter value to an `INTERVAL` type for time-based calculations. See [Interval type](#).

SQL

```
SELECT CAST(:param AS INTERVAL MINUTE)
```

Add a date range using `.min` and `.max`

Date and Timestamp parameters support a range widget. Use `.min` and `.max` to access the start and end of the range.

SQL

```
SELECT * FROM samples.nyctaxi.trips
WHERE tpep_pickup_datetime
BETWEEN :date_range.min AND :date_range.max
```

Set the parameter type to `Date` or `Timestamp` and the widget type to **Range**.

Add a date range using two parameters

SQL

```
SELECT * FROM samples.nyctaxi.trips
WHERE tpep_pickup_datetime
BETWEEN CAST(:date_range_min AS TIMESTAMP) AND CAST(:date_range_max AS
TIMESTAMP)
```

Parameterize rollup granularity

Use `DATE_TRUNC` to aggregate results at a user-selected level of granularity. Pass `DAY`, `MONTH`, or `YEAR` as the parameter value.

SQL

```
SELECT
  DATE_TRUNC(:date_granularity, tpep_pickup_datetime) AS date_rollup,
  COUNT(*) AS total_trips
```

 Ask Genie

```
FROM samples.nyctaxi.trips
GROUP BY date_rollup
```

Pass multiple values as a string

Use `ARRAY_CONTAINS`, `SPLIT`, and `TRANSFORM` to filter on a comma-separated list of values passed as a single string parameter. `SPLIT` parses the comma-separated string into an array. `TRANSFORM` trims whitespace from each element. `ARRAY_CONTAINS` checks whether the table value appears in the resulting array.

SQL

```
SELECT * FROM samples.nyctaxi.trips WHERE
  array_contains(
    TRANSFORM(SPLIT(:list_parameter, ','), s -> TRIM(s)),
    CAST(dropoff_zip AS STRING)
  )
```

NOTE

This example works for string values. To use other data types, wrap the `TRANSFORM` operation with a `CAST` to convert the elements to the desired type.

Syntax migration reference

Use this table when converting queries from mustache syntax to named parameter markers. See [Mustache parameter syntax](#) for more information about the legacy syntax.

Use case	Mustache syntax	Named parameter syntax
Filter by date	<code>WHERE date_field < '{{date_param}}'</code>	<code>WHERE date_field < :date_param</code>
Filter by number	<code>WHERE price < {{max_price}}</code>	<code>WHERE price < :max_price</code>  Ask Genie

Use case	Mustache syntax	Named parameter syntax
Compare strings	<code>WHERE region = '{{region_param}}'</code>	<code>WHERE region = :region_param</code>
Specify a table	<code>SELECT * FROM {{table_name}}</code>	<code>SELECT * FROM IDENTIFIER(:table)</code> — use the full three-level namespace
Specify catalog, schema, and table	<code>SELECT * FROM {{catalog}}. {{schema}}.{{table}}</code>	<code>SELECT * FROM IDENTIFIER(:catalog '.' :schema '.' :table)</code>
Format a string from multiple parameters	<code>"({{area_code}}) {{phone_number}}"</code>	<code>format_string("(%d) %d", :area_code, :phone_number)</code>
Create an interval	<code>SELECT INTERVAL {{p}} MINUTE</code>	<code>SELECT CAST(format_string("INTERVAL '%s' MINUTE", :param) AS INTERVAL MINUTE)</code>

Is this page

as this page helpful?

☐ Yes

☐ No

☐ Send feedback

Last updated on **Apr 29, 2026**

Query snippets

It's often easier to duplicate prior work and modify than to write something from scratch. This is particularly true for common `JOIN` statements or complex `CASE` expressions. As your list of queries grows, it can be difficult to remember which queries contain the statement you need. You can also create a query snippet that contains an insertion point with placeholder text that a user can replace at runtime.

Query snippets are segments of queries that you can share and trigger using autocomplete. Use query snippets for:

- Frequent `JOIN` statements
- Complicated clauses like `WITH` or `CASE`.
- Conditional formatting

Here are examples of snippets:

```
--Simple snippet
WHERE fare_amount > 100

--Snippet with an insertion point for a value to be provided at runtime
WHERE fare_amount > ${1:value}

--Snippet with an insertion point for a value to be provided at runtime
and containing a default value
WHERE fare_amount > ${1:100}

--Snippet with multiple insertion points
WHERE fare_amount > ${2:min_value} AND fare_amount < ${1:max_value} AND
trip_distance < ${0:max_distance}
```

Create query snippets

You can create a query snippet from the SQL, notebook, or file editors. Follow these steps:

1. In the [new SQL editor](#), click the  kebab menu, then click **View**.

In the notebook or file editors, click **View**.

2. Click **Query snippets**.

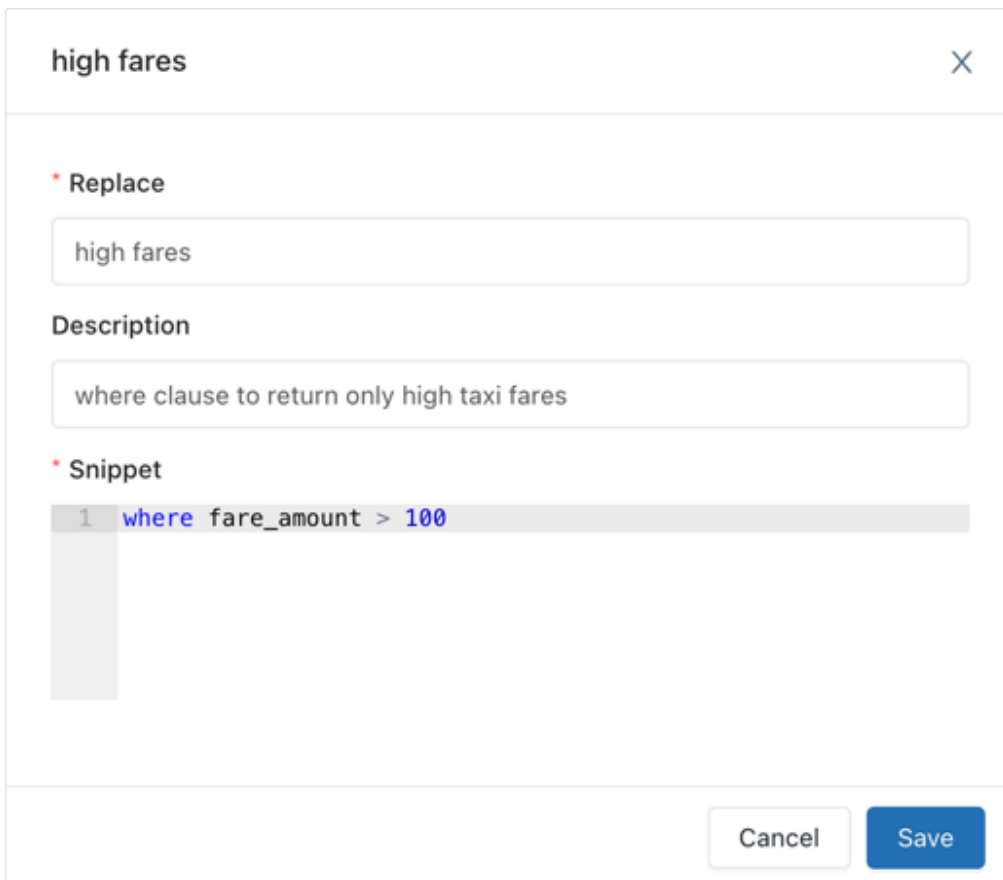
3. Click **Create query snippet**.

4. In the **Replace** field, enter the snippet name. You will use this name when writing a query that uses the snippet.

5. (Optional) Enter a description.

6. In the **Snippet** field, enter the snippet.

7. Click **Create**.



The screenshot shows a modal dialog titled "high fares" with a close button (X) in the top right corner. The dialog contains three main sections:

- * Replace**: A text input field containing the text "high fares".
- Description**: A text input field containing the text "where clause to return only high taxi fares".
- * Snippet**: A code editor area with a line of SQL code: `1 where fare_amount > 100`. The code is highlighted in blue.

At the bottom right of the dialog, there are two buttons: "Cancel" and "Save".

Use a query snippet

Query snippets can be used any SQL query. You can access query snippets in the SQL editor, notebook SQL cells, SQL files, and AI/BI dashboard queries.

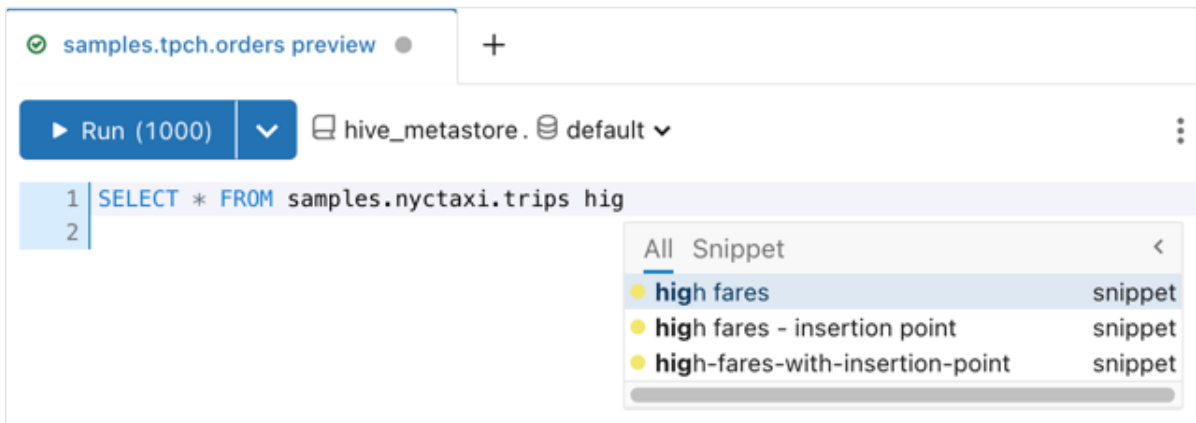
Here's an example of a simple query with which you can use these query snippets:

```
--Simple query  
SELECT * FROM samples.nyctaxi.trips
```

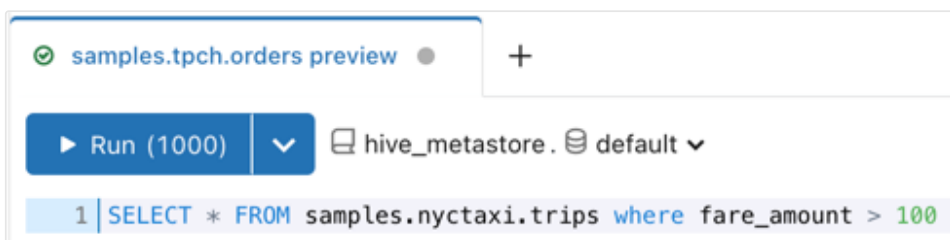
Use the following steps to use a query snippet with this query:

1. Open **SQL Editor**.
2. Type your query in the SQL editor query pane.
3. Start typing the name of your query snippet, then select it from the autocomplete window.

In the SQL editor, you can also manually open the window with **Control+Space** and select a snippet.



4. Execute the query with the **WHERE** clause from the query snippet.



Working with insertion points in query snippets

You designate insertion points by wrapping an integer tab order with a single dollar sign and curly braces `${}`. A text placeholder preceded by a colon `:` is optional but useful for users unfamiliar with your snippet. In the query snippets with insertion points that you created previously, `${1:value}` is an insertion point with a placeholder, and `${1:100}` is an insertion point with a default value for the placeholder that you can override at runtime. When Databricks SQL renders the snippet, the dollar sign `$` and curly braces `{}` are stripped away, and the word `value` or the default of `100` is highlighted for replacement.

When there are multiple insertion points, the text insertion caret jumps to the first insertion point to prompt for the desired value. When you press `Tab`, the caret jumps to the next insertion point for the next value. When you press `Tab` again, the caret will jump to the next insertion point line until it reaches the final insertion point.

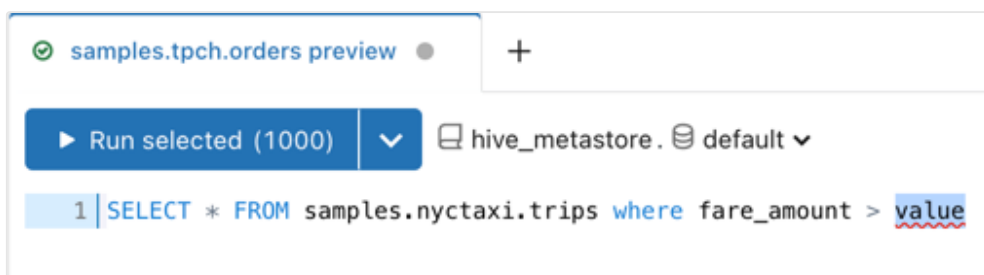
NOTE

An insertion point of zero `${0}` is always the last point in the tab order.

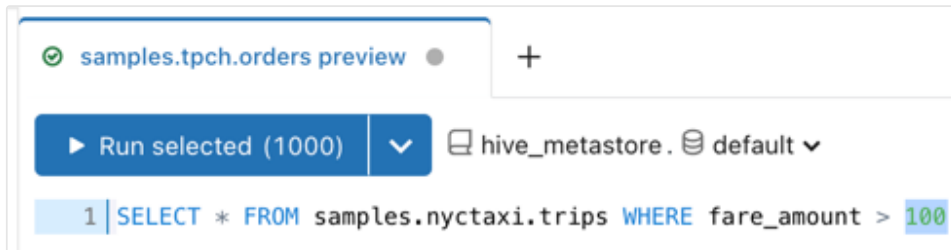
Use the following steps to use the insertion point query snippets with the query:

1. Open **SQL Editor**.
2. Type your query in the SQL editor query pane.
3. Type the first three letters of the name of your query snippet and then select a query snippet with the insertion point without a default value.

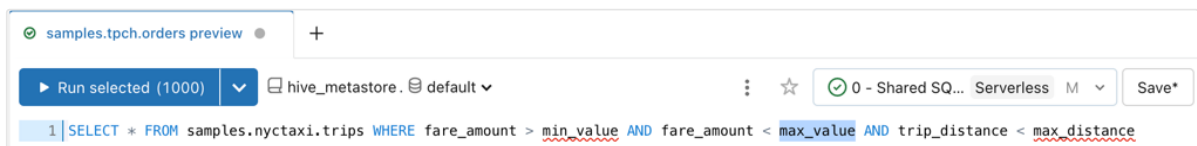
The query snippet is added to the query, and the text insertion caret jumps to the insertion point.



4. Enter a value for the `WHERE` clause, such as `200`.
5. Optionally, run the query with the `WHERE` clause from the query snippet.
6. Repeat the previous steps, but select the query snippet with the insertion point using a default value.



7. Repeat the previous steps, but select the query snippet with multiple insertion points.



8. Enter a value for the first insertion point, tab to the next insertion point, and enter a value, and then tab to the final insertion point and enter a value.

Is this page

as this page helpful?

☐ Yes

☐ No

Send feedback

Last updated on **Apr 29, 2026**

Query profile

You can use a query profile to visualize the details of a query execution. The query profile helps you troubleshoot performance bottlenecks during the query's execution. For example:

- You can visualize each query operator and related metrics, such as the time spent, number of rows processed, rows processed, and memory consumption.
- You can identify the slowest part of a query execution at a glance and assess the impacts of modifications to the query.
- You can discover and fix common mistakes in SQL statements, such as exploding joins or full table scans.

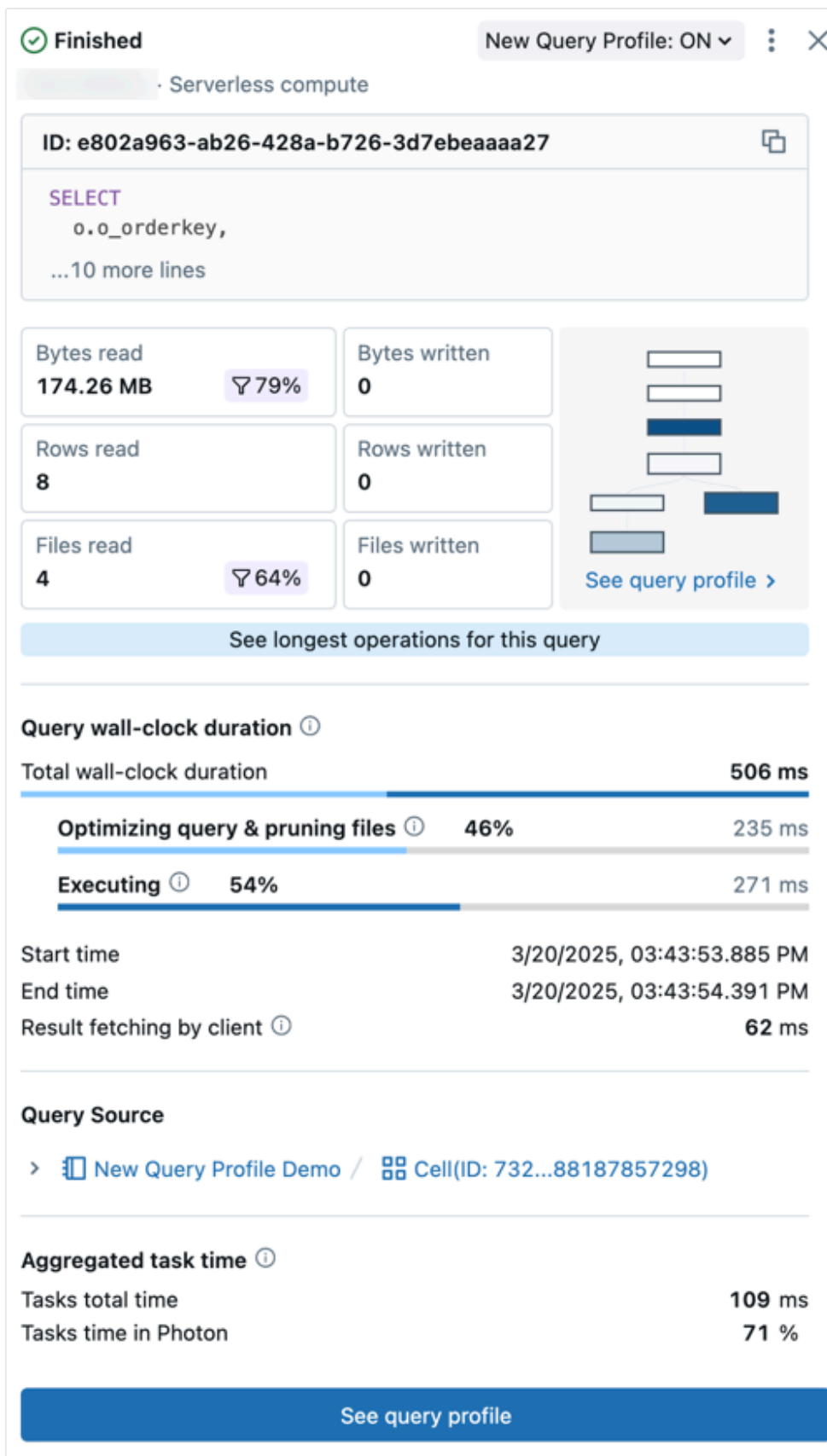
Requirements

To view a query profile, you must either be the owner of the query or you must have at least CAN MONITOR [permission](#) on the SQL warehouse that executed the query.

View a query profile

You can view the query profile from the query history using the following steps:

1. Click  **Query History** in the sidebar.
2. Click the name of a query. A query details panel appears on the right side of the screen.

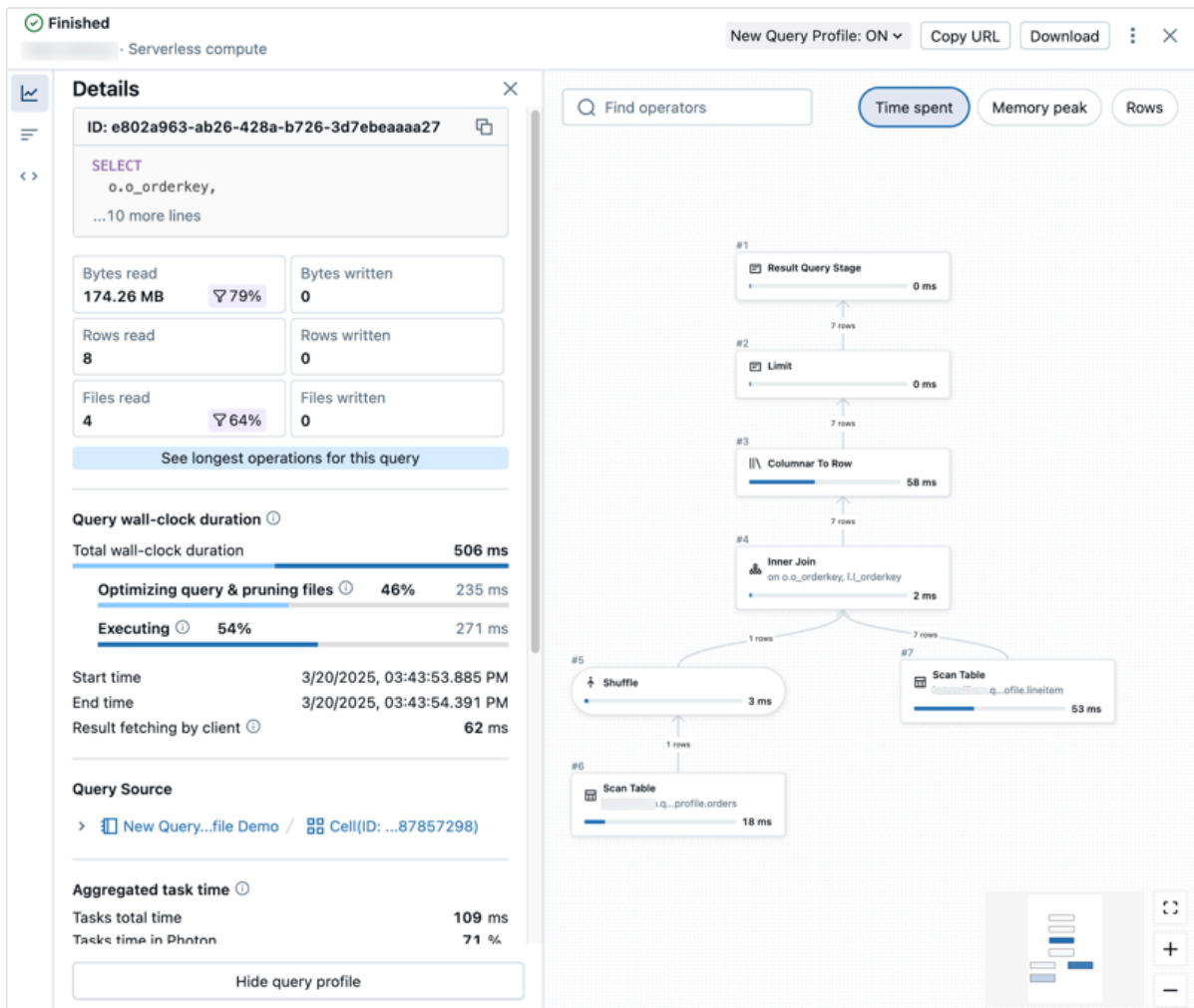


The query summary includes:

- **Query status:** The query is tagged with its current status: **Queued**, **Running**, **Finished**, **Failed**, or **Cancelled**.

- **User and compute details:** See the username, compute type, and runtime details for this query run.
- **ID:** This is the universally unique identifier (UUID) associated with the given query execution.
- **Query statement:** This section includes the complete query statement. If the query is too long to be shown in the preview, click ... **more lines** to view the full text.
- **Query metrics:** Popular metrics for query analysis are shown under the query text. The filter icons that appear with some metrics indicate the percent of data pruned during scanning.
- **See query profile:** A preview of the query profile directed acyclic graph (DAG) is shown in this summary. This can be helpful for quickly estimating query complexity and execution flow. Click **See query profile** to open the detailed DAG.
- **See longest operators for this query:** Click this button to open the **Top operators** panel. This panel shows the longest running operators in the query.
- **Query wall-clock duration:** The total elapsed time between the start of scheduling and the end of the query execution is provided as a summary. A detailed breakdown of scheduling, query optimization and file pruning, and execution time appears below the summary.
- **Query source:** Click on the name of the listed object to go to the query source.
- **Aggregated task time:** View the combined time it took to execute the query across all cores of all nodes. It can be significantly longer than the wall-clock duration if multiple tasks are executed in parallel. It can be shorter than the wall-clock duration if tasks waited for available nodes.
- **Input/Output (IO):** View details about the data read and written during the query execution.

3. Click **See query profile**. A **Details** panel opens on the right side of the screen.



NOTE

If **Query profile is not available** is displayed, no profile is available for this query. A query profile is not available for queries that run from the [query cache](#). To circumvent the query cache, make a trivial change to the query, such as changing or removing the `LIMIT`.

View query profile details

The detailed query profile includes summary metrics on the left side of the panel and a graph view of operators on the right.

Explore query metrics

The left side of the query profile has the following tabs:

1.  **Details:** Opens the **Details** panel which shows [query summary metrics](#).
2.  **Top operators:** Opens the **Top operators** panel which shows the most expensive operators used in your query. This can be useful for identifying optimization opportunities.
3.  **Query text:** Opens the **Query text** panel which shows the full text of the query.

 NOTE

Some non-Photon operations are executed as a group and share common metrics. In this case, all operations have the same value as the parent operator for a given metric.

Explore the DAG

The right half of the query profile shows the directed acyclic graph (DAG) of the query. The graph view shows metrics such as **Time spent**, **Memory peak**, and **Rows**. Click on each metric to change the reporting metric shown.

You can interact with the DAG in the following ways:

- Use the search bar to highlight different operators or columns.
- Zoom in or out focus on different parts of the DAG.
- Click on operators to show detailed metrics and descriptions. A panel on the right side of the graph shows operation details.

For Databricks SQL queries, you can also view the query profile in the Spark UI. Click the  kebab menu near the top of the page, then click **Open in Spark UI**.

By default, metrics for some operations are hidden. These operations are unlikely to be the cause of performance bottlenecks. To see information for all operations, and to see additional metrics, click  at the top of the page, then click **Enable verbose mode**.

Common operations

The most common operations are:

- **Scan:** Data was read from a datasource and output as rows.
- **Join:** Rows from multiple relations were combined (interleaved) into a single set of rows.
- **Union:** Rows from multiple relations that use the same schema were concatenated into a single set of rows.
- **Shuffle:** Data was redistributed or repartitioned. Shuffle operations are expensive with regard to resources because they move data between executors on the cluster.
- **Hash / Sort:** Rows were grouped by a key and evaluated using an aggregate function such as `SUM`, `COUNT`, or `MAX` within each group.
- **Filter:** Input is filtered according to a criteria, such as by a `WHERE` clause, and a subset of rows is returned.

Share a query profile

To share a query profile with another user:

1. View [query history](#).
2. Click the name of the query.
3. To share the query, you have two choices:
 - If the other user has the CAN MANAGE permission on the query, you can share the URL for the query profile with them. Click **Share**. The URL is copied to your clipboard.
 - Otherwise, if the other user does not have the CAN MANAGE permission or is not a member of the workspace, you can download the query profile as a JSON object. **Download**. The JSON file is downloaded to your local system.

Import a query profile

To import the JSON for a query profile:

1. View [query history](#).
2. Click the kebab menu  on the upper right, and select **Import query profile (JSON)**.  Ask Genie

3. In the file browser, select the JSON file that was shared with you and click **Open**. The JSON file is uploaded and the query profile is displayed.

When you import a query profile, it is dynamically loaded into your browser session and does not persist in your workspace. You need to re-import it each time you want to view it.

4. To close the imported query profile, click **X** at the top of the page.

Access the query profile

You can also access the query profile in the following parts of the UI:

- **From the SQL editor:** During and after query execution, a link near the bottom of the page displays the time elapsed and number of rows returned. Click that link to open the **query details** panel. Click **See query profile**.

 3 s 136 ms | 10 rows returned

NOTE

If you have the [new SQL editor](#) enabled (Public Preview), your link appears as it does in a notebook.

- **From a notebook:** If your notebook is attached to a SQL warehouse or serverless compute, you can access the query profile using the link under the cell that contains the query. Click **See performance** to open the run history. Click a statement to open the **query details** panel.

>  See performance (2)

- **From the Lakeflow Spark Declarative Pipelines UI:** You can access the query history and profile from the **Query History** tab in the pipeline UI. See [Access query history for pipelines](#).
- **From the jobs UI:** You can access query profiles for jobs run on SQL warehouses and serverless compute. For jobs run on serverless compute, see [View query details for job runs](#) to learn how to view query details in the jobs UI.  Ask Genie

Next steps

- Learn about accessing query metrics using the [query history API](#)
- Learn more about [query history](#)

Is this page

as this page helpful?

☐ Yes

☐ No

☐ Send feedback

Last updated on **May 11, 2026**

Query performance insights

⚠️ PREVIEW

This feature is in [Private Preview](#). To try it, reach out to your Databricks contact.

When queries run, Databricks might return insights that identify opportunities to improve performance. This page lists the supported insights and their meaning.

For a broader overview of performance best practices, review the [Comprehensive Guide to Optimize Databricks, Spark and Delta Lake Workloads](#).

CONCURRENT_WRITE

- Concurrent writes on the table cause [conflicts](#) that are automatically resolved or fail.
- **Recommendation:** Review the [delta history](#) to identify concurrent writes and consider different scheduling to avoid conflicts.

COVERAGE_FILTER_KEYS_CLUSTERING

- The table is [clustered](#) by one or more keys that aren't used in filtering during the table scan.
- **Recommendation:** Determine which data subset you need for the desired outcome, then add filters on matching clustering keys to reduce bytes read.

COVERAGE_FILTER_KEYS_PARTITIONING

#

 Ask Genie

- The table is [partitioned](#) by one or more keys that aren't used in filtering during the table scan.
- **Recommendation:** Determine which data subset you need for the desired outcome, then add filters on matching partitioning keys to reduce bytes read.

COVERAGE_PHOTON

- [Photon](#) can't accelerate the operation, so the standard runtime engine was used.
- **Recommendation:** Review Photon limitations and consider adjusting the query to use a supported execution strategy for faster runtime.

COVERAGE_STATS_DELTA

- [Delta data skipping](#) statistics are missing or incomplete for the table scan file filters, so the query uses in-file filtering. The following statistics statuses are possible:
 - *Full*: Statistics are available for all filters.
 - *Partial*: Statistics are available on a subset of filters.
 - *Unavailable*: Statistics are not available on any filter.
 - *Unused*: Statistics could not be used on a filter that converts the data type.
- **Recommendation:** [Collect Delta statistics](#) to reduce the number of bytes read.

COVERAGE_STATS_OPTIMIZER

- [Cost-based optimizer](#) statistics are missing or incomplete, so standard heuristics were used to generate the query plan.
- **Recommendation:** [Collect statistics](#) to enable the optimizer to produce a better plan.

DATA_SKEW

- [Data is processed unevenly](#) by available computing resources.

- **Recommendation:** Review the distribution of the data, then salt keys or pre-aggregate the data.

DATA_SPILL

- [Data spill](#) to disk while executing an operator because the data size did not fit in memory.
- **Recommendation:** Increase warehouse [size](#) to increase available memory. Reduce the number of rows, number of columns or size of large column (strings, arrays, maps, structs) to reduce memory usage.

EXCESSIVE_QUEUE_TIME

- Query has been waiting in queue on [warehouse](#).
- **Recommendation:** Increase the maximum [number of clusters](#) on the warehouse to reduce queue time.

EXPLODING_JOIN

- [Join](#) is [generating significantly more rows](#) than it has read.
- **Recommendation:** Determine which result subset is required, then update the join or reduce the number of input rows from both relations.

FLOW_FULL_RECOMPUTE

- [Flow](#) has been planned to be executed as [full recompute](#).
- **Recommendation:** Rewrite the query for [incremental support](#) to reduce the number of bytes read.

IO_THROTTLING

- [Cloud storage](#) request was throttled by your cloud provider.

- **Recommendation:** Contact your administrator to increase your cloud storage request limits with your cloud provider.

REDUNDANT_AGGREGATION

- [Aggregate](#) did not change the query result.
- **Recommendation:** Remove the aggregate or apply [primary and foreign key constraints](#).

SELECTIVE_JOIN

- [Join](#) is generating significantly fewer rows than it has read.
- **Recommendation:** Determine which result subset is required, then add filters before the join to reduce the number of input rows.

WIDE_PROJECTION

- [Projecting](#) all columns on the table.
- **Recommendation:** Determine which result subset is required, then project only those columns to reduce the number of bytes read.

Is this page

as this page helpful?

☐ Yes

☐ No

☐ Send feedback

Last updated on **Apr 29, 2026**

Query optimization using primary key constraints

Primary key constraints, which capture relationships between fields in tables, can help users and tools understand relationships in your data. This article contains examples that show how you can use primary keys with the `RELY` option to optimize some common types of queries.

NOTE

Query optimizations associated with the `RELY` command require queries to run on Photon-enabled compute. See [What is Photon?](#). Photon runs by default on SQL warehouses and serverless compute for notebooks and workflows. To learn more about Photon, see [What is Photon?](#).

Add primary key constraints

You can add a primary key constraint in your table creation statement, as in the following example, or add a constraint to a table using the `ADD CONSTRAINT` clause.

SQL

```
CREATE TABLE customer (  
  c_customer_sk int,  
  PRIMARY KEY (c_customer_sk)  
  ...  
)
```

In this example, `c_customer_sk` is the customer ID key. The primary key constraint specifies that each customer ID value should be unique in the table. Databricks does not enforce key constraints. They can be validated through your existing [Ask Genie](#)

or ETL. See [Manage data quality with pipeline expectations](#) to learn about working expectations on streaming tables and materialized views. See [Constraints on Databricks](#) to learn about working with constraints on Delta tables.

NOTE

It is the user's responsibility to check whether a constraint is satisfied. Relying on a constraint that is not satisfied can lead to incorrect query results.

Use `RELY` to enable optimizations

When you know that a primary key constraint is valid, you can enable optimizations based on the constraint by specifying it with the `RELY` option. See [ADD CONSTRAINT clause](#) for the complete syntax.

The `RELY` option allows Databricks to exploit the constraint to rewrite queries. The following optimizations can only be performed if the `RELY` option is specified in an `ADD CONSTRAINT` clause or `ALTER TABLE` statement.

Using `ALTER TABLE`, you can modify a table's primary key to include the `RELY` option, as shown in the following example.

SQL

```
ALTER TABLE
  customer DROP PRIMARY KEY;
ALTER TABLE
  customer
ADD
  PRIMARY KEY (c_customer_sk) RELY;
```

Optimization examples

The following examples extend the previous example that creates a `customer` table where `c_customer_sk` is a verified unique identifier named as a `PRIMARY KEY` with the `RELY` option specified.

 Ask Genie

Example 1: Eliminate unnecessary aggregations

The following shows a query that applies a `DISTINCT` operation to a primary key.

SQL

```
SELECT
  DISTINCT c_customer_sk
FROM
  customer;
```

Because the `c_customer_sk` column is a verified `PRIMARY KEY` constraint, all values in the column are unique. With the `RELY` option is specified, Databricks can optimize the query by not performing the `DISTINCT` operation.

Example 2: Eliminate unnecessary joins

The following example shows a query where Databricks can eliminate an unnecessary join.

The query joins a fact table, `store_sales` with a dimension table, `customer`. It performs a left outer join, so the query result includes all records from the `store_sales` table and matched records from the `customer` table. If there is no matching record in the `customer` table, the query result shows a `NULL` value for the `c_customer_sk` column.

SQL

```
SELECT
  SUM(ss_quantity)
FROM
  store_sales ss
  LEFT JOIN customer c ON ss.customer_sk = c.c_customer_sk;
```

To understand why this join is unnecessary, consider the query statement. It requires only the `ss_quantity` column from the `store_sales` table. The `customer` table is joined on its primary key, so each row of `store_sales` matches at most one row in `customer`. Because the operation is an outer join, all records from the `store_sales` table are included.

table are preserved, so the join does not change any data from that table. The `SUM` aggregation is the same whether or not these tables are joined.

Using the primary key constraint with `RELY` gives the query optimizer the information it needs to eliminate the join. The optimized query looks more like this:

SQL

```
SELECT
  SUM(ss_quantity)
FROM
  store_sales ss
```

Next steps

See [View the Entity Relationship Diagram](#) to learn how to explore primary key and foreign key relationships in the Catalog Explorer UI.

Is this page

as this page helpful?

☐ Yes

☐ No

Last updated on **Apr 29, 2026**

Use the TPC-DS sample dataset to evaluate system performance

Databricks provides access to the TPC-DS benchmark dataset, a widely used benchmark for testing the performance of systems built for data warehousing and analytics. The dataset is available in two sizes by default in every Unity Catalog-enabled workspace. These datasets are ideal for testing Databricks performance on a standardized benchmark that simulates realistic retail and e-commerce business scenarios. To learn more about this dataset, see the [TPC-DS benchmark](#) documentation.

What's included

The TPC-DS datasets are available in the `samples` catalog with the following schemas:

- `tpcds_sf1` — Small-scale dataset (approximately 1 GB)
- `tpcds_sf1000` — Large-scale dataset (approximately 1 TB)

Both datasets share the following qualities:

- Are read-only and queryable by all users in the workspace
- Are compatible with SQL warehouses and all-purpose clusters
- Follow the TPC-DS specification for standardized benchmarking

Prerequisites

You must have access to a SQL warehouse or an all-purpose cluster.

Preview the data

To explore the data in the Catalog Explorer UI:

1. Click  **Catalog** in the sidebar.
2. Enter **tpcds** in the search bar. Both schemas are in the **samples** catalog. Click the name of the schema that you want to view.
3. The **Overview** tab lists all tables in the schema. Click a table name to open an overview of the columns and data types in that table.
4. Use the top navigation to view the table's **Sample Data** or **Details**.

Query the data

The following queries use the smaller scale dataset, **tpcds_sf1**. To use the larger scale dataset, replace the schema name with **tpcds_sf1000**. Click  **SQL Editor** in the sidebar to open the SQL editor. Then, use the following queries to start exploring the data.

Preview tables

SQL

```
SHOW TABLES IN samples.tpcds_sf1;
```

Explore a table

SQL

```
DESCRIBE TABLE samples.tpcds_sf1.customer;  
SELECT * FROM samples.tpcds_sf1.customer LIMIT 10;
```

Sample join and aggregation

SQL

```
SELECT
  i_category,
  d_year,
  SUM(ss_net_paid) AS total_revenue
FROM samples.tpcds_sf1.store_sales ss
JOIN samples.tpcds_sf1.item i ON ss.ss_item_sk = i.i_item_sk
JOIN samples.tpcds_sf1.date_dim d ON ss.ss_sold_date_sk = d.d_date_sk
WHERE d.d_year = 2001
GROUP BY i_category, d_year
ORDER BY total_revenue DESC
LIMIT 10;
```

Best practices

- Use the [query history](#) and [query profile](#) to understand performance characteristics and identify optimization opportunities.
- Start with the smaller `tpcds_sf1` dataset for initial testing, then scale up to `tpcds_sf1000` for comprehensive performance evaluation.
- Compare query performance across different SQL warehouse sizes to determine optimal configurations for your workloads.
- Use these standardized datasets to establish performance baselines and track improvements over time.

Is this page

as this page helpful?

☒ Yes

☐ No

☐ Send feedback

Last updated on **Apr 29, 2026**

Databricks SQL alerts

This page provides step-by-step guidance for enabling and using Databricks SQL alerts to automate query execution, evaluate custom conditions, and deliver notifications when those conditions are met. With alerts, you can proactively monitor your business data and receive timely notifications whenever reported values fall outside of expected thresholds. When you schedule an alert, its associated query runs and the alert criteria are checked, regardless of any existing schedules on the underlying query. Additionally, you can access an alert history to review the results of past alert evaluations.

To learn how to work with legacy alerts instead, see [What are legacy alerts?](#).

Enable alerts in your workspace

This feature is in [Public Preview](#). Workspace admins can control workspace access to this preview from the [Previews page](#). Enabling this preview doesn't impact your existing alerts; you can continue using both these and [legacy alerts](#) in parallel.

View and organize alerts

Click the  **Alerts** in the sidebar to open the alerts listing page.

Alerts Create alert					
Alerts Legacy alerts					
Filter alerts ☐ Only my alerts Status ▾					
Name ⇅	Status	Schedule	Owner	Created at ⇅	
Frequent restart	Not Run	Every 1 day		May 12, 2025	
OS version compliance check	OK	Every 1 day		May 12, 2025	
Network errors	OK	Every 1 day		May 12, 2025	
Memory usage high	Triggered	Every 1 day		May 12, 2025	
Disk near full	Error	Every 1 day		May 12, 2025	
CPU usage high	Not Run	Every 1 day		May 12, 2025	

- The **Alerts** tab is open by default and shows all alerts in the workspace. You can type to filter by name. Click **Only my alerts** to show only alerts where you are the owner.
- The **Legacy alerts** tab shows alerts that were created using the previous alerts system.
- **Name** shows the string name of each alert.
- **Status** shows whether the alert status is **TRIGGERED**, **OK**, or **ERROR**.
- **Schedule** shows the last updated time or date.
- **Owner** shows the username of the person who owns the alert.
- **Created at** shows the date and time the alert was created.
 - **TRIGGERED** means that on the most recent execution, the **Value** column in your target query met the **Condition** and **Threshold** you configured. If your alert checks whether "cats" is above 1500, your alert will be triggered as long as "cats" is above 1500.
 - **OK** means that on the most recent query execution, the **Value** column did not meet the **Condition** and **Threshold** you configured. This doesn't mean that the **Alert** was not previously triggered. If your "cats" value is now 1470, your alert will show as **OK**.
 - **ERROR** indicates that an error occurred during alert evaluation.

Create an alert

The steps in this section walk you through how to create the alert shown in the following image. Use the labels and explanations to understand each component.

1. **Query editor:** Write and test the query that you want to alert on.

2. **Schedule:** Set a periodic schedule for the alert to run.

3. **Condition:** Select the value threshold that should trigger an alert notification. You can run the query to test the condition from this section.

4. **Notifications:** Indicate which users or notification destinations should be alerted when the threshold value falls outside of the expected range.

5. **Advanced:** Use advanced settings to alert on special values and conditions.

❗ IMPORTANT

During the Public Preview period, alerts do not support queries with [parameters](#). See [What are legacy alerts?](#).

Use the following steps to create an alert:

1. Click  **Alerts** in the sidebar and click **Create Alert**.

2. Copy and paste the following query in the query editor. Click **Run all (1000)**. To learn more about working with the SQL editor, see [Write queries and explore data in the new SQL editor](#).

SQL

```
SELECT
  to_date(tpcp_pickup_datetime) as date,
  SUM(fare_amount) as amount
FROM
  `samples`.`nyctaxi`.`trips`
GROUP BY
  ALL
ORDER BY
  1 DESC
```

3. In the **Condition** field, set the conditions that should trigger the alert.

Condition

Trigger alert when

First row ▾

amount ▾

> ▾

Static Value ▾

4000

Test condition

Apply the following settings as shown:

- Select **Sum** as the data value to check. Alert conditions can be set on the first value of a column in the query result, or you can choose to set an aggregation across all the rows of a single column, such as SUM or AVERAGE.
- Select **amount** as the column to check.
- Select > (greater than) as the logical operator to apply.
- Set the threshold value to **Static value** 4000. **Static value** is selected by default. Enter 4000 into the value field.

4. Click **Test condition** to preview the alert and test whether the alert would trigger with the current data. The provided settings should trigger the alert. You can  Ask Genie

change the threshold value to test further.

5. In the **Notifications** section, search for your user name and click on it when it appears.
6. Click  the calendar icon to open the **Edit schedule** dialog. Use the drop-down menus to set the alert so that it runs **Every 5 minutes** from **0 minutes past the hour**. Click **Save**. Optionally, select the **Show cron syntax** checkbox to edit the schedule in [Quartz Cron Syntax](#).
7. Click **View alert**.

Your alert is saved and runs on the schedule you provided. The data used in this example is static, so if your **Test condition** returned as **Triggered**, it will again the next time it runs. You receive an email notification as an alert. If the **Test condition** returned **OK**, no notification is sent.

Review alert details

A page opens to show your alert details. Additionally, an alert history is shown with evaluation history details. Click the **Run now** button to run the alert query immediately and return the result on this screen.

Click **Edit** in the top right corner of the page to continue editing the alert.

Advanced settings

Click **Advanced settings** to apply the following settings:

- **Notify on ok:** Send a notification when the alert returns as **OK**.
- **Notification frequency:** Periodically send notifications on an alert until the status returns to **OK**.
- **Empty result state:** Set a special status to return when the query returns no results.
- **Template:** You can choose to send notifications using the default template or a custom template. The following list explains each template type:  Ask Genie

- **Use default template:** Alert notification is a message with links to the Alert configuration screen and the Query screen.
- **Use custom template:** Alert notification includes more specific information about the alert.

a. A box displays, consisting of input fields for subject and body. Any static content is valid, and you can incorporate built-in template variables:

- **ALERT_STATUS:** The evaluated alert status (string).
- **ALERT_CONDITION:** The alert condition operator (string).
- **ALERT_THRESHOLD:** The alert threshold (string or number).
- **ALERT_COLUMN:** The alert column name (string).
- **ALERT_NAME:** The alert name (string).
- **ALERT_URL:** The alert page URL (string).
- **QUERY_RESULT_TABLE:** The query result HTML table (string). Results are limited to the first 100 rows. Only email notification destinations can render HTML.
- **QUERY_RESULT_VALUE:** The query result value (string or number).
- **QUERY_RESULT_ROWS:** The query result rows (value array).
- **QUERY_RESULT_COLS:** The query result columns (string array).

An example subject, for instance, could be: `Alert "{{ALERT_NAME}}"`
`changed status to {{ALERT_STATUS}}.`

b. You can use HTML to format messages in a custom template. Only email notification destinations can render HTML. The following tags and attributes are allowed in templates:

- **Allowed tags:** `<a>`, `<b>`, `<blockquote>`, `<br>`, `<code>`, `<del>`, `<div>`, `<em>`, `<h1>`, `<h2>`, `<h3>`, `<h4>`, `<h5>`, `<h6>`, `<hr>`, `<i>`, `<img>`, `<li>`, `<ol>`, `<p>`, `<pre>`, `<span>`, `<table>`, `<thead>`, `<tbody>`, `<td>`, `<th>`, `<tr>`, `<u>`, `<ul>`
- **Allowed attributes:**

Attribute	Supported tags
href	<a>
title	
target	<a>
alt	
src	
start	, <blockquote>, <code>, , , <hr>, <i>, <pre>, <u>

c. Click the **Preview** toggle button to preview the rendered result.

❗ IMPORTANT

The preview is useful for verifying that template variables are rendered correctly. It is not an accurate representation of the eventual notification content, as each notification destination can display notifications differently.

Workspace admins can set up new notification destinations. See [Manage notification destinations](#).

Alert aggregations

An aggregation on an alert works by modifying the original SQL of the Databricks SQL query attached to the alert. The alert wraps the original query text in a common table expression (CTE) and performs a wrapping aggregation query on it to aggregate the query result.

As an example, a `SUM` aggregation on an alert attached to a query with text `SELECT 1 AS column_name` means that whenever the alert is refreshed, the modified SQL that

runs would be: `WITH q AS (SELECT 1 AS column_name) SELECT SUM(column_name) FROM q.`

This means that the original query result (pre-aggregated) cannot be shown in an alert custom body (with parameters such as `QUERY_RESULT_ROWS` and `QUERY_RESULT_COLS`) whenever there is an aggregation on an alert. Instead, those variables will only display the final, post-aggregation query result.

Alert on multiple columns

To set an alert based on multiple columns of a query, your query can implement the alert logic and return a boolean value for the alert to trigger on. For example:

SQL

```
SELECT CASE WHEN drafts_count > 10000 AND archived_count > 5000 THEN 1
ELSE 0 END
FROM (
SELECT sum(CASE WHEN is_archived THEN 1 ELSE 0 END) AS archived_count,
sum(CASE WHEN is_draft THEN 1 ELSE 0 END) AS drafts_count
FROM queries) data
```

This query returns `1` when `drafts_count > 10000` and `archived_count > 5000`. Then you can configure the alert to trigger when the value is `1`.

Alert on metric views

Because metric views are designed to track and measure core business metrics, they pair well with alerts so that you can be automatically notified when a metric falls outside of an expected range. Alerts on metric views work in the same way as an alert on any other dataset. When you write the query to configure the query, use the fully qualified name of the metric view you want to monitor.

How Git integration works with alerts

You can use Databricks Git folders to track and manage changes to alerts. To track alerts with Git, place them in a Databricks Git folder. Newly cloned alerts only appear in the alerts list page or API after a user interacts with them. They have paused schedules and need to be explicitly resumed by users.

Databricks Git folders provide a centralized way to manage common Git operations for alerts and other workspace objects. To learn more, see [Databricks Git folders](#).

Differences from legacy alerts

The latest version of Databricks SQL alerts behaves differently from legacy alerts in a few key ways:

- **Jobs integration:** You can use a SQL alert task to evaluate alerts as part of a Lakeflow Jobs pipeline. This feature is in Beta. To learn more, see [SQL alert task for jobs](#).
- **Query reuse:** An existing saved SQL query cannot be reused when creating an alert. Each alert owns its query definition, which can be authored directly in the new alert editor.
- **Alert status values:** Alert states are simplified and alerts no longer support the `UNKNOWN` status from legacy alerts. Evaluations resolve to `OK`, `TRIGGERED`, or `ERROR`.

You can continue to use both the latest alerts and legacy alerts side by side while you transition.

Configure alert permissions and transfer alert ownership

For alert permission levels, see [Alerts ACLs](#).

1. In the sidebar, click **Alerts**.
2. Click an alert.
3. Click **Share** in the upper-right corner of the alert detail page to open the **Sharing** dialog. Ask Genie

Sharing: My alert

Do you want information about permission levels? [Learn more](#)

Type to add multiple users or groups

Admins

Can Manage (inherited)

Copy link

4. Search for and select the groups and users, and assign the permission level.
5. Click **Add**.

Transfer ownership of an alert

When you save an alert, you become the alert's owner. If an alert's owner is removed from a workspace, the alert no longer has an owner. A workspace admin user can transfer ownership of an alert to a different user. You can also transfer ownership using the [Permissions API](#).

1. As a workspace admin, log in to your Databricks workspace.
2. In the sidebar, click **Alerts**.
3. Click an alert.
4. Click the **Share** button at the top right to open the **Sharing** dialog.
5. Click on the gear icon at the top right and click **Assign new owner**.

Sharing:

Do you want information about permission levels? [Learn more](#)

6. Select the user to assign ownership to.
7. Click **Confirm**.

Is this page

as this page helpful?

☐ Yes

☐ No

☐ Send feedback

Last updated on **Apr 29, 2026**

What are legacy alerts?

Legacy alerts are the first version of Databricks SQL alerts, which function to periodically run queries, evaluate defined conditions, and send notifications if a condition is met. You can set up alerts to monitor your business and send notifications when reported data falls outside of expected limits. Scheduling an alert executes its underlying query and checks the alert criteria. This is independent of any schedule that might exist on the underlying query.

NOTE

The latest version of Databricks SQL alerts unifies all aspects of creating and managing alerts in a single UI. It is in [Public Preview](#). To learn more, see [Databricks SQL alerts](#).

View and organize alerts

Use one of the following options to access alerts:

- Click the  **Workspace** in the sidebar to view alerts in the **Home** folder, where they are stored by default. Users can organize alerts into folders in the workspace browser along with other Databricks objects.
- Click the  **Alerts** in the sidebar to view the alerts listing page.

By default, objects are sorted in reverse chronological order. You can reorder the list by clicking the column headings. Click the **All alerts** tab near the top of the screen to view all alerts in the workspace. Click the **My alerts** tab to view alerts where you are the owner.

- Name** shows the string name of each alert.
- State** shows whether the alert status is `TRIGGERED`, `OK`, or `UNKNOWN`.
- Last Updated** shows the last updated time or date.

 Ask Genie

- **Created at** shows the date and time the alert was created.
 - **TRIGGERED** means that on the most recent execution, the Value column in your target query met the Condition and Threshold you configured. If your alert checks whether “cats” is above 1500, your alert will be triggered as long as “cats” is above 1500.
 - **OK** means that on the most recent query execution, the Value column did not meet the Condition and Threshold you configured. This doesn’t mean that the Alert was not previously triggered. If your “cats” value is now 1470, your alert will show as **OK**.
 - **UNKNOWN** means Databricks does not have enough data to evaluate the alert criteria. You will see this status immediately after creating your Alert and until the query has executed. You will also see this status if there was no data in the query result or if the most recent query result doesn’t include the *Value Column* you configured.

Create an alert

Follow these steps to create an alert on a single column of a query.

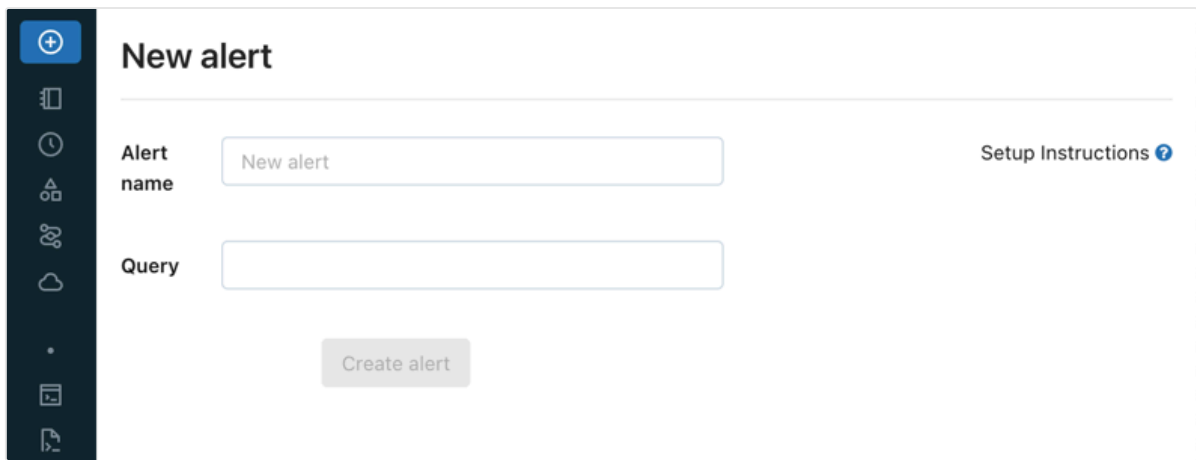
❗ IMPORTANT

- Alerts leveraging queries with [parameters](#) use the default value specified in the SQL editor for each parameter.

1. Do one of the following:

- Click  **New** in the sidebar and select **Alert**.
- Click  **Alerts** in the sidebar and click the **+ New Alert** button.
- Click  **Workspace** in the sidebar and click **+ Create Alert**.

2. In the **Query** field, search for a target query.



New alert

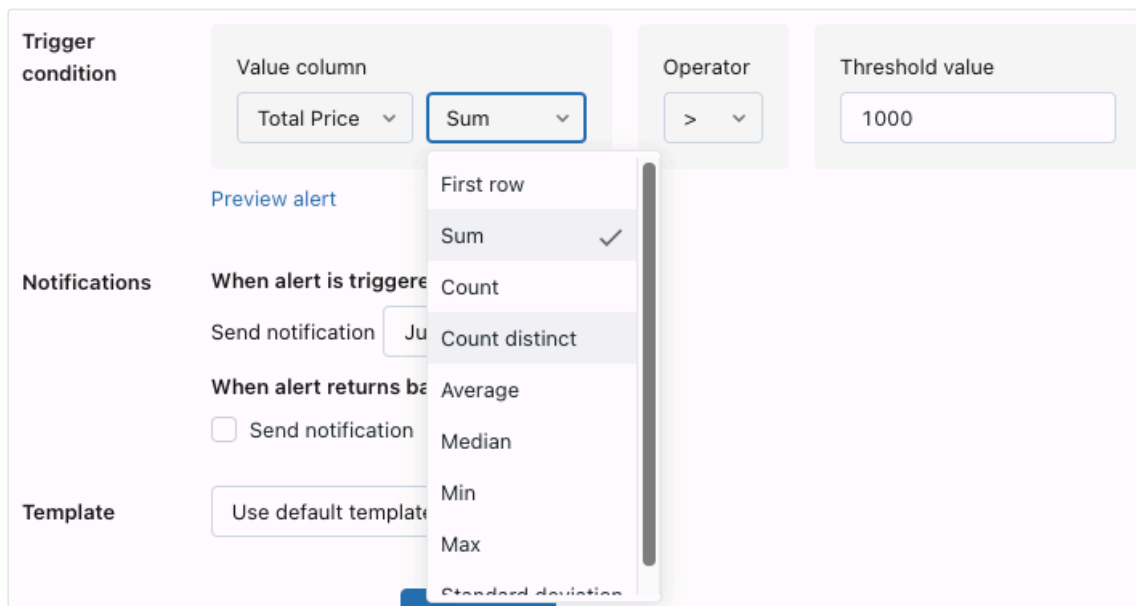
Alert name [Setup Instructions ?](#)

Query

To alert on multiple columns, you need to modify your query. See [Alert aggregations](#).

3. In the **Trigger condition** field, configure the alert.

- The **Value column** drop-down controls which field of your query result is evaluated. Alert conditions can be set on the first value of a column in the query result, or you can choose to set an aggregation across all the rows of a single column, such as SUM or AVERAGE.



Trigger condition

Value column:

Operator:

Threshold value:

[Preview alert](#)

Notifications

When alert is triggered:

Send notification: ☐

When alert returns back: ☐

Template

First row
Sum ✓
Count
Count distinct
Average
Median
Min
Max
Standard deviation

- The **Operator** drop-down controls the logical operation to be applied.
- The **Threshold value** text input is compared against the Value column using the Condition you specify.

Trigger condition	Value column	Operator	Threshold value
	Total Price ▾	Sum ▾	> ▾
			1000

4. Click **Preview alert** to preview the alert and test whether the alert would trigger with the current data.

5. In the **When alert is triggered, send notification** field, select how many notifications are sent when your alert is triggered:

- **Just once:** Send a notification when the **alert status** changes from **OK** to **TRIGGERED**.
- **Each time alert is evaluated:** Send a notification whenever the alert status is **TRIGGERED** regardless of its status at the previous evaluation.
- **At most every:** Send a notification whenever the alert status is **TRIGGERED** at a specific interval. This choice lets you avoid notification spam for alerts that trigger often.

Regardless of which notification setting you choose, you receive a notification whenever the status goes from **OK** to **TRIGGERED** or from **TRIGGERED** to **OK**. The schedule settings affect how many notifications you will receive if the status remains **TRIGGERED** from one execution to the next. For details, see [Notification frequency](#).

6. In the **Template** drop-down, choose a template:

- **Use default template:** Alert notification is a message with links to the Alert configuration screen and the Query screen.
- **Use custom template:** Alert notification includes more specific information about the alert.

a. A box displays, consisting of input fields for subject and body. Any static content is valid, and you can incorporate built-in template variables:

- **ALERT_STATUS:** The evaluated alert status (string).
- **ALERT_CONDITION:** The alert condition operator (string).
- **ALERT_THRESHOLD:** The alert threshold (string or number).
- **ALERT_COLUMN:** The alert column name (string).

- `ALERT_NAME`: The alert name (string).
- `ALERT_URL`: The alert page URL (string).
- `QUERY_NAME`: The associated query name (string).
- `QUERY_URL`: The associated query page URL (string).
- `QUERY_RESULT_TABLE`: The query result HTML table (string). Results are limited to the first 100 rows. Only email notification destinations can render HTML.
- `QUERY_RESULT_VALUE`: The query result value (string or number).
- `QUERY_RESULT_ROWS`: The query result rows (value array).
- `QUERY_RESULT_COLS`: The query result columns (string array).

An example subject, for instance, could be: `Alert "{{ALERT_NAME}}"`
changed status to `{{ALERT_STATUS}}`.

b. You can use HTML to format messages in a custom template. Only email notification destinations can render HTML. The following tags and attributes are allowed in templates:

- Tags: `<a>`, `<abbr>`, `<acronym>`, `<b>`, `<blockquote>`, `<body>`, `<br>`, `<code>`, `<div>`, `<em>`, `<h1>`, `<h2>`, `<h3>`, `<h4>`, `<h5>`, `<h6>`, `<head>`, `<hr>`, `<html>`, `<i>`, `<li>`, `<ol>`, `<p>`, `<span>`, `<strong>`, `<table>`, `<tbody>`, `<td>`, `<th>`, `<tr>`, `<ul>`
- Attributes: href (for `<a>`), title (for `<a>`, `<abbr>`, `<acronym>`)

c. Click the **Preview** toggle button to preview the rendered result.

❗ IMPORTANT

The preview is useful for verifying that template variables are rendered correctly. It is not an accurate representation of the eventual notification content, as each notification destination can display notifications differently.

d. Click the **Save Changes** button.

7. Click **Create Alert**.

8. Click **Add Schedule**.

- Use the dropdown pickers to specify the frequency, period, starting time, and time zone. Optionally, select the **Show cron syntax** checkbox to edit the schedule in [Quartz Cron Syntax](#).
- Choose **More options** to show optional settings. You can also choose:
 - A name for the schedule.
 - A SQL warehouse to power the query. By default, the SQL warehouse used for ad hoc query execution is also used for a scheduled job. Use this optional setting to select a different warehouse to run the scheduled query.

9. Click the **Destinations** tab in the **Add schedule** dialog.

- Use the drop-down to select an available [notification destination](#). Or, start typing a username to add individuals.

❗ IMPORTANT

If you skip this step you *will not* be notified when the alert is triggered.

10. Click **Create**. Your saved alert and notification details appear on the screen.

Workspace > Users > @databricks.com

Demo alert ☆

UNKNOWN

Share [Edit](#) [Refresh](#)

Last triggered at None

Query [Order Revenue](#)

Trigger condition

Value column	Operator	Threshold value
Total Price Sum	>	1000

Notifications

When alert is triggered
Send notification just once, until back to normal.

When alert returns back to normal
☐ Send notification

Template Set to default notification template.

Schedules	Destinations
Demo alert - Every 1 hour Every 1 hour (America/New_York) i@databricks.com No runs Add schedule	

11. Share the schedule.

- To the right of the listed schedule, choose the kebab menu and select **Edit schedule permissions**.
- Choose a user or group from the drop-down menu in the dialog.
- Choose out of the following schedule permissions:
 - NO PERMISSIONS: No permissions have been granted. Users with no permissions cannot see that the schedule exists, even if they are subscribers or included in listed notification destinations.
 - CAN VIEW: Grants permission to view scheduled run results.
 - CAN MANAGE RUN: Grants permission to view scheduled run results.
 - CAN MANAGE: Grants permission to view, modify, and delete schedules. This permission is required in order to make changes to the run interval, update the subscriber list, and pause or unpause the schedule.
 - IS OWNER: Grants all permissions of CAN MANAGE. Additionally, the credentials of the schedule owner will be used to run dashboard queries. Only a workspace admin can change the owner.

❗ IMPORTANT

Permissions for alerts and schedules are separate. Grant access to users and groups in the notifications destinations list so they can view scheduled run results.

12. Share the alert.

- Click [Share](#) near the upper-right corner of the page.

- Add users or groups who should have access to the alert.
- Choose the appropriate permission level, then click **Add**.

❗ IMPORTANT

CAN MANAGE grants permission to view, modify, and delete schedules. This permission is required in order to make changes to the run interval, update the notification destination list, and pause or unpause the schedule.

For more information on alert permission levels, see [Alerts ACLs](#).

Alert aggregations

An aggregation on an alert works by modifying the original SQL in the query attached to the alert. The alert wraps the original query text in a common table expression (CTE) and performs a wrapping aggregation query on it to aggregate the query result.

As an example, a `SUM` aggregation on an alert attached to a query with text `SELECT 1 AS column_name` means that whenever the alert is refreshed, the modified SQL that runs would be: `WITH q AS (SELECT 1 AS column_name) SELECT SUM(column_name) FROM q.`

This means that the original query result (pre-aggregated) cannot be shown in an alert custom body (with parameters such as `QUERY_RESULT_ROWS` and `QUERY_RESULT_COLS`) whenever there is an aggregation on an alert. Instead, those variables will only display the final, post-aggregation query result.

❗ NOTE

All trigger conditions related to aggregations are not supported by the API.

Alert on multiple columns

To set an alert based on multiple columns of a query, your query can implement the alert logic and return a boolean value for the alert to trigger on. For example:

SQL

```
SELECT CASE WHEN drafts_count > 10000 AND archived_count > 5000 THEN 1
ELSE 0 END
FROM (
SELECT sum(CASE WHEN is_archived THEN 1 ELSE 0 END) AS archived_count,
sum(CASE WHEN is_draft THEN 1 ELSE 0 END) AS drafts_count
FROM queries) data
```

This query returns 1 when `drafts_count > 10000` and `archived_count > 5000`. Then you can configure the alert to trigger when the value is 1.

Alert on metric views

Because metric views are designed to track and measure core business metrics, they couple well with alerts so that you can be automatically notified when a metric falls outside of an expected range. To set an alert on a metric view, you must first write and save a query on the metric view. See [Query a metric view](#). When you set an alert, choose the saved query on your metric view as the target query. For details instructions about creating an alert, see [Create an alert](#).

Notification frequency

Databricks sends notifications to your chosen notification destinations whenever it detects that the Alert status has changed from OK to TRIGGERED or vice versa. Consider this example where an Alert is configured on a query that is scheduled to run once daily. The daily status of the Alert appears in the following table. Prior to Monday the alert status was OK.

Day	Alert Status
Monday	OK
Tuesday	OK
Wednesday	TRIGGERED  Ask Genie

Day	Alert Status
Thursday	TRIGGERED
Friday	TRIGGERED
Saturday	TRIGGERED
Sunday	OK

If the notification frequency is set to `Just Once`, Databricks sends a notification on Wednesday when the status changed from `OK` to `TRIGGERED` and again on Sunday when it switches back. It does not send alerts on Thursday, Friday, or Saturday unless you specifically configure it to do so because the Alert status did not change between executions on those days.

Configure alert permissions and transfer alert ownership

You must have at least CAN MANAGE permission on a query to share queries. For alert permission levels, see [Alerts ACLs](#).

1. In the sidebar, click **Alerts**.
2. Click an alert.
3. Click the Share button at the top right to open the **Sharing** dialog.

Sharing: My alert

Do you want information about permission levels? [Learn more](#)

Type to add multiple users or groups

Admins

Can Manage (inherited)

Copy link

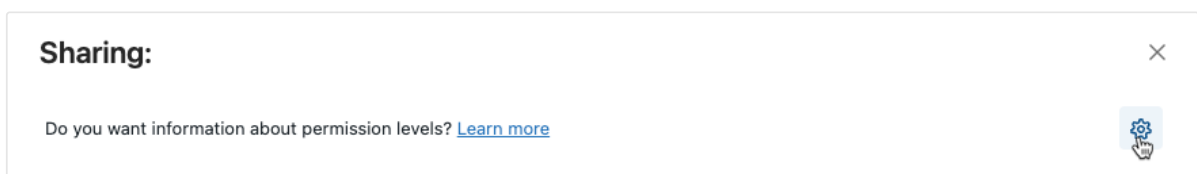
Ask Genie

4. Search for and select the groups and users, and assign the permission level.
5. Click **Add**.

Transfer ownership of an alert

When you save an alert, you become the alert's owner. If an alert's owner is removed from a workspace, the alert no longer has an owner. A workspace admin user can transfer ownership of an alert to a different user. Service principals and groups cannot be assigned ownership of a alert. You can also transfer ownership using the [Permissions API](#).

1. As a workspace admin, log in to your Databricks workspace.
2. In the sidebar, click **Alerts**.
3. Click an alert.
4. Click the **Share** button at the top right to open the **Sharing** dialog.
5. Click on the gear icon at the top right and click **Assign new owner**.



6. Select the user to assign ownership to.
7. Click **Confirm**.

Is this page

Was this page helpful?

Yes

No

Ask Genie

Last updated on **Apr 29, 2026**

Update to the latest Databricks SQL API version

This page describes changes to the Queries, Alerts, Permissions, Data Sources, and Visualizations APIs included in the latest version of the Databricks SQL API. The legacy API is deprecated and support will end soon. Use this page to migrate your applications and integrations to the new API version.

Changes to the Queries API

The new Queries API includes a more user-friendly experience with more descriptive names, paginated responses, and list responses automatically sorted by creation time. The following list describes the changes to the Queries API:

- The API path is now `api/2.0/sql/queries`, replacing the legacy path of `/api/2.0/preview/sql/queries`.
- Includes a new query definition with more descriptive field names and types.
- The update endpoint now supports partial updates by using `PATCH` instead of `POST`.
- The update endpoint now supports transferring the ownership of queries. Previously, this was only allowed using the [transfer object ownership API](#).
- The list endpoint responses are now paginated using token-based pagination.
- The list endpoint no longer supports filtering by name or custom ordering. Instead, all accessible queries are returned and sorted in ascending order by their creation time.
- The restore endpoint is no longer supported. Trashed queries can continue to be restored through the Databricks UI.

For complete documentation on the updated Queries API, see [Queries](#).

 Ask Genie

Changes to the Alerts API

The new Alerts API includes a more user-friendly experience with more descriptive field names and types, paginated responses to list endpoints, and support for partial updates. The following list describes the changes to the Alerts API:

- The API path is now `api/2.0/sql/alerts`, replacing the legacy path of `/api/2.0/preview/sql/alerts`.
- Includes a new alert definition with more descriptive field names and types is used.
- The update endpoint now supports partial updates by using `PATCH` instead of `POST`.
- The update endpoint now supports transferring the ownership of queries. Previously, this was only allowed using the [transfer object ownership API](#).
- The list endpoint responses are now paginated using token-based pagination.
- The delete endpoint now moves the alert to the trash instead of permanently deleting the alert. Trashed alerts will be automatically cleaned up after 30 days. Trashed alerts can be restored within 30 days of deletion through the Databricks UI.

For complete documentation on the updated Alerts API, see [Alerts](#).

Permissions API deprecated

The [Permissions](#) API is now marked as deprecated. Use the [Workspace API](#) to manage object permissions. Use the queries and alerts update APIs to transfer object ownership.

Data Sources API deprecated

The [Data Sources](#) API is now marked as deprecated. Currently, its functionality is limited to getting a list of SQL warehouses. Because the new Queries API supports passing in and returning SQL warehouse IDs instead of data source IDs, a separate API call to convert between data sources and SQL warehouses is no longer required.

Upgrading SDK versions

If you're accessing the legacy Databricks SQL API through Databricks SDKs, upgrade to the latest SDK version and migrate to the new APIs. See [Databricks SDKs](#) for SDK documentation.

Migrating CLI commands

If you're accessing the legacy Databricks SQL API through the [Databricks CLI](#), migrate from these deprecated command groups:

Deprecated	Use Instead
queries-legacy	queries
alerts-legacy	alerts
data-sources	No replacement

Migrating Terraform resources

If you use Terraform to manage your Databricks SQL resources, migrate to the new resource definitions:

Deprecated Resource	Use Instead	Migration Guide
databricks_sql_query	databricks_query	Migration guide
databricks_sql_alert	databricks_alert	Migration guide
databricks_sql_visualization	databricks_dashboard	No migration guide

For [databricks_sql_endpoint](#) and [databricks_sql_warehouse](#):

Upgrade to Databricks Terraform provider v1.105.0 or later. The `data_source_id` field is deprecated. Upgrade to prevent errors when the deprecated data sources API is turned off.

Is this page

as this page helpful?

☐ Yes	☐ No
Send feedback	